

Unity 6000.5 · Entities 6.5

Introducción a Unity DOTS

Entities, Netcode, flujos de trabajo de ECS, jobs, depuración,
optimización y migración.



Youngmin Cho (SillyToolValley)

Edición en español · 2026

Introducción a Unity DOTS

Una guía práctica de Entities 6.5, Netcode for Entities, flujos de trabajo de ECS, optimización y migración.

Edición en español generada a partir del repositorio Unity DOTS Introduction el 2026-05-21.

Autor: Youngmin Cho (SillyToolValley) / UCI (Unity Certified Instructor)

Unity, Entities, Netcode for Entities y los nombres de paquetes relacionados son marcas comerciales o nombres de productos de Unity Technologies. Este libro es un resumen de aprendizaje comunitario y no es documentación oficial de Unity.

Se permite la lectura y la cita públicas. No se permite el scraping automatizado, la inclusión en conjuntos de datos, el entrenamiento de IA ni la ingesta en respuestas de IA sin permiso por escrito.

Introducción a Unity DOTS

Una guía de campo para Entities 6.5, Netcode for Entities, flujos de trabajo ECS, optimización y migración.

Este libro reúne el repositorio de buenas prácticas de Unity DOTS en un único hilo de lectura. El enfoque es siempre práctico: configuración, flujos de trabajo con SubScene y Baker, patrones de componentes y sistemas ECS, planificación de jobs, seguridad ante cambios estructurales, uso de EntityCommandBuffer, Netcode for Entities, optimización, depuración, migración y referencias al glosario.

El stack de referencia es Unity 6000.5+, Entities 6.5.0, Netcode for Entities 6.5.0 y los paquetes core de DOTS que vienen con el editor.

Cómo leer este libro

- Los usuarios nuevos de DOTS deben leer la Parte I y luego los Capítulos 4-15 de la Parte II antes de pasar a Netcode.
- Quien ya trabaje con Entities 1.x puede hojear los capítulos básicos y saltar directamente a la Parte IV para las notas de migración.
- El trabajo de rendimiento conviene abordarlo después de los capítulos del flujo de trabajo ECS, ya que la organización de los chunks y el perfilado se entienden mejor cuando el modelo de datos resulta familiar.

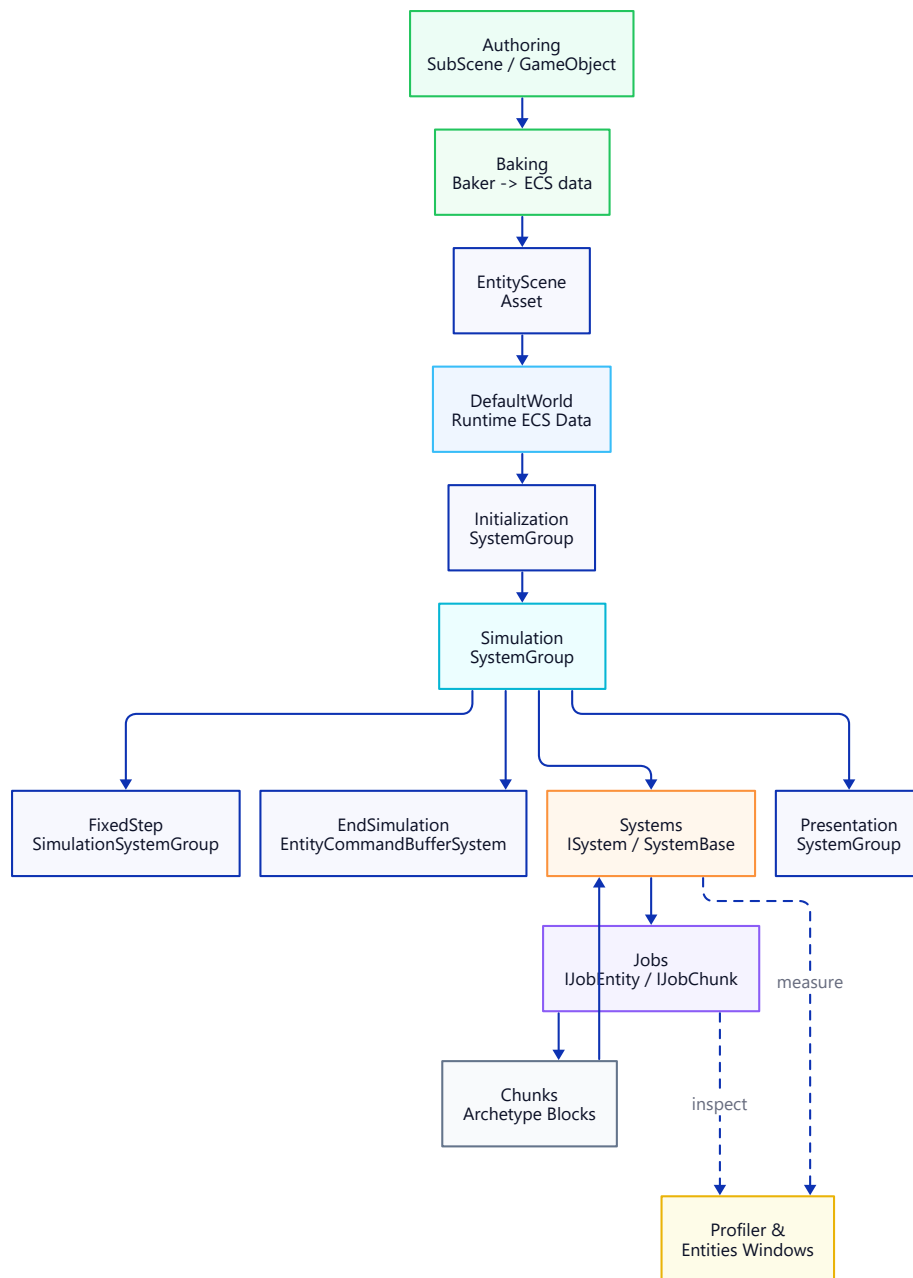


Figure: Unity DOTS Architecture Flow.

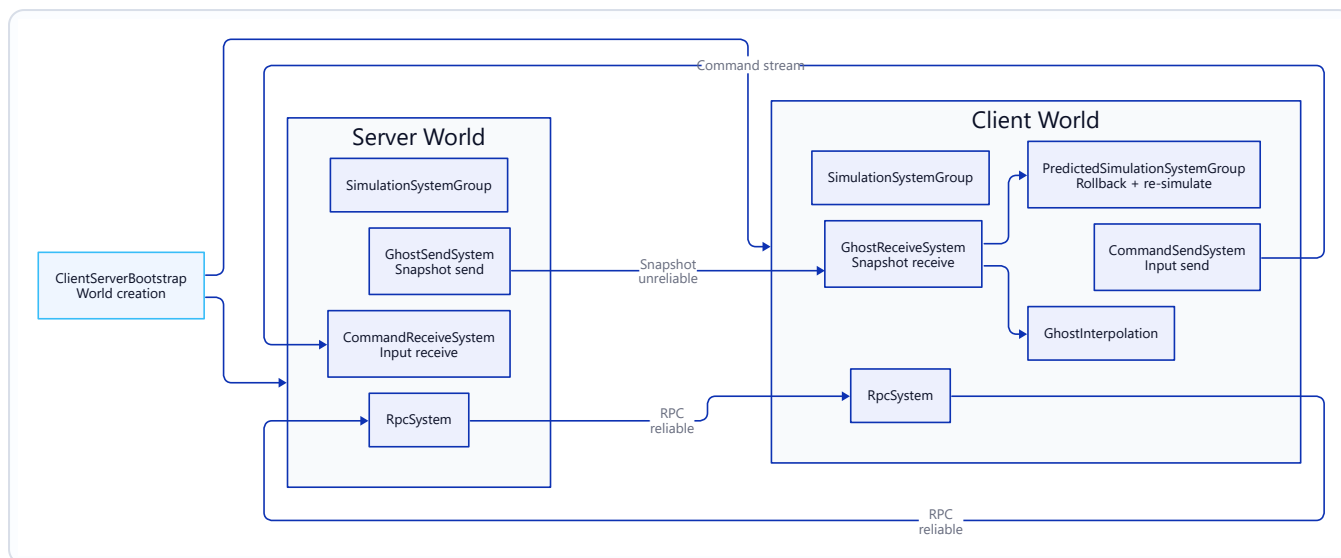


Figure: Netcode for Entities Architecture Flow.

Tabla de contenidos

Sección	Tema
Parte I. Primeros pasos	
Capítulo 1	Configuración del entorno — Unity 6000.5 + Entities 6.5.0
Capítulo 2	Los paquetes core explicados
Capítulo 3	Hola DOTS — Primera Entity
Parte II. Flujos de trabajo DOTS	
Capítulo 4	Patrón Baker & SubScene
Capítulo 5	Ejemplo de Spawner
Capítulo 6	Conceptos fundamentales de ECS
Capítulo 7	Referencias de Entity — Entity · EntityPrefabReference · UnityObjectRef
Capítulo 8	Tipos de Component
Capítulo 9	Enableable Component
Capítulo 10	Singleton Component
Capítulo 11	System — ISystem vs SystemBase
Capítulo 12	System Group & orden de actualización
Capítulo 13	JobSystem & Burst
Capítulo 14	IJobEntity · SystemAPI.Query

Sección	Tema
Capítulo 15	IJobEntity vs IJobChunk
Capítulo 16	Cambio estructural y seguridad
Capítulo 17	EntityCommandBuffer · Entidad diferida
Capítulo 18	ParallelWriter · Reproducción determinista
Capítulo 19	World cliente-servidor de Netcode y Bootstrap
Capítulo 20	Conexión de red y aprobación en Netcode
Capítulo 21	Snapshot de Ghost y sincronización en Netcode
Capítulo 22	Predicción y rollback en Netcode
Capítulo 23	Flujo de comandos e input en Netcode
Capítulo 24	RPC de Netcode
Capítulo 25	Optimización de Ghost, importancia y relevancia en Netcode
Capítulo 26	Integración de física y compensación de lag en Netcode
Capítulo 27	Profiler y depuración de Netcode
Parte III. Optimización y depuración	
Capítulo 28	Disposición de chunks y TypeManager
Capítulo 29	Systems · Entity Inspector · Query Window
Capítulo 30	Profiler · Análisis de cuellos de botella
Capítulo 31	Auditoría de referencias a objetos administrados
Parte IV. Migración	
Capítulo 32	Entities 1.x → 6.5 Resumen general
Capítulo 33	Package Manager → Paquete core
Capítulo 34	Referencias a objetos administrados → UnityObjectRef
Capítulo 35	Migración de foreach → IJobEntity
Capítulo 36	Deprecación de IAspect — migrar fuera de los Aspects
Apéndice. Registros de cambios	
Apéndice A	Cambios clave de Entities 1.4 → 6.5
Apéndice B	Cambios clave de Netcode for Entities 1.4 → 6.5
Referencia. Glosario	
Glosario	Glosario

Parte I. Primeros pasos

Capítulo 1. Configuración del entorno — Unity 6000.5 + Entities 6.5.0

1. Introducción

Desde **Unity 6000.4+**, Entities es un **paquete core**: viene *con* el Editor y su versión queda fijada a la versión del Editor, de modo que ya no se elige a mano una versión de Entities. Eso sí, **no** se añade automáticamente a los proyectos nuevos: hay que añadirlo una vez por proyecto desde el Package Manager (Unity Registry). Esta guía explica cómo instalar el Editor, crear un proyecto, añadir Entities y comprobar que está activo.

Entorno de referencia del proyecto: Unity 6000.5.0b2 · Entities 6.5.0 · Collections / Mathematics / Entities Graphics (todos ellos Core Packages).

2. Instalar Unity 6000.5+

1. Abrir **Unity Hub**. Si aún no se tiene, instalar primero Unity Hub (<https://unity.com/download>).
2. En Hub, ir a **Installs** → **Install Editor**.
3. Elegir cualquier versión **6000.5.x** (beta de la serie b o posterior). Entities 6.5 solo existe en 6000.5+.
4. Módulos: dejar los valores por defecto. ECS no necesita ningún módulo adicional.

No hace falta marcar ningún módulo "DOTS" al instalar el Editor: el paquete Entities viene con el Editor desde la versión 6000.4. Aun así, hay que añadirlo a cada proyecto más adelante (ver los siguientes pasos).

3. Crear un nuevo proyecto

Desde **Hub** → **Projects** → **New project**:

Campo	Valor
Editor Version	6000.5.x
Template	Universal 3D (o 3D (Built-in) , HDRP — cualquiera sirve)
Project Name	cualquier nombre

Hacer clic en **Create project**. El Editor se abre con una escena vacía.

La antigua plantilla específica para DOTS ya no es necesaria en 6000.4+. Cualquier plantilla sirve como base para un flujo de trabajo con Entities.

4. Añadir el paquete Entities y comprobar que está activo

Abrir **Window** → **Package Manager**, cambiar el filtro a **Unity Registry**, seleccionar **Entities** y hacer clic en **Install** (o usar + → **Add package by name...** e introducir `com.unity.entities`). Como Entities es un paquete core, no se elige una versión: el Editor instala la que trae integrada (Entities 6.5 en Unity 6000.5). Al instalar Entities también se incorporan **Collections**, **Mathematics** y **Entities Graphics** como dependencias.

Ahora cambiar el filtro a **In Project** y confirmar que aparecen **Entities**, **Collections**, **Mathematics** y **Entities Graphics**. Para comprobarlo desde los archivos, abrir `Packages/manifest.json`: ahora figura una entrada `com.unity.entities` (las dependencias que arrastra se resuelven automáticamente y puede que no tengan cada una su propia entrada).

A continuación, abrir las ventanas de ECS:

- **Window** → **Entities** → **Hierarchy** — muestra las entidades en tiempo de ejecución.
- **Window** → **Entities** → **Systems** — muestra los sistemas registrados.
- **Window** → **Entities** → **Archetypes** — muestra la organización de los chunks.

Si los tres menús existen, Entities está correctamente configurado.

5. Prueba de humo — crear una SubScene

1. En la Hierarchy, clic derecho → **New Sub Scene** → **Empty Scene...**
2. Nombrarla `MainSubScene.unity` y guardar.
3. Dentro de la nueva SubScene, añadir cualquier primitiva (**3D Object** → **Cube**).
4. Guardar la escena. El Cube se **hornea** como entidad en cuanto la SubScene se cierra (ícono amarillo) o al compilar.
5. Pulsar **Play** y abrir **Window** → **Entities** → **Hierarchy**: debe aparecer una entidad Cube. La prueba de humo se supera.

La SubScene es el punto de entrada del authoring: sus GameObjects se convierten en Entities al guardar y al compilar. El resto del tutorial, en `03_Hello DOTs - First Entity.md`, parte de aquí.

6. Solución de problemas

Síntoma	Causa / Solución
El menú Window → Entities no aparece	Entities aún no está instalado (añadirlo desde Package Manager → Unity Registry → Entities) o el Editor es anterior a 6000.4 (actualizarlo).
No se encuentra <code>Baker<T> / IComponentData</code>	El archivo de definición de ensamblado no incluye <code>Unity.Entities</code> . Añadirlo como referencia o, para una prueba rápida, eliminar el <code>asmdef</code> .
La SubScene sigue siendo un <code>GameObject</code> y nunca se hornea	La SubScene debe estar cerrada (ícono amarillo) para que aparezcan las entidades horneadas. Se abre y se cierra con la casilla situada junto a su nombre en la Hierarchy.
La ventana Entities Hierarchy no muestra nada en modo Play	Comprobar que la SubScene forma parte de la escena actual y que el desplegable "World" de Window → Entities → Hierarchy apunta a <code>DefaultGameObjectInjectionWorld</code> .
Error de compilación: <code>com.unity.entities not found</code>	El paquete no está instalado o <code>manifest.json</code> fija una versión 1.x que no existe en este Editor. Instalar Entities desde el Package Manager para que la entrada se resuelva a la versión Core del Editor. Ver <code>Migration/02_Package Manager</code> → <code>Core Package.md</code> .

Capítulo 2. Los paquetes core explicados

1. Introducción

Con la llegada del **paquete Entities 6.4.0** (que acompaña a Unity Editor 6000.4), los principales paquetes DOTS pasaron a ser **paquetes core**: ahora vienen *con* el Editor (se instalan desde el propio paquete del Editor en lugar de descargarse de un registro remoto) y su versión queda ligada al Editor. El registro de cambios del paquete lo expresa así:

"Package is now a core package embedded in Unity."

El cambio alcanzó a `com.unity.entities`, `com.unity.collections`, `com.unity.mathematics`, `com.unity.entities.graphics` y, en su versión 6.5 equivalente, a Netcode for Entities. Modifica cómo se instalan, se versionan y se gestionan, pero se trata de un cambio de **distribución**, no de API.

Esta página explica:

- Qué paquetes son ahora paquetes core (y cuáles no).
- De dónde salen sus números de versión.
- Qué cambia en `manifest.json` y en `packages-lock.json`.
- Por qué se hizo este cambio.

2. ¿Qué es un paquete core?

Un **paquete core** es un paquete que viene **con el Editor**. Se sigue añadiendo a un proyecto desde el Package Manager como cualquier otro paquete, pero se instala desde el propio paquete del Editor en lugar de un registro remoto, su versión no se puede cambiar al margen del Editor y su ritmo de publicación es el del propio Editor. (Esto *no* es lo mismo que un **paquete Built-in**: un módulo del motor como Physics que solo se activa o se desactiva.)

Característica	Paquete de Package Manager	Paquete core
Paso de instalación	Añadir mediante UPM	Añadir mediante el Package Manager (se instala desde el propio paquete del Editor)

Característica	Paquete de Package Manager	Paquete core
Versión elegida por	El usuario, en manifest.json	Unity, por versión del Editor
Cadencia de versiones	Independiente de UPM	Versiones del Editor
Aparece en manifest.json	Sí	Sí — se añade al instalarlo
Aparece en packages-lock.json	Sí	Sí, marcado como builtin
Se muestra en Package Manager	Bajo "In Project"	Bajo "Unity Registry" para instalarlo y luego "In Project"

3. El mapa de paquetes DOTS en Unity 6000.5+

Paquete	En Unity 6000.5+	Propósito
Entities	Paquete core (6.5)	Runtime de ECS: Entity, Component, System, World
Collections	Paquete core	Tipos de contenedor nativos (NativeList<T>, NativeHashMap<K,V>, etc.)
Mathematics	Paquete core	Tipos optimizados con Burst: float3, quaternion, int4, etc.
Entities Graphics	Paquete core	Renderiza gráficos basados en entidades mediante el SRP
Netcode for Entities	Paquete core (6.5)	Redes cliente-servidor para entidades
Unity Physics	Package Manager	Física sin estado y determinista para entidades
Character Controller	Package Manager	Controlador de personaje basado en ECS
Burst Compiler	Función integrada del motor	Compila jobs y sistemas a código nativo muy optimizado
Job System	Función integrada del motor	Planificación multihilo de jobs

Nota: Job System y Burst nunca fueron paquetes de UPM; forman parte del motor. El cambio de la 6000.4 afecta en concreto a los *paquetes DOTS que se apoyan en ellos*.

4. Números de versión tras la transición

Las versiones de los paquetes core **se alinean con el Editor**:

Editor	Entities core
Unity 6000.4	Entities 6.4
Unity 6000.5	Entities 6.5

La antigua línea 1.x sigue disponible para los proyectos que todavía no pueden migrar a 6000.4+. Ambas líneas son independientes: para pasar a la 6.x basta con actualizar el Editor.

Las notas de transición de Entities están en `Changelog/Entities 1.4 → 6.5 Key Changes.md` y los cambios propios de Netcode en `Changelog/Netcode for Entities 1.4 → 6.5 Key Changes.md`.

5. Efecto en los archivos del proyecto

`manifest.json`

Al añadir Entities, aquí aparece una entrada `com.unity.entities`. Los paquetes que arrastra (`com.unity.collections`, `com.unity.mathematics`, `com.unity.entities.graphics`) se resuelven como dependencias y normalmente no necesitan su propia entrada. Al actualizar un proyecto 1.x, reemplazar las versiones antiguas fijadas y dejar que el Package Manager las resuelva a las versiones Core del Editor; ver `Migration/02_Package Manager → Core Package.md`.

`packages-lock.json`

Los paquetes core aparecen con `"source": "builtin"` en lugar de una URL de versión.

Definiciones de ensamblado (`.asmdef`)

Los archivos `.asmdef` siguen referenciando los mismos nombres de ensamblado (por ejemplo, `Unity.Entities` o `Unity.Collections`). Aquí no cambia nada: lo único que cambió es la capa de distribución del paquete.

6. ¿Por qué el cambio?

El objetivo, según Unity, es que el equipo de ECS publique novedades de forma más rápida y gradual al ligar las actualizaciones de DOTS a la versión del Editor en vez de a un calendario de paquetes aparte. Además, permite que el motor dependa internamente de ECS, algo incómodo mientras los paquetes eran opcionales.

7. Solución de problemas

Síntoma	Causa / Solución
El Package Manager solo ofrece Entities en una versión 1.x antigua	El Editor en uso es anterior a 6000.4. Actualizar el Editor o seguir en la línea 1.x.
Error al resolver manifest.json después de cambiar la versión de com.unity.entities	Borrar packages-lock.json y dejar que el Editor lo regenere.
Un sample o un paquete del Asset Store apunta a com.unity.entities@1.x	En 6000.5+ la API 1.x está prácticamente sustituida; consultar la carpeta Migration para ver los equivalentes.
Dos instalaciones del Editor indican versiones distintas de Entities	Es lo esperado: la versión depende de cada instalación del Editor.

Capítulo 3. Hola DOTS — Primera Entity

1. Qué se va a construir

Un cubo que gira sobre su eje Y, movido por un sistema ECS. En una sola sesión se recorren todos los conceptos clave: **SubScene**, **authoring** → **baker**, **IComponentData**, **ISystem** y las ventanas **Entities Hierarchy / Systems**. Al final se sustituye el sistema del hilo principal por un **IJobEntity** para ver cómo entra en juego el paralelismo.

Requisito previo: [01_Environment Setup.md](01_Environment Setup (Unity 6000.5 + Entities 6.5.0).md) — un proyecto Unity 6000.5 con una SubScene comprobada y el menú Entities.

2. Crear la SubScene

1. Abrir la escena vacía.
2. En la **Hierarchy**, clic derecho → **New Sub Scene** → **Empty Scene...**
3. Guardarla como `HelloDots.unity` dentro de `Assets/Scenes/`.
4. Dentro de la nueva SubScene, clic derecho → **3D Object** → **Cube**. Renombrarlo como `RotatingCube`.
5. Guardar la escena (`Ctrl/⌘ + S`) y cerrar la SubScene con la casilla situada junto a su nombre en la Hierarchy: el ícono se pone amarillo, señal de que su contenido se está horneando como entidades.

A partir de aquí, cualquier cambio en los componentes de `RotatingCube` provoca un rehorneado automático al volver a abrir y cerrar la SubScene.

3. Definir el componente — `RotationSpeed`

Crear `Assets/Scripts/RotationSpeed.cs`:

```
using Unity.Entities;

public struct RotationSpeed : IComponentData
```

```
{  
    public float RadiansPerSecond;  
}
```

Notas:

- IComponentData sobre un struct **no administrado** es la opción por defecto: reside directamente en los chunks y es compatible con Burst.
- Conviene que los campos sean blittables (tipos primitivos, tipos de `Unity.Mathematics` u otros structs no administrados).

4. Definir el authoring + baker

Crear Assets/Scripts/RotationSpeedAuthoring.cs:

```
using Unity.Entities;  
using Unity.Mathematics;  
using UnityEngine;  
  
public class RotationSpeedAuthoring : MonoBehaviour  
{  
    public float DegreesPerSecond = 90f;  
  
    class Baker : Baker<RotationSpeedAuthoring>  
    {  
        public override void Bake(RotationSpeedAuthoring authoring)  
        {  
            // Dynamic – this entity will have its transform written each frame.  
            var entity = GetEntity(TransformUsageFlags.Dynamic);  
  
            AddComponent(entity, new RotationSpeed  
            {  
                RadiansPerSecond = math.radians(authoring.DegreesPerSecond)  
            });  
        }  
    }  
}
```

Adjuntar el authoring al cubo

1. Seleccionar RotatingCube en la SubScene.
2. **Inspector** → **Add Component** → **Rotation Speed Authoring**.
3. Dejar DegreesPerSecond = 90.

En cuanto la SubScene queda cerrada (ícono amarillo), el baker se ejecuta y el cubo pasa a ser una entidad con un componente RotationSpeed.

Verificar el horneado

- **Window** → **Entities** → **Hierarchy** — RotatingCube debe figurar en la lista.
- Al seleccionarlo, el **Inspector** de la derecha muestra sus componentes; buscar RotationSpeed { RadiansPerSecond: ~1.5708 } ($\pi/2$).

5. Escribir el sistema — RotationSystem

Crear Assets/Scripts/RotationSystem.cs:

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct RotationSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (transform, speed) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<RotationSpeed>>())
        {
            transform.ValueRW = transform.ValueRO.RotateY(speed.ValueRO.RadiansPerSecond * dt);
        }
    }
}
```

Notas:

- `partial struct + ISystem` es la forma de **sistema no administrado**. Se puede compilar con Burst y se ejecuta sin sobrecarga del GC.
- `SystemAPI.Query<RefRW<A>, RefRO<B>>()` es la API de iteración moderna, que sustituye al obsoleto `Entities.ForEach`.
- `LocalTransform.RotateY(radians)` devuelve una copia ya rotada, que se reasigna con `transform.ValueRW = ....`

Pulsar Play

El cubo gira. En **Window** → **Entities** → **Systems** aparece RotationSystem dentro del grupo de simulación, con su tiempo por fotograma.

6. Inspeccionar lo que se ha construido

Ventanas útiles una vez en modo Play:

Ventana	Qué muestra
Entities Hierarchy	Lista en vivo de entidades. Al seleccionar una entidad se ven su chunk y sus componentes.
Systems	Todos los sistemas en ejecución, agrupados por su SystemGroup. Aquí se ve el tiempo de cada sistema.
Archetypes	Chunks agrupados por su firma de componentes. Útil para detectar fragmentación.

Conviene confirmar:

1. La entidad RotatingCube existe y tiene LocalTransform y RotationSpeed.
2. RotationSystem aparece dentro de SimulationSystemGroup y acumula tiempo de fotograma.

7. Cambiar a IJobEntity (paralelismo)

Sustituir RotationSystem.cs por una versión basada en jobs para ver cómo encaja el paralelismo:

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct RotateJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref LocalTransform transform, in RotationSpeed speed)
    {
        transform = transform.RotateY(speed.RadiansPerSecond * DeltaTime);
    }
}

[BurstCompile]
public partial struct RotationSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new RotateJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```

Notas:

- `IJobEntity.Execute(ref A, in B)` se transforma, mediante generación de código, en un job a nivel de chunk. Los parámetros son los componentes con los que se trabaja.
- `.ScheduleParallel()` reparte los chunks entre los hilos de trabajo. Con un solo cubo no se nota ninguna mejora, pero el código escala a miles de entidades sin tocar nada.
- La dependencia del sistema se resuelve sola: `state.Dependency` se combina de forma automática al llamar a `ScheduleParallel()` desde `ISystem.OnUpdate`.

8. Qué leer a continuación

Si se desea...	Leer
Conocer en detalle cómo se mapea el authoring a entidades	DOTS Workflows/01_Baker Pattern & SubScene.md
Crear muchas entidades en tiempo de ejecución	DOTS Workflows/02_Spawner Example.md
Comprender Entity / Component / Chunk	DOTS Workflows/03_ECS Core Concepts.md
Elegir entre <code>ISystem</code> y <code>SystemBase</code>	DOTS Workflows/08_System – ISystem vs SystemBase.md

9. Solución de problemas

Síntoma	Causa / Solución
El cubo no aparece en Entities Hierarchy	La SubScene sigue abierta (ícono blanco, no amarillo). Cerrarla con la casilla de la Hierarchy.
El cubo aparece pero no tiene el componente RotationSpeed	<code>RotationSpeedAuthoring</code> no estaba adjunto, o la SubScene no se volvió a hornear tras añadirlo. Desmarcar y volver a marcar la casilla de la SubScene.
El cubo no gira	<code>RotationSystem</code> no aparece en la ventana Systems. Lo más habitual: falta la palabra clave <code>partial</code> en el struct, o el script tiene un error de compilación.
Error de compilación: TransformUsageFlags does not exist	Falta <code>using Unity.Entities;</code> en el archivo de authoring.
Error de compilación: math.radians does not exist	Falta <code>using Unity.Mathematics;</code> .

Síntoma	Causa / Solución
No se encuentra <code>SystemAPI.Query<...></code>	Falta using <code>Unity.Entities;</code> , o el struct no está marcado como <code>partial</code> .
El Inspector indica que falta <code>LocalTransform</code> en la entidad	El baker llamó a <code>GetEntity(TransformUsageFlags.None)</code> en lugar de <code>Dynamic</code> . Corregir el flag.
<code>ScheduleParallel()</code> lanza "race condition on LocalTransform"	Otro sistema escribe en <code>LocalTransform</code> en el mismo fotograma sin un orden definido. Añadir <code>[UpdateBefore]</code> / <code>[UpdateAfter]</code> o usar un <code>EntityCommandBuffer</code> compartido.

Parte II. Flujos de trabajo DOTS

Capítulo 4. Patrón Baker & SubScene

1. Introducción

Las entidades no se crean directamente en la Hierarchy. Lo que se hace es crear **GameObjects** dentro de una **SubScene**, y un **Baker** los convierte en entidades al cerrar o compilar la SubScene.

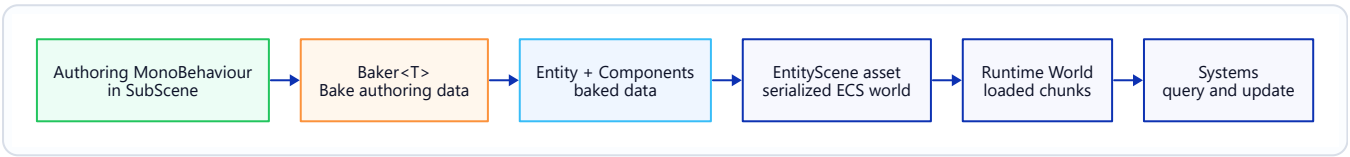


Figure: Baker and SubScene Pipeline.

2. Ciclo de vida de la SubScene

Una SubScene tiene tres estados visibles:

Ícono en Hierarchy	Estado	Qué significa
 (casilla blanca)	Abierta	Sus GameObjects son editables; aún son GameObjects y no hay horneado.
 (casilla amarilla)	Cerrada	Los Bakers ya se han ejecutado; el contenido solo existe como entidades.
	Horneando	Horneado incremental en progreso (breve).

Crear una SubScene

- **Hierarchy** → **clic derecho** → **New Sub Scene** → **Empty Scene...** — guarda un asset `.unity` y añade un componente SubScene al GameObject padre.
- Se abre y se cierra con la casilla situada junto al nombre de la SubScene. Al cerrarla se dispara un horneado.

Horneado incremental

Cuando la SubScene está abierta y se edita un GameObject, solo se reejecutan los Bakers afectados por el authoring modificado. Por eso conviene evitar operaciones costosas dentro de `Bake()`.

3. API de Baker<T>

Un Baker traduce un MonoBehaviour de authoring a una o varias entidades con sus componentes.

```
public class EnemyAuthoring : MonoBehaviour
{
    public float MaxHealth = 100f;
    public GameObject ProjectilePrefab;

    class Baker : Baker<EnemyAuthoring>
    {
        public override void Bake(EnemyAuthoring authoring)
        {
            // Primary entity for this GameObject.
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new Health { Value = authoring.MaxHealth });

            // Bake a GameObject reference into an Entity reference.
            AddComponent(self, new ProjectilePrefabRef
            {
                Value = GetEntity(authoring.ProjectilePrefab, TransformUsageFlags.Dynamic)
            });
        }
    }
}
```

APIs principales:

API	Propósito
GetEntity(TransformUsageFlags)	Entity del GameObject de authoring actual.
GetEntity(Object, TransformUsageFlags)	Entity que corresponde a otro GameObject o prefab (registra una entidad prefab).
CreateAdditionalEntity(TransformUsageFlags)	Una segunda entidad vinculada al mismo authoring (p. ej., subpartes).
AddComponent<T>(entity, value)	Asigna datos de componente.
AddBuffer<T>(entity) / SetBuffer<T>	Asigna un DynamicBuffer<T>.
AddSharedComponent<T>	Asigna un ISharedComponentData.
DependsOn(asset)	Declara una dependencia de un asset para que el Baker se reejecute cuando ese asset cambie.

4. TransformUsageFlags

El horneado solo añade LocalTransform, LocalToWorld o Parent cuando se piden explícitamente con TransformUsageFlags. Los componentes de transformación que no se usan agrandan los chunks sin motivo, así que conviene elegir el flag **más restrictivo** que aún funcione.

Flag	Añade	Uso típico
None	Nada	Entidades de datos puros (configuración, singletons, solo lógica).
ManualOverride	Nada; las transformaciones las gestiona el propio código	Disposiciones de transformación personalizadas.
Renderable	LocalToWorld	Malla estática que nunca se mueve.
Dynamic	LocalTransform, LocalToWorld (+ Parent si tiene padre)	Todo lo que se mueve o lo modifica un system.
WorldSpace	LocalTransform, LocalToWorld; impide el parentado	Entidades que ignoran la jerarquía.
NonUniformScale	Añade PostTransformMatrix	Escala no uniforme (poco común en DOTs).

Dynamic es el valor por defecto más seguro para todo lo que un system vaya a mover o girar.

5. Baking systems (avanzado)

Un Baker produce componentes a partir de un solo authoring. Un **Baking System**, en cambio, se ejecuta **después** de todos los Bakers y ve el mundo horneado completo: resulta útil para el posprocesado entre entidades (resolver referencias o calcular datos derivados).

```
[WorldSystemFilter(WorldSystemFilterFlags.BakingSystem)]
public partial struct EnemyWaveLinkSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Runs during bake only. Query entities tagged earlier by Bakers
        // and wire them together.
    }
}
```

Conviene recurrir a esto cuando un Baker no conoce todo lo que necesita (por ejemplo, recuentos o referencias cruzadas). Debe ser determinista: el horneado tiene que dar el mismo resultado en cada ejecución.

6. Ejemplo — authoring de un spawner de proyectiles

Authoring:

```
using Unity.Entities;
using UnityEngine;

public class ProjectileSpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
    public float Interval = 0.5f;

    class Baker : Baker<ProjectileSpawnerAuthoring>
    {
        public override void Bake(ProjectileSpawnerAuthoring authoring)
        {
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new ProjectileSpawner
            {
                Prefab = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic),
                Interval = authoring.Interval,
                TimeLeft = authoring.Interval
            });
        }
    }
}
```

Component:

```
using Unity.Entities;

public struct ProjectileSpawner : IComponentData
{
    public Entity Prefab;
    public float Interval;
    public float TimeLeft;
}
```

Un system en tiempo de ejecución puede crear instancias de Prefab a intervalos regulares. Véase `02_Spawner Example.md`.

7. Solución de problemas

Síntoma	Causa / Solución
La entidad aparece pero le falta un componente	El Baker no se ejecutó (el authoring se añadió después de cerrar la SubScene). Desmarcar y volver a marcar la SubScene para rehornear.
<code>GetEntity(prefab, ...)</code> devuelve <code>Entity.Null</code>	El <code>GameObject</code> referenciado no es un asset de prefab ni un objeto de escena accesible desde el Baker. Comprobar que sea un prefab o que esté en la misma SubScene.
El Baker se ejecuta en cada fotograma en el Editor	<code>Bake()</code> contiene lógica no determinista (por ejemplo, <code>Random</code> o <code>Time.time</code>). Los Bakers deben ser funciones puras del authoring.
La entidad tiene <code>LocalToWorld</code> pero no <code>LocalTransform</code>	Se usó <code>TransformUsageFlags.Renderable</code> . Cambiar a <code>Dynamic</code> si la entidad se va a mover.
Los cambios en un campo de authoring no provocan rehorneado	El Baker no lee ese campo, o no llamó a <code>DependsOn</code> para un asset externo.
El horneado es lento	Hay operaciones pesadas dentro de <code>Bake()</code> . Trasladar la lógica entre entidades a un Baking System o a un inicializador de una sola ejecución en runtime.

Capítulo 5. Ejemplo de Spawner

1. Descripción general

Un spawner es una sola entidad que crea otras entidades, ya sea al inicio, a intervalos regulares o en respuesta a un evento. Esta sección lo desarrolla de principio a fin y abarca:

- Crear el authoring del spawner en una SubScene.
- Guardar una referencia a una **entidad prefab** (horneada a partir de un prefab de GameObject).
- Crear instancias en tiempo de ejecución mediante un **EntityCommandBuffer**, para que los cambios estructurales se apliquen de forma segura.

Leer antes `01_Baker Pattern & SubScene.md` facilitará seguir esta sección.

2. Modelo de datos

```
using Unity.Entities;
using Unity.Mathematics;

public struct Spawner : IComponentData
{
    public Entity   Prefab;           // baked prefab entity
    public float3   SpawnPoint;
    public float    Interval;
    public float    TimeLeft;
    public int      RemainingCount;  // stop when 0
}
```

Conviene resaltar dos decisiones de diseño:

- Prefab es una **Entity**, no un GameObject. Los Bakers convierten el prefab de GameObject en una entidad prefab; en tiempo de ejecución solo hay entidades.
 - TimeLeft y RemainingCount residen en la entidad spawner para que el system no tenga estado propio.
-

3. Authoring + Baker

```
using Unity.Entities;
using Unity.Mathematics;
using UnityEngine;

public class SpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
    public float Interval = 0.5f;
    public int Count = 100;

    class Baker : Baker<SpawnerAuthoring>
    {
        public override void Bake(SpawnerAuthoring authoring)
        {
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new Spawner
            {
                Prefab = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic),
                SpawnPoint = authoring.transform.position,
                Interval = authoring.Interval,
                TimeLeft = 0f, // fires immediately on first update
                RemainingCount = authoring.Count
            });
        }
    }
}
```

Colocar SpawnerAuthoring en un GameObject vacío dentro de la SubScene, asignar un Prefab y fijar un número de instancias. Al cerrar la SubScene, el spawner ya es una entidad.

4. System en tiempo de ejecución

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct SpawnerSystem : ISystem
{
    [BurstCompile]
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<Spawner>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;
```

```

// Use the EndSimulation ECB so structural changes flush at a safe point.
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged);

foreach (var (spawnerRW, spawnerEntity) in
    SystemAPI.Query<RefRW<Spawner>>().WithEntityAccess())
{
    ref var spawner = ref spawnerRW.ValueRW;

    if (spawner.RemainingCount <= 0)
    {
        ecb.DestroyEntity(spawnerEntity); // done – remove the spawner
        continue;
    }

    spawner.TimeLeft -= dt;
    if (spawner.TimeLeft > 0f) continue;

    spawner.TimeLeft = spawner.Interval;
    spawner.RemainingCount--;

    var instance = ecb.Instantiate(spawner.Prefab);
    ecb.SetComponent(instance, LocalTransform.FromPosition(spawner.SpawnPoint));
}
}
}

```

Lo importante:

- `RequireForUpdate<Spawner>()` hace que el system se omita por completo mientras no exista ningún spawner.
- El singleton `EndSimulationEntityCommandBufferSystem` ofrece un ECB compartido que se vuelca al final del fotograma; es más seguro que modificar el estado con `EntityManager` dentro del bucle.
- `ecb.Instantiate(prefab)` crea una **entidad diferida**, y `ecb.SetComponent(instance, ...)` programa la escritura de sus componentes. El handle `instance` solo es válido durante la reproducción de este ECB. Véase `14_EntityCommandBuffer · Deferred Entity.md`.

5. Escalado — spawner paralelo

Si hace falta generar miles de entidades por fotograma desde muchos spawners, conviene pasar a un `IJobEntity` con un escritor paralelo de ECB:

```

[BurstCompile]
public partial struct SpawnJob : IJobEntity
{
    public float
                                DeltaTime;
}

```

```

public EntityCommandBuffer.ParallelWriter ECB;

void Execute([ChunkIndexInQuery] int chunkIndex,
            Entity entity,
            ref Spawner spawner)
{
    if (spawner.RemainingCount <= 0)
    {
        ECB.DestroyEntity(chunkIndex, entity);
        return;
    }

    spawner.TimeLeft -= DeltaTime;
    if (spawner.TimeLeft > 0f) return;

    spawner.TimeLeft = spawner.Interval;
    spawner.RemainingCount--;

    var inst = ECB.Instantiate(chunkIndex, spawner.Prefab);
    ECB.SetComponent(chunkIndex, inst, LocalTransform.FromPosition(spawner.SpawnPoint));
}
}

```

Y en el system:

```

var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();

new SpawnJob { DeltaTime = SystemAPI.Time.DeltaTime, ECB = ecb }
    .ScheduleParallel();

```

La clave de ordenación chunkIndex es lo que garantiza que la reproducción paralela sea determinista. Véase [15_ParallelWriter · Deterministic Playback.md](#).

6. Solución de problemas

Síntoma	Causa / Solución
No aparece nada	La entidad Spawner no existe. Abrir la SubScene, comprobar que SpawnerAuthoring esté adjunto y que la SubScene esté cerrada (ícono amarillo).
El prefab aparece en el origen del mundo	El ECB nunca asignó LocalTransform. Añadir <code>ecb.SetComponent(instance, LocalTransform.FromPosition(...))</code> o comprobar que el prefab ya tenga una transformación con la posición deseada.
El prefab es visible pero no se mueve	El prefab se horneó con <code>TransformUsageFlags.Renderable</code> en lugar de <code>Dynamic</code> .

Síntoma	Causa / Solución
SystemAPI.GetSingleton<...ECBS.Singleton>() lanza una excepción	El ECB system no está registrado; suele pasar en worlds muy recortados. Como alternativa, usar <code>state.EntityManager</code> con un ECB manual, o registrar el ECB system en el bootstrap.
El spawner nunca se detiene	<code>RemainingCount</code> se decrementó sobre una copia en caché de Spawner en lugar de sobre <code>spawnerRW.ValueRW</code> . Escribir siempre a través de <code>RefRW.ValueRW</code> .
ecb.SetComponent sobre una instancia diferida no surte efecto	Se usaron ECB distintos para <code>Instantiate</code> y <code>SetComponent</code> . Ambas llamadas deben usar la misma instancia de ECB.

Capítulo 6. Conceptos fundamentales de ECS

1. Los cinco sustantivos

El ECS de Entities se resume en cinco sustantivos:

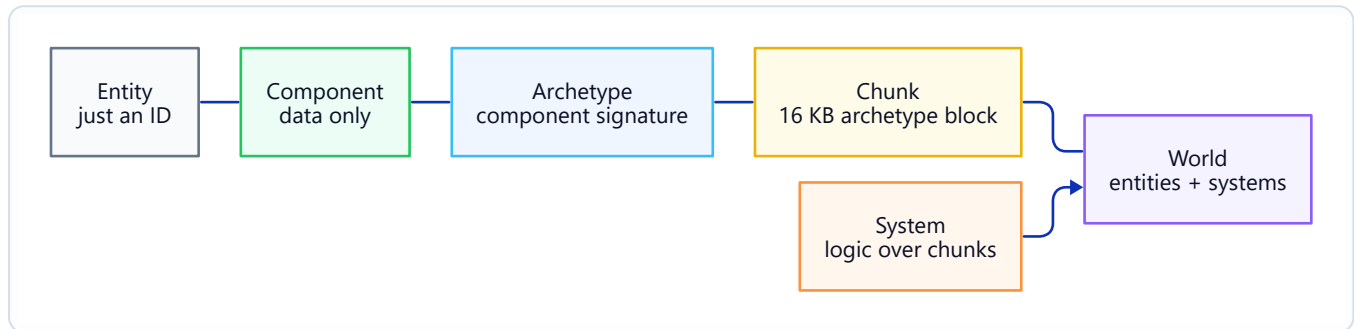


Figure: ECS Core Nouns.

Asimilar este diagrama es buena parte de lo que hace falta para leer código DOTS ya escrito.

2. Entity

Una **entity** no es más que un handle de 64 bits: `struct Entity { int Index; int Version; }`. No tiene datos ni más métodos que los de identidad; todos los datos residen en los componentes.

```
Entity e = state.EntityManager.CreateEntity(); // fresh, no components
state.EntityManager.AddComponent<Health>(e); // now it has data
state.EntityManager.DestroyEntity(e);         // handle becomes stale
```

El campo `Version` invalida las referencias antiguas: si se destruye una entidad y se vuelve a crear en el mismo `Index`, el valor antiguo de `Entity` deja de coincidir.

3. Component

Un **component** es un contenedor de datos sin más. No incluye comportamiento: el código vive en los systems.

```
public struct Health : IComponentData
{
    public float Value;
```

```
public float Max;  
}
```

Hay varias variantes de componente: no gestionados, gestionados, de buffer, compartidos, de cleanup, de tag, de chunk y habilitables. Véase `05_Component Types.md`.

4. Archetype

Un **archetype** es un conjunto único de tipos de componente que comparten algunas entidades.

- Una entidad con { `LocalTransform`, `Health`, `Enemy` } pertenece al archetype A.
- Una entidad con { `LocalTransform`, `Health`, `Enemy`, `Burning` } pertenece al archetype B, uno distinto, porque añadir `Burning` cambia el conjunto de componentes.

Pasar de un archetype a otro (añadir o quitar un componente) es un **cambio estructural** y traslada físicamente la entidad a otro chunk. Véase `13_Structural Change & Safety.md`.

5. Chunk

Las entidades del mismo archetype se empaquetan en **chunks de 16 KB**. Dentro de un chunk, cada tipo de componente forma un array contiguo, algo excelente para la localidad de caché.

Dato	Implicación
Los chunks son de 16 KB	Cada componente ocupa una parte; cuantos más componentes tenga el archetype, o más grandes sean, menos entidades caben por chunk.
Los datos se organizan por columnas (SoA), una por componente	<code>IJobChunk</code> puede recorrer un array de componentes cada vez y vectorizarlo con <code>Burst</code> .
Añadir o quitar un componente mueve la entidad	Es un cambio estructural. Los iteradores de query dan por hecho que el archetype no cambia durante la iteración.
Los chunks registran las versiones de cambio de cada componente	Los systems pueden saltarse los chunks que no han cambiado desde la última ejecución (<code>EntityQuery.SetChangedVersionFilter</code>).

La organización de los chunks en vivo se puede inspeccionar en **Window → Entities → Archetypes**.

6. System

Un **system** es una unidad de lógica que se ejecuta en cada fotograma, normalmente sobre una query de entidades.

```
[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var health in SystemAPI.Query<RefRW<Health>>())
        {
            health.ValueRW.Value = math.min(health.ValueRW.Value + dt, health.ValueRW.Max);
        }
    }
}
```

Los systems se registran de forma automática y residen dentro de un **SystemGroup**. Véanse [08_System – ISystem vs SystemBase.md](#) y [09_System Group & Update Order.md](#).

7. World

Un **World** es un contenedor que posee:

- Un **EntityManager** (crea, destruye, consulta entidades).
- Un conjunto de **systems** y sus SystemGroups.
- Los propios chunks (a través del EntityManager).

El world de tiempo de ejecución por defecto es `DefaultGameObjectInjectionWorld`. La mayoría de los juegos usan un único world en producción; las configuraciones con varios worlds aparecen en Netcode (servidor y cliente) o en las herramientas del Editor. La división de worlds en Netcode se explica en [16_Netcode Client-Server World & Bootstrap.md](#).

```
World world = World.DefaultGameObjectInjectionWorld;
EntityManager em = world.EntityManager;

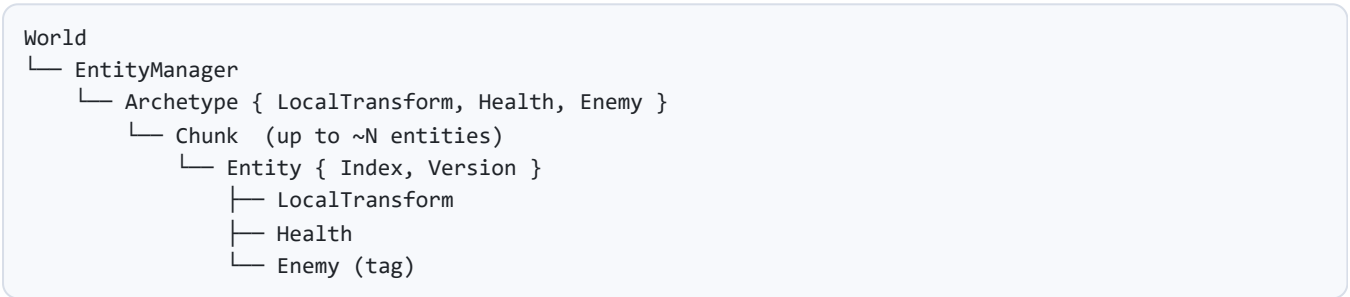
Entity e = em.CreateEntity(typeof(Health));
em.SetComponentData(e, new Health { Value = 100, Max = 100 });
```

8. EntityManager

El EntityManager es la API del hilo principal para los cambios estructurales: crear, destruir, añadir o quitar componentes, crear instancias de entidades prefab, copiar worlds, etc. Los cambios estructurales hechos con EntityManager son inmediatos y pueden invalidar cualquier iterador de query en curso o referencia de job activa.

Para cambios estructurales diferidos desde jobs o en mitad de una iteración, se usa un **EntityCommandBuffer**; véase [14_EntityCommandBuffer](#) · [Deferred Entity.md](#).

9. Todo junto



Un system que consulta { LocalTransform, Health } recorre todos los chunks cuyo archetype **contiene ambos componentes**. Eso incluye el archetype anterior y cualquier otro que, además de Enemy, tenga componentes adicionales.

10. Solución de problemas

Síntoma	Causa / Solución
Las búsquedas de Entity devuelven datos obsoletos	La entidad se destruyó y se recreó, así que el Version ya no coincide. Volver a resolver la entidad.
La query no encuentra nada aunque la entidad tiene el componente	El componente se añadió en un world distinto, o el componente es habilitable y actualmente está desactivado.
El número de Archetype se dispara	Hay demasiadas combinaciones únicas de componentes, a menudo por tags por entidad de alta cardinalidad. Consolidar con componentes compartidos o flags habilitables.
El chunk contiene solo 1 o 2 entidades	El archetype es demasiado grande. Quitar los componentes que la entidad no necesita, o repartirla en varias entidades o archetypes.

Síntoma	Causa / Solución
<code>InvalidOperationException</code> : ... <code>invalidated</code> ... durante la iteración	Se produjo un cambio estructural en mitad de la query. Usar un ECB o agrupar los cambios después del bucle.

Capítulo 7. Referencias de Entity — Entity · EntityPrefabReference · UnityObjectRef

1. Descripción general

El código DOTS usa un conjunto reducido de tipos de referencia. Es fácil confundirlos, porque todos apuntan a "algo", pero corresponden a etapas distintas del pipeline que va del authoring al runtime.

Tipo	Espacio de nombres	Qué referencia	Uso típico
Entity	Unity.Entities	Una entidad dentro de un World ECS.	Relaciones en tiempo de ejecución y campos de componente.
EntityPrefabReference	Unity.Entities.Serialization	Un prefab de entidad ya horneado que se puede cargar más adelante.	Flujos de trabajo de streaming y contenido.
UnityObjectRef<T>	Unity.Entities	Un asset u objeto UnityEngine.Object mediante un wrapper no gestionado.	Referenciar mallas, materiales, texturas, clips de audio y assets de authoring desde datos ECS.

Esta sección trata solo los tipos de referencia de DOTS / Entities. Las API de identificación de UnityEngine.Object que abarcan todo el motor quedan fuera del alcance de este manual.

2. Entity

Entity es el handle de ECS en tiempo de ejecución. Es un valor no gestionado que identifica una fila de datos de componentes dentro de un World.

```
using Unity.Entities;

public struct Target : IComponentData
{
    public Entity Value;
}
```

Reglas importantes:

- Un valor de Entity solo tiene sentido dentro del World que lo creó.
- Destruir una entidad invalida su handle anterior.

- Los índices de entidad se pueden reutilizar; antes de desreferenciar un handle guardado que no se acaba de consultar, comprobarlo con `EntityManager.Exists(entity)`.
- No conviene guardar valores de `Entity` en disco ni pasarlos entre worlds independientes como si fueran IDs estables.

```
if (state.EntityManager.Exists(target.Value))
{
    var transform = state.EntityManager.GetComponentData<LocalTransform>(target.Value);
}
```

3. Referencias de entidad en el baking

Cuando un Baker convierte objetos de authoring en datos ECS, `GetEntity` transforma un objeto de authoring o un prefab referenciado en una referencia de `Entity`.

```
using Unity.Entities;
using UnityEngine;

public class SpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
}

public struct Spawner : IComponentData
{
    public Entity Prefab;
}

public class SpawnerBaker : Baker<SpawnerAuthoring>
{
    public override void Bake(SpawnerAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new Spawner
        {
            Prefab = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic)
        });
    }
}
```

El resultado es una referencia ECS de tiempo de ejecución, no una referencia gestionada a un `GameObject`.

4. EntityPrefabReference

EntityPrefabReference sirve para contenido de prefabs que debe cargarse a través del pipeline de contenido de Entities, en lugar de incrustarse directamente como referencia de Entity en la escena actual.

```
using Unity.Entities;
using Unity.Entities.Serialization;
using UnityEngine;

public class ProjectileLibraryAuthoring : MonoBehaviour
{
    public GameObject ProjectilePrefab;
}

public struct ProjectileLibrary : IComponentData
{
    public EntityPrefabReference ProjectilePrefab;
}

public class ProjectileLibraryBaker : Baker<ProjectileLibraryAuthoring>
{
    public override void Bake(ProjectileLibraryAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new ProjectileLibrary
        {
            ProjectilePrefab = new EntityPrefabReference(authoring.ProjectilePrefab)
        });
    }
}
```

Conviene usar EntityPrefabReference cuando el prefab forma parte de un flujo de gestión de contenido o de streaming. Si el prefab está horneado en la misma SubScene y se carga junto con la escena, basta con una referencia de prefab Entity simple.

5. UnityObjectRef<T>

UnityObjectRef<T> permite que los datos ECS no gestionados referencien un UnityEngine.Object sin que el componente pase a ser gestionado.

```
using Unity.Entities;
using UnityEngine;

public struct ProjectileVisual : IComponentData
{
    public UnityObjectRef<Mesh> Mesh;
    public UnityObjectRef<Material> Material;
}
```

Hornearlo a partir de datos de authoring:

```
using Unity.Entities;
using UnityEngine;

public class ProjectileVisualAuthoring : MonoBehaviour
{
    public Mesh Mesh;
    public Material Material;
}

public class ProjectileVisualBaker : Baker<ProjectileVisualAuthoring>
{
    public override void Bake(ProjectileVisualAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new ProjectileVisual
        {
            Mesh = new UnityObjectRef<Mesh>(authoring.Mesh),
            Material = new UnityObjectRef<Material>(authoring.Material)
        });
    }
}
```

Desreferenciar `.Value` únicamente en el hilo principal o en código de presentación gestionado:

```
Mesh mesh = visual.Mesh.Value;
Material material = visual.Material.Value;
```

Reglas:

- El wrapper almacenado no es gestionado y puede residir dentro de un `IComponentData`.
- El objeto de destino sigue siendo un objeto Unity gestionado; `.Value` no es una operación apta para un Burst job.
- El objeto referenciado debe seguir cargado en el momento de desreferenciarlo.

6. Elegir la referencia correcta

Necesidad	Usar
Apuntar desde una entidad a otra dentro del mismo world	Entity
Guardar un prefab horneado en la misma SubScene	Referencia de prefab Entity desde <code>GetEntity(authoring.Prefab, ...)</code>
Guardar un prefab para streaming de contenido o carga diferida	EntityPrefabReference
Guardar en datos ECS no gestionados una referencia a una malla, material, textura, clip de audio o ScriptableObject	UnityObjectRef<T>

Necesidad	Usar
Conservar una entidad entre sesiones	Un ID estable propio del juego, no un Entity en crudo

7. Solución de problemas

Síntoma	Causa / Solución
La Entity guardada ya no existe	La entidad se destruyó o pertenece a otro world. Volver a resolverla o comprobarla con <code>EntityManager.Exists</code> .
La referencia de prefab es <code>Entity.Null</code> tras el baking	El Baker no llamó a <code>GetEntity</code> sobre el prefab, o el prefab no se incluyó en un flujo de baked scene o de contenido.
<code>UnityObjectRef<T>.Value</code> devuelve null	El objeto Unity referenciado no está cargado o se destruyó. Proteger la desreferencia y volver a cargar o resolver el asset.
Un Burst job no puede usar una referencia a una malla o un material	El almacenamiento de <code>UnityObjectRef<T></code> no es gestionado, pero <code>.Value</code> accede a objetos Unity gestionados. Trasladar la desreferencia al código de presentación del hilo principal.
El archivo de guardado almacena valores de Entity en crudo	Los handles de entidad son locales al world y no son estables. Guardar en su lugar un ID propio del juego o una clave de contenido.

Capítulo 8. Tipos de Component

1. Descripción general

En ECS, un "componente" es cualquier contenedor de datos asociado a una entidad. Entities ofrece varias variantes, con distintos perfiles de almacenamiento, vida útil y coste. Elegir la adecuada es la mitad del trabajo al diseñar un system ECS.

Mapa rápido:

Interfaz / Forma	Almacenamiento	¿Cambio estructural al mutar?	Uso típico
IComponentData (struct no gestionado)	En chunk, SoA	No (al cambiar el valor)	Componente predeterminado
IComponentData (class gestionada)	Tabla aparte	No	Tipos de referencia (mallas, referencias a GameObject)
IBufferElementData	En chunk, con desbordamiento al heap	No	Arrays dinámicos por entidad
ISharedComponentData	Por chunk	Sí (mueve la entidad a un nuevo chunk)	Agrupar entidades que comparten configuración
ICleanupComponentData	En chunk	Normal	Protocolo de limpieza al destruir entidades
ICleanupSharedComponentData	Por chunk	Normal	Protocolo de limpieza de recursos compartidos
Tag (IComponentData vacío)	Ocupa cero bytes en el chunk	No	Marcador de filtro, sin datos
Chunk component	Por chunk	Sí	Metadatos calculados una vez por chunk
Enableable (IEnableableComponent)	En chunk + máscara de bits	No	Activar o desactivar el comportamiento por entidad

El resto de la sección desarrolla cada fila.

2. IComponentData no gestionado

La opción por defecto. Reside directamente en el chunk como datos SoA bien empaquetados. Compatible con Burst.

```
public struct Velocity : IComponentData
{
    public float3 Value;
}
```

Reglas:

- Debe ser un struct, sin campos de referencia (nada de class, string ni List<T>).
 - Tipos de campo permitidos: primitivos, tipos de Unity.Mathematics, otros structs no gestionados, Entity, FixedString*, BlobAssetReference<T> y UnityObjectRef<T>.
 - Se lee con RefRO<T> y se escribe con RefRW<T> desde SystemAPI.Query.
-

3. IComponentData gestionado

Reside en una tabla aparte indexada por entidad. Admite tipos de referencia, pero conlleva una asignación de GC y rompe la compatibilidad con Burst de ese componente.

```
public class EnemyAI : IComponentData
{
    public AIModel Brain;        // a managed class
    public List<Entity> Allies;
}
```

Conviene usarlo solo cuando se necesita de verdad una referencia gestionada (una malla, una textura o un ScriptableObject de authoring). Cuando basta con una referencia débil a un UnityEngine.Object, es preferible UnityObjectRef<T> dentro de un componente no gestionado.

4. IBufferElementData — buffers dinámicos

Un DynamicBuffer<T> es un array de tamaño variable, uno por entidad.

```
public struct Waypoint : IBufferElementData
{
    public float3 Position;
}
```

```
var buffer = state.EntityManager.AddBuffer<Waypoint>(entity);
buffer.Add(new Waypoint { Position = new float3(1, 0, 0) });
```

Almacenamiento: hasta `InternalBufferCapacity` elementos caben en el chunk; a partir de ahí el buffer se desborda al heap. La capacidad se ajusta con `[InternalBufferCapacity(N)]`.

5. `ISharedComponentData` — agrupación

Los componentes compartidos llevan un **valor que agrupa chunks**. Dos entidades con el mismo valor de `ISharedComponentData` acaban en el mismo chunk; con valores distintos van a chunks distintos.

```
public struct RenderMeshGroup : ISharedComponentData
{
    public int MaterialID;
}
```

Consecuencias:

- Cambiar el valor traslada la entidad a otro chunk: es un **cambio estructural** y resulta costoso.
 - Ideal para datos que cambian poco pero varían según el grupo (juegos de materiales, IDs de equipo, regiones de bake).
 - Si el tipo contiene referencias, los valores se guardan en una tabla gestionada (un `ISharedComponentData` puede ser gestionado).
-

6. Componentes de limpieza — `ICleanupComponentData`

Marca una entidad de modo que, al "destruirse", siga viva solo con sus componentes de limpieza hasta que el system los procese.

```
public struct ConnectionCleanup : ICleanupComponentData
{
    public int ConnectionHandle;
}

// DestroyEntity(e) removes all normal components but leaves ConnectionCleanup
// behind so you can close the socket in a dedicated system, then destroy the
// cleanup component explicitly.
```

Su equivalente `ICleanupSharedComponentData` aplica el mismo protocolo a los recursos compartidos.

Casos de uso: liberar recursos nativos, darse de baja de systems externos o destruir una entidad por etapas a lo largo de varios fotogramas.

7. Componentes tag

Un **tag** es un `IComponentData` vacío:

```
public struct Enemy : IComponentData {}
```

Ocupa cero bytes en el chunk y se puede consultar como cualquier otro componente. Ideal como marcador del tipo "es un X". Abusar de los tags (cientos de tipos distintos) fragmenta los archetypes; para estados de alta cardinalidad conviene usar componentes habilitables o compartidos.

8. Chunk components

Un chunk component lleva **un valor por chunk**, no por entidad.

```
state.EntityManager.AddChunkComponentData<RegionBounds>(query, new RegionBounds { ... });
```

Uso típico: datos precalculados que valen para todas las entidades del chunk (bounds, ID de región, clave de partición). Modificar un chunk component provoca un cambio estructural.

9. Enableable components (`IEnableableComponent`)

Véase la página dedicada `06_Enableable Component.md` para más detalle. Resumen aquí: un componente que puede activarse o desactivarse por entidad **sin** un cambio estructural, mediante una máscara de bits por chunk.

```
public struct Stunned : IComponentData, IEnableableComponent
{
    public float Duration;
}

state.EntityManager.SetComponentEnabled<Stunned>(entity, false); // no structural change
```

Las queries descartan de forma automática las entidades cuyos enableable components están desactivados; es una manera barata y habitual de excluir entidades de un system de forma temporal.

10. Guía de decisión

Quiero...	Usar
Guardar datos numéricos por entidad	ICollectionData no gestionado
Mantener una referencia gestionada por entidad	ICollectionData gestionado o UnityObjectRef<T>
Guardar una lista de longitud variable por entidad	IBufferElementData
Agrupar entidades para procesamiento por lotes	ISharedComponentData
Activar o desactivar un componente por fotograma	Enableable component
Solo marcar un tipo	Tag (ICollectionData vacío)
Precalcular datos para un chunk completo	Chunk component
Ejecutar código cuando se destruye una entidad	Cleanup component

11. Solución de problemas

Síntoma	Causa / Solución
InvalidOperationException: The type T must be unmanaged	Se usó un tipo gestionado donde hace falta uno no gestionado. Cambiar el componente o la API (por ejemplo, ComponentLookup<T> en vez de ManagedAPI.GetComponent<T>).
Cambiar el valor de un componente compartido provoca tirones	Cada cambio es un cambio estructural. Agruparlos con un ECB, o asumir que este valor se modifica pocas veces.
DynamicBuffer<T> se desborda al heap	[InternalBufferCapacity(N)] se queda corto para el uso real. Aumentarlo o mantener los tamaños acotados.
La entidad reaparece tras DestroyEntity	Todavía hay un cleanup component adjunto. Eliminarlo de forma explícita una vez terminada la limpieza.
Demasiados archetypes	Explosión de tags. Convertir los "estados" únicos por entidad en enableable components.
El componente tag no filtra una query	Los tag components no tienen datos, así que no se puede pedir un RefRW<Tag> ni un RefRO<Tag> en la tupla de SystemAPI.Query<...>. Hay que aplicarlos con .WithAll<Tag>() o .WithNone<Tag>().

Capítulo 9. Enableable Component

1. Por qué existen los enableable components

Añadir o quitar un componente es un **cambio estructural**: la entidad pasa a otro chunk, las queries tienen que volver a resolverse y las referencias pueden invalidarse. Si esto ocurre en cada fotograma y por entidad (por ejemplo, al activar y desactivar "Stunned"), ese coste acapara el fotograma.

Un **enableable component** permite activar y desactivar comportamiento **sin** ningún cambio estructural. El componente siempre está presente en el chunk; un bit de una máscara por chunk determina si, para una query, cada entidad lo "tiene" o no.

2. La interfaz

Implementar tanto IComponentData como IEnableableComponent:

```
using Unity.Entities;

public struct Stunned : IComponentData, IEnableableComponent
{
    public float Duration;
}
```

También es posible hacer habilitable un IBufferElementData.

3. Activación y desactivación

Desde un system:

```
// Turn Stunned off without removing the component
state.EntityManager.SetComponentEnabled<Stunned>(entity, false);

// From an ECB (deferred, safe mid-iteration)
ecb.SetComponentEnabled<Stunned>(entity, true);

// From a job with a ComponentLookup
var stunnedLookup = SystemAPI.GetComponentLookup<Stunned>();
stunnedLookup.SetComponentEnabled(entity, false);
```

Leer el estado actual:

```
bool isStunned = state.EntityManager.IsComponentEnabled<Stunned>(entity);
```

Las tres vías actúan sobre una máscara de bits; ninguna provoca un cambio estructural.

4. Comportamiento de las queries

`SystemAPI.Query<...>` y la iteración basada en `EntityQuery` **descartan** las entidades cuyos enableable components estén desactivados en ese momento; por defecto, solo se ven las entidades en las que todos los componentes indicados están habilitados.

```
foreach (var stunned in SystemAPI.Query<RefRW<Stunned>>())
{
    // only fires for entities where Stunned is enabled
}
```

Para ver también las entidades desactivadas, se puede sobrescribir este comportamiento:

```
foreach (var (stunnedRW, entity) in SystemAPI
    .Query<RefRW<Stunned>>()
    .WithEntityAccess()
    .WithOptions(EntityQueryOptions.IgnoreComponentEnabledState))
{
    // fires for ALL entities with the component, enabled or not
}
```

5. Ejemplo — stun que expira

Un mismo `IJobEntity` no puede recibir a la vez `ref Stunned` (un `RefRW<T>`) y `EnabledRefRW<Stunned>` para el mismo componente: Entities prohíbe expresamente envolver el mismo componente en ambas formas dentro de un job. Hay dos patrones válidos:

5.1 Leer y escribir el valor, y conmutar el bit de habilitado mediante ECB

```
[BurstCompile]
public partial struct StunTickJob : IJobEntity
{
    public float DeltaTime;
    public EntityCommandBuffer.ParallelWriter ECB;

    void Execute([ChunkIndexInQuery] int chunkIndex,
        Entity entity,
        ref Stunned stunned)
    {
        stunned.Duration -= DeltaTime;
        if (stunned.Duration <= 0f)
    }
```

```

        ECB.SetComponentEnabled<Stunned>(chunkIndex, entity, false);
    }
}

```

La reproducción se produce en el siguiente límite de ECB (por ejemplo, EndSimulationECBS); véase 14_EntityCommandBuffer · Deferred Entity.md. No hay cambio estructural: solo se invierte un bit en un punto bien definido.

5.2 Conmutar el bit directamente cuando no hace falta el valor del componente

```

[BurstCompile]
public partial struct StunDisableJob : IJobEntity
{
    void Execute(EnabledRefRW<Stunned> stunnedEnabled)
    {
        stunnedEnabled.ValueRW = false;    // no structural change
    }
}

```

EnabledRefRW<T> es la única referencia a Stunned en este job, por lo que compila. Conviene usar esta forma cuando un system se limita a conmutar.

¿Hace falta leer el valor **y** conmutar el bit de habilitado de la misma entidad en una sola pasada? Conviene hacerlo desde el hilo principal con EntityManager.SetComponentEnabled<T>(entity, false), o repartirlo en dos jobs encadenados mediante state.Dependency.

6. Cuándo preferir enableable frente a añadir/quitar

Situación	Elegir
Conmutación frecuente (por fotograma o por evento)	Enableable
Estado que no siempre está presente (un modificador por entidad con campos variables)	Enableable
Cambio de archetype poco frecuente (por ejemplo, añadir una capacidad permanente)	Añadir o quitar el componente de la forma habitual
Filtrado uniforme en muchos systems	Enableable (las queries lo recogen automáticamente)

Regla general: si el bit de "tiene este componente" cambia varias veces durante la vida de una entidad, conviene hacerlo habilitable.

7. Interacción con otras características

Característica	Comportamiento
<code>WithChangeFilter<T></code>	Conmutar el bit también incrementa la versión de cambio del componente.
<code>ICleanupComponentData</code>	Un cleanup component no puede ser también habilitable.
Netcode for Entities	Los enableable components pueden sincronizarse como ghosts, pero los cambios de estado deben pasar por el lado autoritativo.
Serialización / exportación de world	Los bits de habilitación se preservan.

8. Solución de problemas

Síntoma	Causa / Solución
La query incluye entidades que deberían estar "desactivadas"	El componente no implementa <code>IEnableableComponent</code> , o se ha aplicado <code>WithOptions(EntityQueryOptions.IgnoreComponentEnabledState)</code> .
<code>SetComponentEnabled<T></code> lanza "type T does not support being enableable"	El struct solo implementa <code>IComponentData</code> . Añadir también <code>IEnableableComponent</code> .
El parámetro <code>EnabledRefRW<T></code> es rechazado por el source generator	Al job o system le falta <code>partial</code> , o el componente no es habilitable.
Error del source generator "cannot have both RefRW and EnabledRefRW for the same component"	Repartir la lógica: usar <code>ref T o EnabledRefRW<T></code> por job, no ambos sobre el mismo componente. Los dos patrones están en §5.
El filtro de cambio se dispara sin motivo aparente	Conmutar <code>Enabled</code> cuenta como cambio para <code>WithChangeFilter</code> . Es lo esperado: conviene combinarlo con lecturas de <code>EnabledRefRO</code> en lugar de conmutar.
El <code>SetComponentEnabled</code> del ECB no surte efecto	Se llamó sobre una entidad diferida antes de que se volcara el ECB que la creó. Ambas operaciones deben estar en el mismo ciclo de ECB.

Capítulo 10. Singleton Component

1. Qué es (y qué no es) un singleton

En Entities, un **singleton** es sin más un componente que se espera que exista en **una sola entidad** del world. No hay una interfaz especial ISingleton: cualquier IComponentData es un singleton por convención, y SystemAPI ofrece accesorios cómodos que verifican la regla de "exactamente uno".

Usos típicos: estado del juego, configuración global, fuentes de tiempo y de tick, estado de red o registros centralizados de ECB.

```
public struct GameState : IComponentData
{
    public int    Score;
    public float  TimeScale;
    public bool   Paused;
}
```

2. Los accesorios

Dentro de cualquier ISystem / SystemBase:

```
// Reads
GameState gs      = SystemAPI.GetSingleton<GameState>();
RefRW<GameState> gsRW = SystemAPI.GetSingletonRW<GameState>();
bool has         = SystemAPI.HasSingleton<GameState>();
bool got         = SystemAPI.TryGetSingleton<GameState>(out var gs2);

// Writes
SystemAPI.SetSingleton(new GameState { Score = 10 });

// Entity handle, if you need it
Entity e         = SystemAPI.GetSingletonEntity<GameState>();

// Buffers work too
DynamicBuffer<Waypoint> buf = SystemAPI.GetSingletonBuffer<Waypoint>();
```

Las variantes Get* lanzan una excepción si hay cero singletons o más de uno. Las variantes TryGet* devuelven false en esos casos. Las variantes RW marcan el componente como modificado para el seguimiento de cambios.

3. Crear el singleton

Dos patrones comunes:

3.1 Hornearlo desde una SubScene

```
public class GameStateAuthoring : MonoBehaviour
{
    public float InitialTimeScale = 1f;

    class Baker : Baker<GameStateAuthoring>
    {
        public override void Bake(GameStateAuthoring authoring)
        {
            var e = GetEntity(TransformUsageFlags.None);
            AddComponent(e, new GameState
            {
                Score      = 0,
                TimeScale = authoring.InitialTimeScale,
                Paused     = false
            });
        }
    }
}
```

Adjuntar GameStateAuthoring a un solo GameObject de la SubScene; el Baker produce exactamente una entidad con el singleton.

3.2 Crearlo al arrancar el system

```
public void OnCreate(ref SystemState state)
{
    var e = state.EntityManager.CreateEntity(typeof(GameState));
    state.EntityManager.SetComponentData(e, new GameState { TimeScale = 1f });
}
```

Es la opción para datos que no deben residir en el authoring (configuración inyectada por el servidor, generada de forma procedural, etc.).

4. Condicionar un system al singleton

Si un system solo tiene sentido cuando existe el singleton:

```
public void OnCreate(ref SystemState state)
{
    state.RequireForUpdate<GameState>();
}
```

`RequireForUpdate<T>()` indica al system que **se salte `OnUpdate` por completo** mientras el componente no esté presente. Así se evitan comprobaciones de nulidad defensivas dentro de `OnUpdate`.

5. Rendimiento de solo lectura frente a lectura-escritura

- `GetSingleton<T>()` — lee una copia, sin seguimiento de cambios.
- `GetSingletonRW<T>()` — devuelve un `RefRW<T>` que apunta al chunk. Asignar `.ValueRW` marca el componente como modificado, que es justo lo que interesa si otros systems usan `WithChangeFilter<T>`.
- `SetSingleton` — escritura completa; siempre incrementa la versión.

En systems con muchas lecturas conviene `GetSingleton`; usar `GetSingletonRW` solo cuando de verdad se modifica el valor.

6. Múltiples candidatos — la trampa del "singleton con más de uno"

`GetSingleton<T>` lanza `InvalidOperationException: expected exactly one entity with component T, found N` cuando dos entidades terminan con el mismo componente singleton.

Causas habituales:

- Dos `GameObjects` de la `SubScene` llevan el script de authoring.
- Un fixture de prueba creó el singleton en tiempo de ejecución **y** la `SubScene` también lo hornea.
- Un system crea el singleton en `OnCreate` sin protegerlo con `HasSingleton`.

Patrón defensivo:

```
public void OnCreate(ref SystemState state)
{
    if (!SystemAPI.HasSingleton<GameState>())
    {
        var e = state.EntityManager.CreateEntity(typeof(GameState));
        state.EntityManager.SetComponentData(e, new GameState { TimeScale = 1f });
    }
}
```

7. Buffer singletons

Un `DynamicBuffer<T>` también puede ser un singleton:

```
public struct QueuedEvent : IBufferElementData
{
    public int Type;
    public int Payload;
}

// Access:
DynamicBuffer<QueuedEvent> events = SystemAPI.GetSingletonBuffer<QueuedEvent>();
events.Add(new QueuedEvent { Type = 1 });
```

Resulta útil para colas de eventos globales y flujos de comandos.

8. Solución de problemas

Síntoma	Causa / Solución
<code>InvalidOperationException: expected exactly one entity with component T, found 0</code>	La entidad singleton nunca se creó. Añadir un baker, o crearla en <code>OnCreate</code> protegida por <code>HasSingleton</code> .
<code>... found 2</code>	Dos entidades llevan el componente. Eliminar el duplicado en el <code>authoring</code> o limpiarlo al arrancar.
<code>GetSingletonRW</code> no parece marcar el componente como modificado	Se escribió sobre <code>.ValueRO</code> o se ignoró el <code>RefRW</code> . Solo <code>.ValueRW</code> activa el seguimiento de cambios.
El system se ejecuta antes de que se cree el singleton	Usar <code>RequireForUpdate<T></code> , u ordenar los systems para que el creador se ejecute antes (<code>[UpdateBefore]</code>).
Dos systems modifican el singleton sin un orden definido	Añadir <code>[UpdateAfter]</code> / <code>[UpdateBefore]</code> o unificar la lógica. Las condiciones de carrera en singletons son especialmente difíciles de depurar.
Netcode: el valor del singleton diverge entre cliente y servidor	Sincronizar el singleton como ghost, o reconstruirlo de forma determinista a partir de inputs sincronizados.

Capítulo 11. System — ISystem vs SystemBase

1. Descripción general

Entities ofrece dos maneras de declarar un system: **ISystem** (un struct no gestionado) y **SystemBase** (una class gestionada). Conviene elegir ISystem salvo que se necesite en concreto lo que aporta SystemBase.

	ISystem	SystemBase
Se declara como	partial struct : ISystem	partial class : SystemBase
Memoria	No gestionada, ubicable en la pila	Gestionada, propiedad del GC
Compilable con Burst	Sí (añadir [BurstCompile])	No
Campos gestionados	No	Sí
Acceso a API gestionada	Mediante EntityManager + ManagedAPI	Directamente
Opción predeterminada		Solo si es imprescindible

2. ISystem — la opción por defecto

```
using Unity.Burst;
using Unity.Entities;

[BurstCompile]
public partial struct EnemyAISystem : ISystem
{
    [BurstCompile]
    public void OnCreate(ref SystemState state)
    {
        // Called once when the system is created.
        state.RequireForUpdate<EnemySpawnConfig>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var config = SystemAPI.GetSingleton<EnemySpawnConfig>();
        // ... logic
    }

    [BurstCompile]
    public void OnDestroy(ref SystemState state)
    {
    }
}
```

```
        // Called once when the system is destroyed.  
    }  
}
```

Requisitos obligatorios:

- El struct debe ser `partial`: el source generator le añade un wrapper oculto.
- Los tres métodos (`OnCreate`, `OnUpdate` y `OnDestroy`) son opcionales; basta con omitir los que no se necesiten.
- Hay que poner `[BurstCompile]` en el struct **y** en cada método que se quiera compilar con Burst.

3. SystemBase — cuando un system gestionado es inevitable

```
using Unity.Entities;  
  
public partial class PlayerInputSystem : SystemBase  
{  
    private InputActions _input;    // managed field – the reason SystemBase exists  
  
    protected override void OnCreate()  
    {  
        _input = new InputActions();  
        _input.Enable();  
    }  
  
    protected override void OnUpdate()  
    {  
        var move = _input.Gameplay.Move.ReadValue<Vector2>();  
        // ... dispatch to components  
    }  
  
    protected override void OnDestroy() => _input.Dispose();  
}
```

Conviene recurrir a `SystemBase` cuando:

- Hay un campo gestionado que debe residir junto al system (objetos del Input System, handles de UI Toolkit, servicios gestionados, etc.).
- Hace falta llamar a APIs gestionadas directamente dentro de `OnUpdate` sin pasar por el `EntityManager`.

Todo lo de un `SystemBase` se ejecuta en el hilo principal y sin Burst.

4. Ciclo de vida del system

Las dos variantes comparten los mismos nombres de hook:

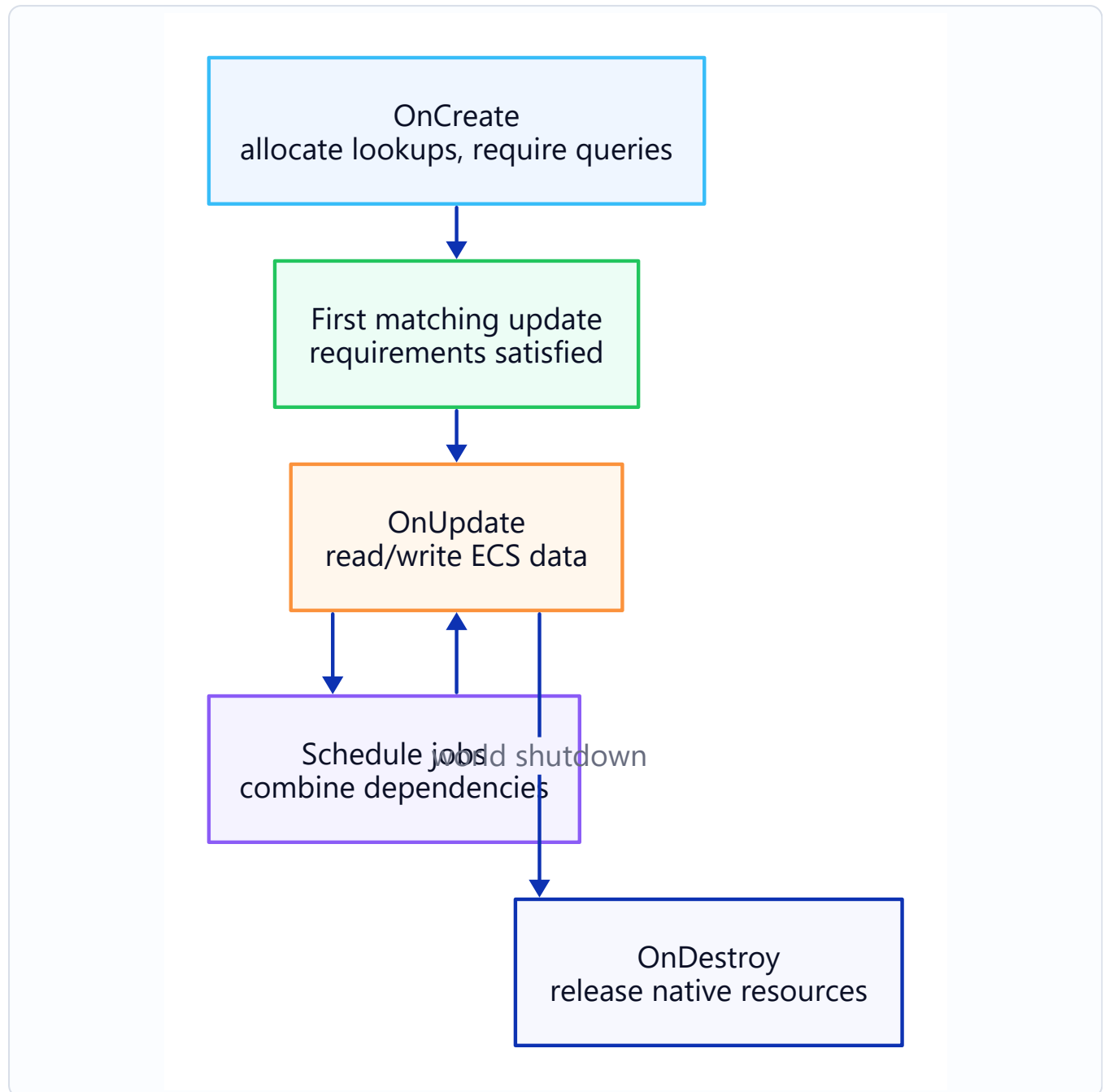


Figure: System Lifecycle.

Hook	Cuándo se dispara	Uso típico
OnCreate	Una vez, al crear el system	Cachear queries, RequireForUpdate<T>, asignar contenedores nativos
OnStartRunning	Una query pasa de vacía a no vacía	Reiniciar el estado de la sesión
OnUpdate	En cada fotograma, dentro del SystemGroup padre	Trabajo principal

Hook	Cuándo se dispara	Uso típico
OnStopRunning	Una query pasa de no vacía a vacía	Volcar el estado almacenado en buffer
OnDestroy	Una vez, al destruir el world	Liberar recursos nativos o gestionados

RequireForUpdate<T>()

Saltarse OnUpdate por completo cuando falta un componente o singleton requerido:

```
public void OnCreate(ref SystemState state)
{
    state.RequireForUpdate<EnemySpawnConfig>();
    state.RequireForUpdate<GameRunning>();
}
```

Es la forma habitual de condicionar un system a que exista la configuración o a que haya empezado el juego.

5. Elegir entre los dos

Reglas prácticas:

1. **Empezar con ISystem.** Es compatible con Burst y no genera basura para el GC.
2. **Pasar a SystemBase solo cuando un campo gestionado lo obligue.** Y aun así, conviene plantearse dividirlo: dejar el trabajo gestionado en un SystemBase ligero que escriba los resultados en componentes, y ejecutar la lógica pesada en un ISystem que lea esos componentes.
3. **No mezclar** añadiendo un campo gestionado a un struct ISystem: no compila. La división anterior es la solución idiomática.

Ejemplo de división:

```
// Managed – bridges Input System into components.
public partial class InputBridgeSystem : SystemBase
{
    private InputActions _input;
    // ... fills a PlayerInput singleton each frame
}

// Unmanaged + Burst – reads the PlayerInput singleton and drives gameplay.
[BurstCompile]
public partial struct PlayerMoveSystem : ISystem
{
    [BurstCompile]
```

```
public void OnUpdate(ref SystemState state) { /* ... */ }  
}
```

6. Referencia rápida de SystemState

Dentro de un `ISystem`, `ref SystemState state` es la vía de acceso al world.

Miembro	Propósito
<code>state.EntityManager</code>	Cambios estructurales en el hilo principal.
<code>state.World</code>	El World propietario.
<code>state.Dependency</code>	El handle de dependencia de jobs actual; debe combinarse con los jobs programados.
<code>state.GetEntityQuery(...)</code>	Crear o recuperar una query cacheada.
<code>state.RequireForUpdate<T>()</code>	Condicionar el system a la presencia de un componente.
<code>state.Enabled</code>	Ponerlo en false para saltarse el system sin destruirlo.

En un `SystemBase`, estos son miembros de la propia clase (por ejemplo, `this.EntityManager`).

7. Solución de problemas

Síntoma	Causa / Solución
<code>error CS0246: The type or namespace name 'Entities' could not be found</code>	Falta <code>using Unity.Entities;</code> .
<code>OnUpdate</code> nunca se dispara	<code>RequireForUpdate<T></code> está condicionando a un componente que aún no existe. Comprobar que el componente se cree en algún otro punto.
Error de compilación: "partial struct must be declared partial"	<code>ISystem</code> exige <code>partial</code> para que el source generator pueda ampliar el struct.
Advertencia de Burst: "method is not burst compiled"	Falta <code>[BurstCompile]</code> en el método. No basta con tenerlo solo en el struct.
El system se ejecuta en el world incorrecto (cliente frente a servidor con Netcode)	Añadir <code>[WorldSystemFilter(...)]</code> para elegir entre <code>ServerSimulation</code> , <code>ClientSimulation</code> o <code>LocalSimulation</code> .
No se puede añadir un campo gestionado a <code>ISystem</code>	Es lo esperado: separar la parte gestionada en un <code>SystemBase</code> , como se muestra arriba.

Síntoma	Causa / Solución
El system no aparece en Window → Entities → Systems	Falta partial, está fuera del world por defecto, o lo excluye [DisableAutoCreation].

Capítulo 12. System Group & orden de actualización

1. Descripción general

Cada system reside dentro de un **SystemGroup**. Los grupos deciden cuándo se ejecutan sus hijos en cada fotograma, y los atributos de ordenación determinan la secuencia **dentro** de cada grupo.

Acertar con la agrupación es lo que evita escrituras que llegan un fotograma tarde, condiciones de carrera y esos errores del tipo "por qué tiembla esta entidad".

2. Grupos predefinidos

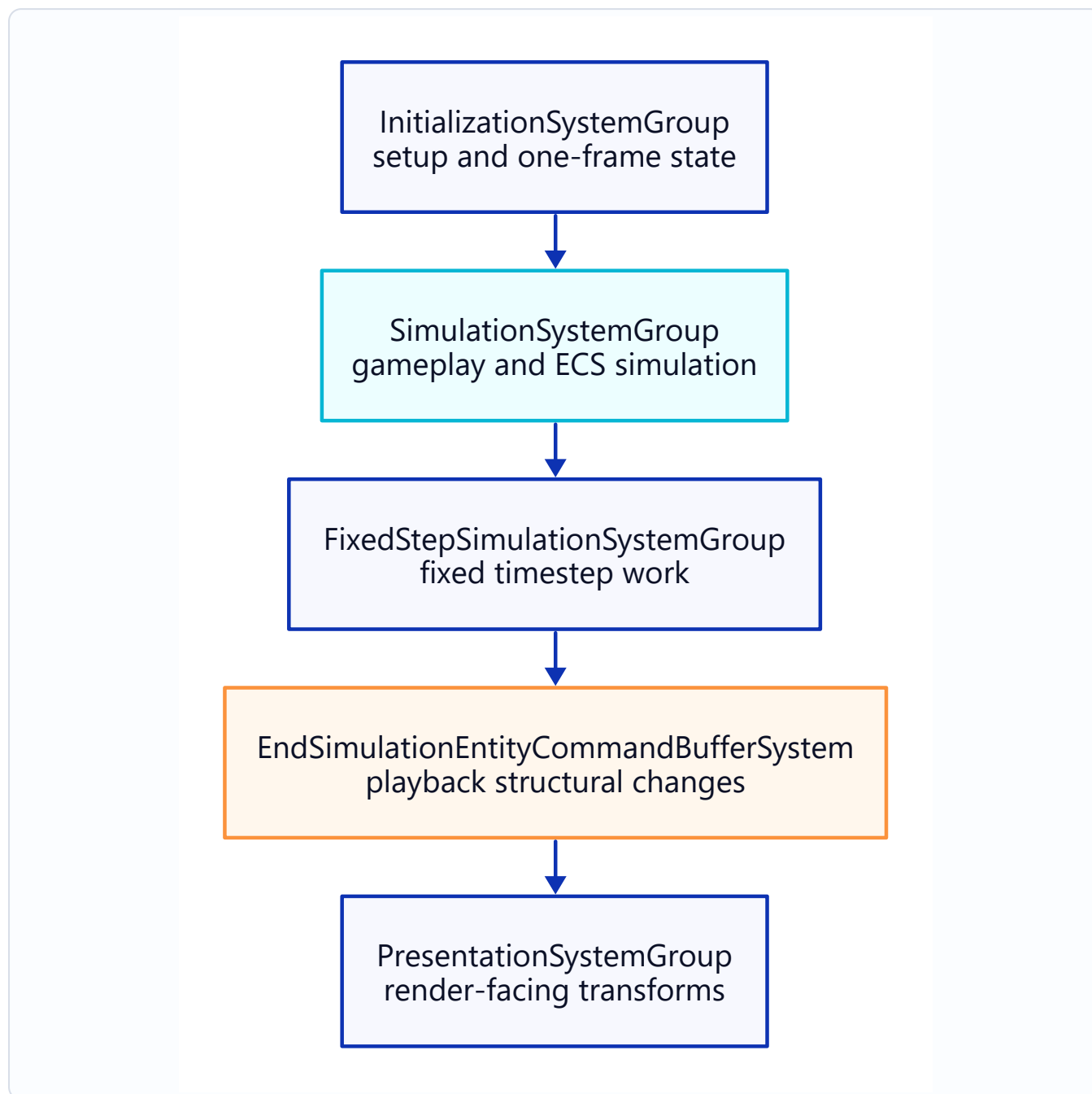


Figure: System Group and Update Order.

Grupo	Cuándo se ejecuta	Contenido típico
InitializationSystemGroup	Una vez por fotograma, antes de la simulación	Inicializadores de una sola ejecución, tareas de bootstrap
SimulationSystemGroup	Una vez por fotograma	La mayor parte de la lógica de juego
FixedStepSimulationSystemGroup	Paso de tiempo fijo (60 Hz por defecto); puede ejecutarse 0, 1 o N veces por fotograma	Física, lógica de paso determinista

Grupo	Cuándo se ejecuta	Contenido típico
PresentationSystemGroup	Una vez por fotograma, después de la simulación	Preparación de render, interpolación
BeginSimulationEntityCommandBufferSystem	Lo primero dentro de Simulation	Reproducción ordenada de ECB (temprana)
EndSimulationEntityCommandBufferSystem	Lo último dentro de Simulation	Reproducción ordenada de ECB (tardía)

Netcode for Entities añade otros (**PredictedSimulationSystemGroup**, **GhostSendSystem**, etc.) que se anidan dentro de los anteriores. Véanse [16_Netcode Client-Server World & Bootstrap.md](#) y [19_Netcode Prediction & Rollback.md](#).

3. Colocar un system en un grupo

```
using Unity.Entities;

[UpdateInGroup(typeof(SimulationSystemGroup))]
public partial struct EnemyAISystem : ISystem { /* ... */ }
```

Si se omite `[UpdateInGroup]`, el system se asigna por defecto a `SimulationSystemGroup`.

Anidamiento

Un grupo puede estar dentro de otro grupo:

```
[UpdateInGroup(typeof(SimulationSystemGroup), OrderFirst = true)]
public partial class GameplaySystemGroup : ComponentSystemGroup { }

[UpdateInGroup(typeof(GameplaySystemGroup))]
public partial struct MovementSystem : ISystem { /* ... */ }
```

`ComponentSystemGroup` es la clase base de los grupos personalizados. Funcionan a la vez como systems y como contenedores.

4. Ordenación dentro de un grupo

```
[UpdateInGroup(typeof(SimulationSystemGroup))]
[UpdateAfter(typeof(EnemyAISystem))]
[UpdateBefore(typeof(MovementSystem))]
public partial struct EnemyPathfindingSystem : ISystem { /* ... */ }
```

Atributo	Significado
[UpdateBefore(typeof(X))]	Ejecutarse antes que X (ambos deben estar en el mismo grupo).
[UpdateAfter(typeof(X))]	Ejecutarse después de X.
OrderFirst = true (en [UpdateInGroup])	Ejecutarse lo antes posible dentro del grupo.
OrderLast = true	Ejecutarse lo más tarde posible.

Consejo: es preferible encadenar [UpdateAfter] que usar OrderFirst/Last; se lee mejor en las revisiones de código y aguanta mejor las refactorizaciones.

5. [RequireMatchingQueriesForUpdate]

Un atributo más reciente (Entities 1.x+) que indica a un system que **se salte OnUpdate salvo que al menos una entidad coincida con cada una de sus queries**:

```
[RequireMatchingQueriesForUpdate]
public partial struct EnemyAISystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Only runs when there is at least one entity matching every query the system uses.
    }
}
```

Conviene compararlo con `state.RequireForUpdate<T>()`, que exige un componente o singleton **concreto**. [RequireMatchingQueriesForUpdate] es más implícito: depende de las queries que acabe teniendo el system.

6. Grupos personalizados

Razones para crear uno:

- Una fase de juego cohesionada (por ejemplo, "aplicar el daño antes de la animación").
- Una frecuencia de tick distinta (por ejemplo, un paso fijo propio a 30 Hz).
- Un interruptor de funcionalidad: activar o desactivar el grupo entero.

```
using Unity.Entities;

[UpdateInGroup(typeof(SimulationSystemGroup))]
```

```

public partial class DamagePipelineGroup : ComponentSystemGroup
{
    // You can override OnUpdate to customise how the group ticks.
}

[UpdateInGroup(typeof(DamagePipelineGroup), OrderFirst = true)]
public partial struct ApplyDamageSystem : ISystem { /* ... */ }

[UpdateInGroup(typeof(DamagePipelineGroup))]
[UpdateAfter(typeof(ApplyDamageSystem))]
public partial struct DeathSystem : ISystem { /* ... */ }

```

Frecuencia de tick personalizada

```

public partial class SlowTickGroup : ComponentSystemGroup
{
    private float _accum;
    protected override void OnUpdate()
    {
        _accum += (float)SystemAPI.Time.DeltaTime;
        if (_accum < 0.1f) return; // 10 Hz
        _accum = 0f;
        base.OnUpdate(); // tick children
    }
}

```

7. Paso de tiempo fijo frente a variable

Si importa el determinismo (física, predicción en red, reproducción determinista), conviene poner el system en FixedStepSimulationSystemGroup:

```

[UpdateInGroup(typeof(FixedStepSimulationSystemGroup))]
public partial struct PhysicsStepSystem : ISystem { /* ... */ }

```

Puntos a tener en cuenta:

- FixedStepSimulationSystemGroup puede ejecutarse **varias veces** por fotograma (o ninguna) para ponerse al día con el tiempo real.
- Usar SystemAPI.Time.DeltaTime: dentro de este grupo es el dt del paso fijo.
- Todo lo que lea input o genere presentación debe quedarse en SimulationSystemGroup o PresentationSystemGroup.

8. Inspeccionar el orden en tiempo de ejecución

Window → **Entities** → **Systems** muestra:

- Cada grupo y sus hijos, en el orden de ejecución real.
- Tiempo por system.
- El estado habilitado o deshabilitado (de los systems que se conmutan en tiempo de ejecución).

Si el orden declarado con atributos no coincide con lo que se ve, casi siempre hay una dependencia circular o un [UpdateInGroup] que se ha olvidado.

9. Solución de problemas

Síntoma	Causa / Solución
El system se ejecuta en la fase del fotograma equivocada	Falta [UpdateInGroup] o es incorrecto. Por defecto es SimulationSystemGroup.
Se ignora [UpdateAfter]	El system de destino está en otro grupo. Los dos deben estar en el mismo grupo.
Error de dependencia circular al inicio	Una cadena de [UpdateBefore] / [UpdateAfter] forma un ciclo. La ventana Systems imprime el ciclo.
El system se ejecuta varias veces por fotograma sin esperarlo	Está en FixedStepSimulationSystemGroup, un grupo que recupera el tiempo acumulado. Si no debe hacerlo, moverlo a SimulationSystemGroup.
El grupo personalizado se ejecuta indefinidamente	El override de OnUpdate nunca llamó a base.OnUpdate().
El system se ejecuta pero no hace nada	No se cumple RequireForUpdate<T>, o todas las queries están vacías. Comprobarlo en Window → Entities → Query .

Capítulo 13. JobSystem & Burst

1. Descripción general

El rendimiento de DOTS se asienta sobre dos funciones del motor que ya existían antes de Entities y que siguen siendo su base:

- **Job System:** reparte trabajo entre los hilos de trabajo mediante grafos de dependencias.
- **Burst Compiler:** compila jobs de C# a código nativo muy compacto (SIMD, sin GC, sin despacho virtual).

Ninguno de los dos es un concepto propio de Entities. IJobEntity e IJobChunk son capas finas sobre IJob y Burst. Entender la capa base hace que todos los patrones de más alto nivel se lean con claridad.

2. Job System — los tipos fundamentales

Interfaz	Forma	Usar cuando...
IJob	void Execute()	Trabajo puntual en un solo hilo.
IJobParallelFor	void Execute(int index)	Bucle de 0 a N en paralelo.
IJobParallelForBatch	void Execute(int start, int count)	Bucle en lotes paralelos (aprovecha mejor la caché que el modo por índice).
IJobChunk	void Execute(ArchetypeChunk chunk, int chunkIndex, bool useEnabledMask, in v128 enabledMask)	Iterar chunks ECS en paralelo.
IJobEntity	void Execute([component params])	Iteración de entidades ECS; el source generator lo convierte en un chunk job.

Todos son **structs** y todos son compatibles con Burst.

Ejemplo mínimo de IJobParallelFor

```
using Unity.Burst;
using Unity.Collections;
```

```
using Unity.Jobs;

[BurstCompile]
struct ScaleJob : IJobParallelFor
{
    public NativeArray<float> Data;
    public float Factor;

    public void Execute(int index) => Data[index] *= Factor;
}

// Scheduling:
var handle = new ScaleJob { Data = arr, Factor = 2f }.Schedule(arr.Length, 64);
handle.Complete();
```

Notas:

- Los campos deben ser blittables.
- Escribir en el mismo NativeArray desde varios jobs exige encadenar las dependencias (véase más abajo).

3. Programación y dependencias

```
JobHandle a = jobA.Schedule();
JobHandle b = jobB.Schedule(a);           // b runs after a
JobHandle c = jobC.Schedule(JobHandle.CombineDependencies(a, b));
c.Complete();
```

Reglas:

- Cada `Schedule()` devuelve un `JobHandle`. Si el nuevo job lee o escribe datos compartidos, hay que pasarle el handle anterior como dependencia.
- **Nunca hay que omitir `.Complete()`** sobre el handle final antes de que el hilo principal lea los resultados (el Safety System lo exige).
- En un `ISystem`, `state.Dependency` es el handle del system; los jobs programados se combinan con él de forma automática al usar `.Schedule(state.Dependency)` o helpers generados por código como `IJobEntity.ScheduleParallel(...)`.

Dentro de un ISystem

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
{
    var job = new MyJob { Dt = SystemAPI.Time.DeltaTime };
    state.Dependency = job.ScheduleParallel(state.Dependency);
}
```

`IJobEntity.ScheduleParallel()`, sin dependencia explícita, se combina de forma implícita con `state.Dependency`. Al encadenar varios jobs conviene indicar la dependencia de forma explícita.

4. Burst — qué es y por qué

Burst compila un subconjunto de C# (métodos marcados con `[BurstCompile]`, sin asignaciones de GC, con datos blittables) a código nativo muy optimizado mediante LLVM.

Qué compila bien Burst

- Métodos de struct `IJob*` / `ISystem` etiquetados con `[BurstCompile]`.
- Cálculos con tipos de `Unity.Mathematics` (`float3`, `quaternion`, `int4`, etc.).
- Bucles compactos sobre `NativeArray<T>`, `NativeList<T>`, `DynamicBuffer<T>` o los arrays de columna de `ArchetypeChunk`.

Qué no puede compilar Burst

- Referencias gestionadas (`class`, `string`, `List<T>` o cualquier cosa que no sea `unmanaged`).
- Excepciones (se pueden capturar, pero en general conviene evitarlas).
- Despacho dinámico a través de interfaces (la instanciación genérica explícita sí es válida).
- `System.IO`, `Reflection` y la mayor parte del BCL más allá de las matemáticas y los tipos primitivos.

Activar Burst

```
using Unity.Burst;

[BurstCompile]
public partial struct MyJob : IJobEntity
{
    // attribute on the struct marks the type for Burst
    // methods need [BurstCompile] too if you want each compiled
    [BurstCompile]
    void Execute(ref Velocity v) { /* ... */ }
}
```

Para systems:

```
[BurstCompile]
public partial struct MySystem : ISystem
{
    [BurstCompile] public void OnCreate(ref SystemState state) { }
```

```
[BurstCompile] public void OnUpdate(ref SystemState state) { }
}
```

El atributo en el tipo **no** basta: cada método que se quiera compilar necesita además su propio `[BurstCompile]`.

Burst Inspector

Window → **Analysis** → **Burst Inspector** muestra, por método, el IR de LLVM, el ensamblador y el uso de vectores. Es la herramienta para responder a la pregunta de "por qué esto no va rápido".

5. Contenedores nativos

Burst y el Safety System esperan **contenedores nativos**, no colecciones gestionadas:

Gestionado	Equivalente nativo
<code>T[]</code>	<code>NativeArray<T></code>
<code>List<T></code>	<code>NativeList<T></code>
<code>Dictionary<K,V></code>	<code>NativeHashMap<K,V></code> , <code>NativeParallelHashMap<K,V></code>
<code>HashSet<T></code>	<code>NativeHashSet<T></code> , <code>NativeParallelHashSet<T></code>
<code>Queue<T></code>	<code>NativeQueue<T></code>
<code>string</code>	<code>FixedString32Bytes</code> / <code>FixedString64Bytes</code> / etc.

Reserva de memoria:

```
var arr = new NativeArray<int>(1024, Allocator.TempJob);
// ...
arr.Dispose();
```

La vida útil importa: Burst y el Safety System verifican que los contenedores se liberen y que no se acceda a ellos una vez terminado el job.

6. Allocators

Allocator	Vida útil	Uso típico
<code>Allocator.Temp</code>	Dentro del fotograma actual y solo en el hilo principal	Contenedores temporales en <code>OnUpdate</code>
<code>Allocator.TempJob</code>	Lo que dura un job, hasta 4 fotogramas	Contenedores que se pasan a los jobs
<code>Allocator.Persistent</code>	Requiere <code>Dispose()</code> manual	Contenedores del propio system que persisten entre fotogramas
<code>state.WorldUpdateAllocator</code>	Se vacía al final de la actualización del system group	La opción preferida para datos transitorios de cada system

Dentro de los systems es preferible `state.WorldUpdateAllocator` (o su allocator `Rewindable`): evita tener que gestionar los `Dispose()` a mano.

7. Safety System

En el Editor con "Jobs > Leak Detection" y "Jobs > Safety Checks" activados, Unity valida que:

- Dos jobs no escriben en el mismo contenedor nativo sin una arista de dependencia.
- Una lectura del hilo principal no entra en conflicto con una escritura en curso.
- Los contenedores nativos no se usan tras un `Dispose()` ni más allá de la vida útil de su allocator.

Los errores de seguridad aparecen como excepciones con un mensaje claro del tipo "el job A y el job B acceden los dos a X". **No conviene desactivar las comprobaciones de seguridad durante el desarrollo:** la ganancia de rendimiento es pequeña y el coste de depuración, alto.

8. Solución de problemas

Síntoma	Causa / Solución
Advertencia de Burst: "method not burst compiled"	Falta <code>[BurstCompile]</code> en el método, o Burst se topó con un tipo no compatible (revisar en el Burst Inspector la línea que falla).
<code>InvalidOperationException: ... writes to ... without declaring dependency</code>	Falta una arista de dependencia: pasar el <code>JobHandle</code> anterior o encadenar mediante <code>state.Dependency</code> .
<code>ObjectDisposedException</code> en un <code>NativeArray</code>	Se liberó antes de que terminara el job. Usar <code>arr.Dispose(handle)</code> o esperar antes a que el handle termine.

Síntoma	Causa / Solución
El job programado nunca termina	Falta <code>.Complete()</code> en una cadena, o hay una dependencia circular.
Regresión de rendimiento tras añadir <code>[BurstCompile]</code>	El método se recompila en cada recarga de dominio; comprobar la caché de Burst. O se ha colado un tipo incompatible con Burst y el método ha vuelto en silencio a ejecutarse como IL.
La enumeración de <code>NativeHashMap</code> lanza excepción	La enumeración no se lleva bien con las escrituras concurrentes; en su lugar, enumerar una copia en un <code>NativeArray<K></code> .
El hilo principal se queda bloqueado esperando un job	Se llamó a <code>.Complete()</code> demasiado pronto. Llevar el punto de finalización lo más tarde posible dentro del fotograma.

Capítulo 14. IJobEntity · SystemAPI.Query

1. Descripción general

Dos APIs cubren el 95% de la iteración de componentes en Entities 6.5:

API	Dónde se ejecuta	Usar cuando...
SystemAPI.Query<...>()	Hilo principal, dentro de OnUpdate	El trabajo es trivial (unos cientos de entidades) o requiere APIs del hilo principal.
IJobEntity	Hilos de trabajo, mediante Schedule / ScheduleParallel	El trabajo es más pesado o se beneficia del paralelismo.

Son la alternativa recomendada al antiguo `Entities.ForEach` (marcado como obsoleto en Entities 1.4; en 6.5 todavía compila, con advertencias de deprecación; el changelog de Unity indica que su eliminación está prevista para una futura versión mayor).

2. SystemAPI.Query<T>

Recorre en el hilo principal las entidades que coinciden. A los componentes se accede mediante `RefRO<T>` o `RefRW<T>`.

```
[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (health, regen) in
            SystemAPI.Query<RefRW<Health>, RefRO<Regen>>())
        {
            health.ValueRW.Value = math.min(
                health.ValueRO.Value + regen.ValueRO.Rate * dt,
                health.ValueRW.Max);
        }
    }
}
```

Reglas:

- El struct del system debe ser `partial`.
- `RefRO<T>` para solo lectura y `RefRW<T>` para lectura-escritura. No se debe escribir a través de `RefRO`.
- Si se necesita el id, basta con añadir una `Entity` a la tupla: `SystemAPI.Query<RefRW<Health>>().WithEntityAccess()`.

Filtros

```
foreach (var health in SystemAPI.Query<RefRW<Health>>())
    .WithAll<Enemy>()
    .WithNone<Dead>()
    .WithAny<Burning, Poisoned>())
{
    // only entities that are Enemy AND not Dead AND (Burning OR Poisoned)
}
```

- `WithAll<T>()`: debe tener todos los componentes indicados.
- `WithNone<T>()`: no debe tener ninguno de ellos.
- `WithAny<T...>()`: debe tener al menos uno de ellos.
- `WithChangeFilter<T>()`: solo los chunks donde el componente T ha cambiado desde la última ejecución.
- `WithEntityAccess()`: devuelve también el id de `Entity`.

3. IJobEntity

Una iteración paralela creada por el source generator. La firma de `Execute` declara con qué componentes se trabaja.

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;

[BurstCompile]
public partial struct HealthRegenJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Regen regen)
    {
        health.Value = math.min(health.Value + regen.Rate * DeltaTime, health.Max);
    }
}

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
```

```
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new HealthRegenJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```

Reglas de la firma:

- `void Execute(...)`: cada parámetro se corresponde uno a uno con un componente.
- `ref Component`: lectura-escritura. `in Component`: solo lectura (no se puede escribir).
- Se permite `Entity entity` como parámetro: el source generator lo rellena.
- El struct del job debe ser `partial`.

Variantes de Schedule

Llamada	Hilos
<code>.Run()</code>	Hilo principal; termina antes de devolver el control. Útil para depurar o para cargas de trabajo pequeñas.
<code>.Schedule()</code>	Un solo hilo de trabajo.
<code>.ScheduleParallel()</code>	Reparte los chunks entre los hilos de trabajo.

Los tres actualizan `state.Dependency` de forma automática dentro de `ISystem.OnUpdate`.

Filtros en jobs

```
new HealthRegenJob { DeltaTime = dt }
    .ScheduleParallel(SystemAPI.QueryBuilder()
        .WithAll<Enemy, Regen>()
        .WithNone<Dead>()
        .Build());
```

O con los atributos `[WithAll]`, `[WithNone]` y `[WithAny]` sobre el propio struct del job.

Acceso a entidad en un job

```
void Execute(Entity entity, ref Health health) { /* ... */ }
```

Índice de chunk / `[ChunkIndexInQuery]`

Resulta útil al combinarlo con un ECB de `ParallelWriter` para una reproducción determinista: se pasa el índice de chunk como `sortKey`. Véase [15_ParallelWriter · Deterministic Playback.md](#).

```
void Execute([ChunkIndexInQuery] int chunkIndex, in Health health, Entity entity)
{
    if (health.Value <= 0)
        ecb.DestroyEntity(chunkIndex, entity);
}
```

4. Cuándo usar cada uno

Situación	Elegir
De decenas a cientos de entidades, con cálculos triviales	<code>SystemAPI.Query</code>
Miles de entidades o más, con trabajo independiente por entidad	<code>IJobEntity.ScheduleParallel</code>
Trabajo por chunk (bounds, sumas, partición espacial)	IJobChunk — véase 12_IJobEntity vs IJobChunk.md
Hace falta usar una API gestionada dentro del bucle	<code>SystemAPI.Query</code> desde un <code>SystemBase</code>
Hacen falta cambios estructurales diferidos	Cualquiera de las dos formas, pero escribiendo mediante <code>EntityCommandBuffer.ParallelWriter</code> ; véase 14_EntityCommandBuffer · Deferred Entity.md

5. Puntos a tener en cuenta

- **No capturar `this` ni campos gestionados dentro de un job.** El source generator da error si se intenta.
- **`RefRW<T>.ValueRW` activa el seguimiento de cambios.** Leer a través de `ValueRW` sin escribir igualmente marca el chunk como sucio.
- **`WithChangeFilter<T>` trabaja a nivel de chunk.** Si cualquier entidad del chunk escribe en `T`, el chunk entero se considera modificado.
- **Los Aspects están obsoletos (aunque no eliminados).** La interfaz de agrupación `IAAspect` se deprecó en la 1.4 y todavía figura como tipo de parámetro válido de `Execute` en la referencia de la API 6.5. El código nuevo debe usar parámetros de componente normales; los aspects ya existentes siguen compilando en 6.5.

6. Solución de problemas

Síntoma	Causa / Solución
Error de compilación: IJobEntity must be partial	Añadir <code>partial</code> al struct del job.
ScheduleParallel lanza un error de condición de carrera	Dos jobs escriben en el mismo componente sin una arista de orden. Comprobar que <code>state.Dependency</code> se combine bien, o añadir <code>[UpdateAfter]</code> entre los systems.
El job no se ejecuta con Burst	Falta <code>[BurstCompile]</code> en el struct del job, o se captura un tipo incompatible con Burst (una referencia gestionada, o un <code>string</code> en lugar de un <code>FixedString</code>).
SystemAPI.Query no devuelve nada	La combinación de filtros está vacía. Inspeccionar la query en Window → Entities → Query para ver qué coincide realmente.
Escribir mediante RefRO (bug silencioso)	El compilador ya da error ante <code>refRO.ValueRW =</code> . Si se cuela con versiones antiguas del source generator, cambiar a <code>RefRW</code> .
El recuento de entidades no coincide entre SystemAPI.Query y una EntityQuery creada a mano	Los filtros son distintos. SystemAPI.Query incluye por defecto el filtrado de enableable components; EntityQueryBuilder hay que configurarlo de forma explícita.

Capítulo 15. IJobEntity vs IJobChunk

1. La respuesta corta

- **IJobEntity**: para "hacer lo mismo con cada entidad".
- **IJobChunk**: para "hacer en cada chunk algo que no sea un simple mapeo por entidad".

IJobEntity se genera sobre IJobChunk: ambos usan la misma maquinaria de iteración. IJobChunk se limita a exponer el chunk completo al código.

2. IJobEntity — ergonomía por entidad

```
[BurstCompile]
public partial struct DamageTickJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Burn burn)
    {
        health.Value -= burn.DamagePerSecond * DeltaTime;
    }
}
```

Ventajas:

- Un método y una vista por entidad. Fácil de escribir y fácil de leer.
- Toda la infraestructura la genera el source generator: los parámetros Entity, [ChunkIndexInQuery] y [EntityIndexInQuery] se rellenan solos.
- Filtros mediante los atributos [WithAll], [WithNone] y [WithAny] sobre el struct, o con un QueryBuilder que se pasa a .Schedule*.

Desventajas:

- No hay visibilidad entre entidades dentro de Execute. Sin estado adicional, no es fácil "sumar todo el chunk".
 - Desde Execute no se accede a los metadatos del chunk (la versión de cambio, el valor del componente compartido).
-

3. IJobChunk — control completo del chunk

```
[BurstCompile]
public partial struct HealthAggregateJob : IJobChunk
{
    [ReadOnly] public ComponentTypeHandle<Health> HealthHandle;
    public NativeArray<float> PerChunkTotal; // size = chunk count

    public void Execute(in ArchetypeChunk chunk,
                        int unfilteredChunkIndex,
                        bool useEnabledMask,
                        in v128 chunkEnabledMask)
    {
        var healths = chunk.GetNativeArray(ref HealthHandle);
        float total = 0f;
        for (int i = 0; i < chunk.Count; i++)
            total += healths[i].Value;

        PerChunkTotal[unfilteredChunkIndex] = total;
    }
}
```

Ventajas:

- Acceso directo a los arrays del chunk: `chunk.GetNativeArray(ref handle)`, `chunk.GetBufferAccessor(ref bufferHandle)`.
- `chunk.Has<T>()` y `chunk.GetSharedComponent<T>(...)` para trabajo condicional.
- Ve la máscara de habilitados y la versión de cambio de cada chunk.
- Ideal para reducciones, partición espacial y registro de ECB por lotes.

Coste:

- Más código repetitivo: hay que declarar los `ComponentTypeHandle<T>` en el system y actualizarlos cada fotograma con `handle.Update(ref state)`.
- El bucle for interno lo gestiona el propio código, así que hay más sitios donde equivocarse.

Configurar los handles en un system

```
[BurstCompile]
public partial struct HealthAggregateSystem : ISystem
{
    private EntityQuery _query;
    private ComponentTypeHandle<Health> _healthHandle;

    public void OnCreate(ref SystemState state)
    {
        _query = state.GetEntityQuery(ComponentType.ReadOnly<Health>());
        _healthHandle = state.GetComponentTypeHandle<Health>(true);
    }
}
```

```

public void OnUpdate(ref SystemState state)
{
    _healthHandle.Update(ref state);

    var totals = CollectionHelper.CreateNativeArray<float>(
        _query.CalculateChunkCount(), state.WorldUpdateAllocator);

    state.Dependency = new HealthAggregateJob
    {
        HealthHandle = _healthHandle,
        PerChunkTotal = totals
    }.ScheduleParallel(_query, state.Dependency);
}

```

handle.Update(ref state) refresca cada fotograma el handle de tipo cacheado; si se olvida, Burst acaba usando índices obsoletos.

4. Máscara de habilitados (useEnabledMask)

En las queries que incluyen un IEnableableComponent, Execute recibe useEnabledMask = true y una máscara v128 con los bits de habilitado. Hay que respetar la máscara o se procesarán entidades desactivadas.

Hay un helper para ello:

```

var enumerator = new ChunkEntityEnumerator(useEnabledMask, chunkEnabledMask, chunk.Count);
while (enumerator.NextEntityIndex(out int i))
{
    // i is the index of the next enabled entity in the chunk
}

```

IJobEntity aplica la máscara de forma automática; es otra razón para preferirlo salvo que se necesite visibilidad a nivel de chunk.

5. Guía de decisión

Situación	Elegir
Actualizar la salud, trasladar la posición, aplicar el input: cálculos puros por entidad	IJobEntity
Sumar los valores de todas las entidades de un chunk	IJobChunk
Registrar un comando ECB por chunk en lugar de por entidad	IJobChunk

Situación	Elegir
Acceder a un valor de componente compartido dentro del bucle	IJobChunk
Se necesita la versión de cambio del chunk	IJobChunk
Iterar un <code>DynamicBuffer<T></code> en cada entidad	Cualquiera funciona; IJobEntity es más limpio
Lógica por entidad, pero se necesita el índice de chunk de la entidad	IJobEntity con <code>[ChunkIndexInQuery] int chunkIndex</code>
El coste de programación importa y el trabajo es pequeño	Conviene medir: reducir el número de chunks compensa más que reducir el de entidades. Con miles de chunks suele ganar IJobChunk.

6. Interoperabilidad — usar ambos juntos

Un patrón habitual: recopilar datos por chunk con IJobChunk y, después, escribir por entidad con IJobEntity, encadenando ambos mediante dependencias.

```
var gather = new HealthAggregateJob { ... }
    .ScheduleParallel(query, state.Dependency);

var act = new HealJob { PerChunkTotal = totals }
    .ScheduleParallel(gather);

state.Dependency = act;
```

7. Solución de problemas

Síntoma	Causa / Solución
IJobChunk.Execute lee valores basura	Se olvidó <code>handle.Update(ref state)</code> antes de programar el job, o se usó la query equivocada.
No se encuentra <code>ComponentTypeHandle<T></code>	Falta <code>using Unity.Entities;</code> , o el handle se declaró con el modo de acceso equivocado. Para solo lectura, usar <code>GetComponentTypeHandle<T>(true)</code> .
Se omiten entidades habilitadas	No se respetó la máscara; usar <code>ChunkEntityEnumerator</code> como se ha mostrado. IJobEntity evita este problema.
El job escribe en el mismo <code>NativeArray</code> desde dos chunks	Reservar un slot por chunk (como en <code>HealthAggregateJob</code>) o usar un <code>NativeParallelHashMap</code> con el índice de chunk como clave.

Síntoma	Causa / Solución
Programar un IJobChunk sin el argumento de query	ScheduleParallel(query, ...) exige indicar la query de forma explícita. IJobEntity.ScheduleParallel() la obtiene de los atributos o del source generator.
No se encuentra el tipo v128	Falta using Unity.Burst.Intrinsics;

Capítulo 16. Cambio estructural y seguridad

1. Qué cuenta como cambio estructural

Un **cambio estructural** es cualquier operación capaz de mover entidades entre chunks o de alterar el inventario de chunks. Todas estas son estructurales:

Operación	Por qué es estructural
CreateEntity / DestroyEntity	Reserva o libera un slot dentro de un chunk.
AddComponent<T> / RemoveComponent<T>	El archetype de la entidad cambia, así que se traslada físicamente a otro chunk.
SetSharedComponent<T>	Un valor distinto de componente compartido implica otro chunk (se agrupa por valor).
AddChunkComponent<T> / RemoveChunkComponent<T>	Cambia los metadatos propios del chunk.
SetName(entity, ...)	Modifica los metadatos de la entidad en el EntityManager.
MoveEntitiesFrom	Copia entidades entre worlds.
Instantiate(prefab)	Reserva una entidad nueva (puede pre-rellenar un archetype).

Escribir el **valor** de un componente con SetComponentData o RefRW<T>.ValueRW **no** es estructural: el componente ya está en el chunk y solo se cambian unos bytes.

2. Por qué los cambios estructurales son peligrosos

Cuando el archetype cambia, todo lo que mantenga una referencia posicional dentro de un chunk queda obsoleto:

- Los iteradores de query en curso (SystemAPI.Query<...>) pueden saltarse entidades o volver a visitarlas.
- Las entradas de ComponentLookup<T> dejan de ser válidas para las entidades afectadas.
- Los handles de ArchetypeChunk de un job apuntan al chunk **antiguo**.
- Otros systems que se ejecutan a la vez ven un estado inconsistente.

El Safety System detecta la mayoría de estos casos con una `InvalidOperationException` en el Editor. En una build se traducen en crashes o entidades corruptas.

3. Regla general

No realizar cambios estructurales mientras se itera una query en la misma actualización del system.

Dos patrones válidos, en orden de preferencia:

1. **Usar un `EntityCommandBuffer`.** Se registra la operación durante la iteración y se reproduce después. Véase `14_EntityCommandBuffer · Deferred Entity.md`.
2. **Volcar las entidades en una `NativeList<Entity>` y aplicar los cambios tras el bucle.** Más sencillo para casos pequeños y sin la sobrecarga de un ECB.

Ejemplo del patrón 2:

```
var toKill = new NativeList<Entity>(state.WorldUpdateAllocator);

foreach (var (health, e) in SystemAPI
    .Query<RefRO<Health>>()
    .WithEntityAccess())
{
    if (health.ValueRO.Value <= 0f)
        toKill.Add(e);
}

state.EntityManager.DestroyEntity(toKill.AsArray());
```

4. Enableable components — la vía de escape

Si se acaba añadiendo y quitando un componente en cada fotograma, casi con seguridad debería ser un **enableable component**: conmutar el bit de habilitado **no** es estructural. Véase `06_Enableable Component.md`.

5. Jobs y cambios estructurales

Los cambios estructurales son **operaciones del hilo principal**: no se pueden hacer dentro de un job.

Las formas idiomáticas de mutar desde un job:

Enfoque	Cuándo
Un <code>EntityCommandBuffer.ParallelWriter</code> que se registra en el job y se reproduce en el hilo principal	Cambios estructurales por entidad desde código paralelo.
Un <code>EntityCommandBuffer</code> (de un solo hilo) que se registra en un <code>IJob</code> o <code>IJobChunk</code> y se reproduce en el hilo principal	Cambios por lotes o con alcance de chunk.
Conmutar un enableable component mediante <code>EnabledRefRW<T></code> dentro de un job	Activar o desactivar por entidad sin cambios estructurales.

Un ECB guarda la operación como un comando; la reproducción se produce en un punto bien definido del hilo principal, normalmente en un límite de `EntityCommandBufferSystem`.

6. Combinar cambios estructurales y lecturas

Si dos systems se ejecutan uno tras otro y el system A hace un cambio estructural del que depende el system B, conviene fijar el orden de forma explícita:

```
[UpdateInGroup(typeof(SimulationSystemGroup))]  
[UpdateBefore(typeof(MovementSystem))]  
public partial struct SpawnSystem : ISystem { /* uses ECB */ }
```

Cuando la reproducción del ECB system queda entre A y B (por ejemplo, `EndSimulationECBS` con una cadena de `[UpdateAfter]` bien planteada), el cambio estructural se aplica antes de que B lea.

7. Modelo de coste

Orden de magnitud aproximado para un único cambio:

<code>SetComponentData (value only)</code>	~ 1x
<code>SetComponentEnabled (enableable bit)</code>	~ 1x (per-chunk bitmask write)
<code>AddComponent (structural)</code>	~ 10x (entity moves chunk)
<code>SetSharedComponent (structural)</code>	~ 10x
<code>DestroyEntity (structural)</code>	~ 10x
Per-frame add + remove pattern	pathological

Son órdenes de magnitud; las cifras exactas dependen del tamaño del archetype, de la ocupación del chunk y de la presión sobre el allocator.

8. Diagnosticar el desgaste estructural

Síntomas:

- En Window → Entities → Systems, un system consume un tiempo desproporcionado para la lógica que tiene.
- El marcador del Profiler EntityManager.SetArchetype o Move Entity acapara los fotogramas.
- El número de archetypes crece con el tiempo.

Tácticas:

- Revisar las llamadas a AddComponent y RemoveComponent: ¿hay alguna por fotograma? Sustituirlas por enableable components.
 - Examinar los conjuntos de valores de los componentes compartidos: ¿hay alguna clave que sea por entidad sin querer?
 - Buscar en la ventana Archetypes chunks con una sola entidad; eso es fragmentación por conjuntos de componentes demasiado granulares.
-

9. Solución de problemas

Síntoma	Causa / Solución
InvalidOperationException: EntityCommandBuffer has already been played back	Se grabó en un ECB después de reproducirlo. Cada ECB es de un solo uso: hay que pedir uno nuevo al ECBS en cada fotograma.
InvalidOperationException: ... invalidated ... durante un foreach	Hay un cambio estructural dentro del bucle. Pasarlo a un ECB o usar el patrón de almacenar primero y aplicar después.
Las entidades desaparecen sin avisar	Se ejecutó un DestroyEntity, pero otros systems siguen guardando el handle antiguo de Entity. Comprobar los handles con EntityManager.Exists.
El patrón de añadir y quitar consume tiempo de fotograma	Sustituirlo por enableable components.
RemoveComponent provoca el error "cleanup component present"	La entidad tiene un cleanup component. Procesar primero la limpieza (véase 05_Component Types.md).

Síntoma	Causa / Solución
El conjunto de valores del componente compartido se dispara (cientos de valores distintos)	El valor es demasiado granular. Agruparlo en intervalos (por ejemplo, redondear al N más cercano) o guardar la parte variable como un componente normal.

Capítulo 17. EntityCommandBuffer · Entidad diferida

1. Por qué existe el ECB

EntityCommandBuffer (ECB) registra cambios estructurales (crear, destruir, añadir, quitar, asignar) y los reproduce más tarde en el hilo principal, en un punto seguro. Permite modificar el world desde:

- Dentro de un foreach de una query (donde una llamada directa a EntityManager invalidaría el iterador).
- Un job (donde no se permiten cambios estructurales del hilo principal).
- Un job paralelo que abarca muchos chunks (mediante ParallelWriter).

El contexto de por qué esto es peligroso está en 13_Structural Change & Safety.md.

2. El ciclo de vida del ECB

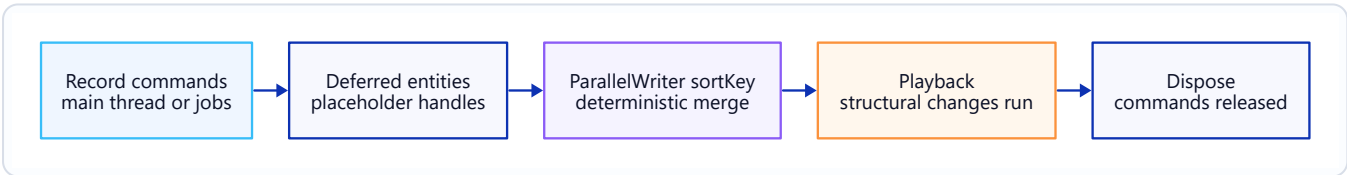


Figure: EntityCommandBuffer Lifecycle.

Un ECB es de **un solo uso**: tras la reproducción se libera. En cada fotograma se pide uno nuevo al EntityCommandBufferSystem adecuado.

3. ECB systems predefinidos

System	Cuándo reproduce	Uso típico
BeginInitializationEntityCommandBufferSystem	Inicio del grupo Initialization	Preparación temprana del world a partir del fotograma anterior
EndInitializationEntityCommandBufferSystem	Fin del grupo Initialization	—

System	Cuándo reproduce	Uso típico
BeginSimulationEntityCommandBufferSystem	Inicio de Simulation	Aplicar los cambios pendientes antes de que corra la lógica de juego
EndSimulationEntityCommandBufferSystem	Fin de Simulation	Opción por defecto para creaciones y destrucciones que dirige la lógica de juego
BeginPresentationEntityCommandBufferSystem	Inicio de Presentation	Poco común; configuración solo visual

Elegir el adecuado determina **cuándo verán los cambios los demás systems**. Si se crea un proyectil en `SimulationSystemGroup` y se quiere que se mueva **en ese mismo fotograma**, hay que grabar en `BeginSimulation` del fotograma siguiente; o en `EndSimulation` si vale con el fotograma siguiente.

4. Obtener un ECB

Dentro de cualquier `ISystem.OnUpdate`:

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged);
```

El tipo anidado `.Singleton` es un token, uno por system, que el source generator expone para cada ECB system predefinido. `CreateCommandBuffer` devuelve un ECB nuevo que se reproducirá cuando ese system se ejecute.

Para jobs paralelos:

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();
```

El contrato de la reproducción paralela está en `15_ParallelWriter · Deterministic Playback.md`.

5. Grabación — creación de entidades

```
Entity e = ecb.CreateEntity();
ecb.AddComponent(e, new Health { Value = 100 });
ecb.AddComponent(e, LocalTransform.FromPosition(0, 0, 0));
```

O crear una instancia a partir de una entidad prefab:

```
Entity inst = ecb.Instantiate(prefab);
ecb.SetComponent(inst, LocalTransform.FromPosition(spawnPos));
```

Tanto `CreateEntity` como `Instantiate` devuelven una **entidad diferida** (véase §6).

6. Entidades diferidas

La entidad que devuelven `ecb.CreateEntity()` o `ecb.Instantiate()` **aún no existe**: tiene un índice negativo provisional. Con ella se puede:

- Pasarla a llamadas posteriores del **mismo ECB**: `AddComponent`, `SetComponent` o `AppendToBuffer`.
- Guardarla en otro componente mediante `ecb.SetComponent(other, new ReferenceTo { Entity = inst })`.

No se puede:

- Leerla ni escribir en ella mediante `EntityManager` antes de que el ECB se reproduzca.
- Pasarla entre ECBs distintos (el valor provisional solo se resuelve dentro del ECB que la creó).

Durante la reproducción, el ECB system sustituye el valor provisional por el handle real de Entity en todos los puntos donde se utilizó.

7. Grabación — cambios estructurales

```
ecb.AddComponent(entity, new Stunned { Duration = 2f });
ecb.RemoveComponent<Stunned>(entity);
ecb.SetComponent(entity, new Health { Value = 50 });

ecb.SetComponentEnabled<Stunned>(entity, false);

var buf = ecb.AddBuffer<Waypoint>(entity);
buf.Add(new Waypoint { Position = new float3(1, 0, 0) });
ecb.AppendToBuffer(entity, new Waypoint { Position = new float3(2, 0, 0) });

ecb.DestroyEntity(entity);
```

Todos quedan en cola como comandos; ninguno surte efecto hasta la reproducción.

8. ECB desde un IJobEntity

```
[BurstCompile]
public partial struct KillOnHealthZeroJob : IJobEntity
{
    public EntityCommandBuffer ECB;

    void Execute(Entity entity, in Health health)
    {
        if (health.Value <= 0)
            ECB.DestroyEntity(entity);
    }
}

// In OnUpdate:
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged);

new KillOnHealthZeroJob { ECB = ecb }.Schedule();
```

Para `.ScheduleParallel()` hay que cambiar a `.AsParallelWriter()` y propagar un `sortKey`; se explica en la página siguiente.

9. ECBs independientes (manuales)

A veces hace falta controlar la reproducción a mano, por ejemplo dentro de un solo `OnUpdate`:

```
var ecb = new EntityCommandBuffer(Allocator.Temp);
// ... record ...
ecb.Playback(state.EntityManager);
ecb.Dispose();
```

Es la opción para patrones de modificación autónomos y puntuales. Para todo lo que cruce los límites de un system o se ejecute en un job, usar los ECBS predefinidos anteriores.

10. Determinismo

En los ECB de escritor único, los comandos se reproducen en el **orden en que se grabaron**. En un `ParallelWriter`, el orden de reproducción depende del `sortKey` que se indique; véase `15_ParallelWriter · Deterministic Playback.md`.

En sistemas de Netcode o de replay, la reproducción determinista es innegociable. Hay que pasar siempre una clave de ordenación estable (índice de chunk, índice dentro de la query) y evitar grabar en rutas de código del hilo principal que se ejecuten en un orden no determinista.

11. Solución de problemas

Síntoma	Causa / Solución
InvalidOperationException: ECB has already been played back	Se reutilizó entre fotogramas. Obtener un ECB nuevo en cada OnUpdate con CreateCommandBuffer.
El handle de entidad diferida es Entity.Null tras la reproducción	Se referenció desde dos ECBs distintos. Mantener tanto la creación como el SetComponent posterior en el mismo ECB.
La entidad creada no aparece en este fotograma	Se grabó en el ECBS de EndSimulation, cuya reproducción es al final del fotograma. Cambiar a BeginSimulation del fotograma siguiente para verla antes que la mayoría de los systems.
ecb.SetComponent lanza "component does not exist"	La entidad de destino aún no tiene el componente (añadirlo antes con AddComponent) o todavía no se ha producido la reproducción.
Se llama a Dispose() en un ECB gestionado por ECBS	No llamar a Dispose() en los ECB obtenidos de CreateCommandBuffer: de su vida útil se encarga el ECBS. Solo se liberan los independientes (new EntityCommandBuffer(Allocator.Temp)).
La reproducción del ECB paralelo reordena los comandos	El sortKey no es determinista. Usar [ChunkIndexInQuery].

Capítulo 18. ParallelWriter · Reproducción determinista

1. Por qué existe un writer paralelo separado

EntityCommandBuffer es de escritor único. Si dos hilos grabaran en el mismo ECB, competirían por su cadena interna. EntityCommandBuffer.ParallelWriter lo resuelve dando a cada llamador una **cadena propia por hilo** y exigiendo una **clave de ordenación** en cada comando.

En el momento de la reproducción, el ECB system:

1. Aplana todas las cadenas por hilo en una sola.
2. Ordena los comandos por sortKey (con una ordenación estable).
3. Los ejecuta en el hilo principal.

El resultado es determinista mientras las claves de ordenación lo sean.

2. Obtener un ParallelWriter

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();
```

AsParallelWriter() devuelve un struct EntityCommandBuffer.ParallelWriter que se puede copiar dentro de un IJobEntity, un IJobChunk o un IJobParallelFor.

3. El contrato de sortKey

Cada llamada de un ECB paralelo recibe un sortKey como primer argumento:

```
ecb.Instantiate(sortKey, prefab);
ecb.SetComponent(sortKey, entity, value);
ecb.DestroyEntity(sortKey, entity);
ecb.AddComponent(sortKey, entity, component);
```

Reglas para una **buena** clave de ordenación:

- **Determinista.** La misma entrada produce la misma clave, sin importar cómo reparta el trabajo el planificador.
- **Suficientemente única.** Dos comandos sobre la misma entidad no deberían colisionar, salvo que se quiera un orden de reproducción ambiguo.
- **Barata.** Un int del que ya se dispone, no un hash.

En la práctica, las opciones preferidas son:

Fuente	Cómo obtenerla	Cuándo usar
[ChunkIndexInQuery] en IJobEntity	La rellena el source generator	El 99% de los casos
unfilteredChunkIndex en IJobChunk.Execute	Lo proporciona la API	Jobs de IJobChunk
[EntityIndexInQuery] en IJobEntity	Source generator	Cuando los comandos de cada entidad deben ordenarse uno a uno
Un índice determinista calculado manualmente	Lo calcula el propio código	Poco habitual; estructuras de job a medida

4. Ejemplo completo — spawn paralelo

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct SpawnParallelJob : IJobEntity
{
    public float DeltaTime;
    public EntityCommandBuffer.ParallelWriter ECB;

    void Execute([ChunkIndexInQuery] int chunkIndex,
                Entity spawnerEntity,
                ref Spawner spawner)
    {
        if (spawner.RemainingCount <= 0)
        {
            ECB.DestroyEntity(chunkIndex, spawnerEntity);
            return;
        }

        spawner.TimeLeft -= DeltaTime;
        if (spawner.TimeLeft > 0f) return;

        spawner.TimeLeft = spawner.Interval;
        spawner.RemainingCount--;
    }
}
```

```

        var inst = ECB.Instantiate(chunkIndex, spawner.Prefab);
        ECB.SetComponent(chunkIndex, inst, LocalTransform.FromPosition(spawner.SpawnPoint));
    }
}

[BurstCompile]
public partial struct SpawnerParallelSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var ecb = SystemAPI
            .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
            .CreateCommandBuffer(state.WorldUnmanaged)
            .AsParallelWriter();

        new SpawnParallelJob
        {
            DeltaTime = SystemAPI.Time.DeltaTime,
            ECB = ecb
        }.ScheduleParallel();
    }
}

```

Pasar `chunkIndex` tanto a `Instantiate` como al `SetComponent` posterior garantiza que ambos comandos se ordenen en la misma posición y que la referencia a la entidad diferida de `Instantiate` se resuelva al destino correcto.

5. Determinismo — qué garantiza y qué no

Garantizado: con el mismo contenido de ECB, la reproducción produce un estado final idéntico, sin importar qué hilo de trabajo grabó cada comando.

No garantizado automáticamente:

- **Los IDs de entidad.** Dos ejecuciones pueden generar entidades con valores de `Index` distintos si los comandos previos fueron diferentes.
- **El orden de los chunks entre systems.** Si el system A genera entidades en paralelo y el system B las lee en orden de `ChunkIndexInQuery`, el orden de iteración aún puede depender del historial de creación de archetypes.

Si hace falta determinismo bit a bit entre ejecuciones (replays, rollback de Netcode), conviene combinar:

1. Un ECB paralelo con claves de ordenación deterministas.
2. Systems de paso fijo dentro de `FixedStepSimulationSystemGroup`.
3. No depender de `Entity.Index`; usar IDs de juego propios y controlados.

6. Patrones de clave de ordenación

6.1 Por chunk (la opción por defecto)

Usar `[ChunkIndexInQuery]` `int chunkIndex`. Las colisiones no son problema si todas las entidades del chunk generan un comando idéntico (por ejemplo, un chunk entero marcado como aturdido).

6.2 Por entidad

Usar `[EntityIndexInQuery]` `int entityIndex` cuando hace falta un orden estricto por entidad (por ejemplo, registrar eventos de daño por entidad que deben reproducirse en orden de entidad).

6.3 Índice determinista personalizado

En contadas ocasiones se construye un índice plano a partir de las entradas, por ejemplo `int sortKey = baseOffset + localIndex;`. Solo conviene hacerlo cuando las dos opciones habituales no encajan; los índices a medida son una fuente frecuente de bugs de determinismo.

7. Evitar estos patrones

- **Usar `JobsUtility.ThreadIndex`** como `sortKey`. No es determinista.
 - **Construir la clave con `Random`**. No es determinista.
 - **Mezclar ECBs**. Grabar en dos ECBs distintos hace perder el orden total.
 - **Grabar en un ECB de un system desde varios systems**. Un ECB, un solo grabador (o un `parallel-writer` que envuelva un único ECB).
-

8. Solución de problemas

Síntoma	Causa / Solución
La entidad diferida se resuelve al destino equivocado	<code>Instantiate</code> y el <code>SetComponent</code> posterior usaron claves de ordenación distintas. Deben ser la misma.
Los comandos se ejecutan en distinto orden en cada ejecución	La clave de ordenación no es determinista. Usar <code>[ChunkIndexInQuery]</code> .

Síntoma	Causa / Solución
Error de compilación "JobEntity must have partial"	Añadir partial al struct del job.
ECB.SetComponent en un job paralelo lanza "wrong thread"	Se está usando el EntityCommandBuffer no paralelo dentro de un job paralelo. Llamar antes a .AsParallelWriter() y usar la API paralela.
El replay de Netcode se desincroniza	Algún ECB paralelo no usa una clave de ordenación determinista, o un IJobEntity capturó un valor no determinista. Revisar toda la cadena.
AsParallelWriter() sobre un ECB ya liberado	El ECB ya se reprodujo; pedir uno nuevo en este fotograma.

Capítulo 19. World cliente-servidor de Netcode y Bootstrap

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Descripción general

Netcode for Entities utiliza un **modelo cliente-servidor** y separa la lógica de cliente y servidor en worlds ECS distintos.

World	Rol
Server World	Simulación autoritativa.
Client World	Simulación del jugador local, predicción, interpolación y presentación.
Thin Client World	Cliente ficticio para desarrollo y pruebas de carga.

Cuando el dispositivo del cliente aloja además el servidor, el proyecto funciona con una configuración de **servidor alojado en el cliente**.

2. Tipos de world y filtrado de systems

2.1 WorldSystemFilter

WorldSystemFilter sirve para declarar en qué world de Netcode puede ejecutarse un system.

```
using Unity.Entities;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerOnlySystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Runs only in the server world.
    }
}
```

Flag	Significado
LocalSimulation	World local, sin systems de Netcode.

Flag	Significado
ServerSimulation	World de simulación de servidor.
ClientSimulation	World de simulación de cliente.
ThinClientSimulation	World de simulación de thin client.

2.2 Filtrado de grupos de system

Algunos grupos de Netcode solo existen en worlds concretos, de modo que colocar un system en el grupo también filtra el world.

```
using Unity.Entities;
using Unity.NetCode;

[UpdateInGroup(typeof(GhostInputSystemGroup))]
public partial struct MyInputSystem : ISystem
{
    // GhostInputSystemGroup exists in Client and Local worlds.
    // The system is excluded from Server worlds.
}
```

System group	Worlds donde existe
GhostInputSystemGroup	Client, Local
PredictedSimulationSystemGroup	Client, Server
PresentationSystemGroup	Solo Client; no Server ni Thin Client

3. Proceso de bootstrap

ClientServerBootstrap crea en tiempo de ejecución los worlds de servidor y de cliente.

3.1 Comportamiento por defecto

El bootstrap por defecto crea al arrancar un Server World y un Client World. **No** los conecta de forma automática, así que el juego sigue decidiendo cuándo escuchar y cuándo conectar.

3.2 Bootstrap personalizado

```
using Unity.NetCode;

public class GameBootstrap : ClientServerBootstrap
{
    public override bool Initialize(string defaultWorldName)
    {
    }
```

```

        // Create only the default local world.
        CreateLocalWorld(defaultWorldName);
        return true;
    }
}

```

Después se pueden crear worlds desde un botón de juego, un flujo de lobby, un entorno de pruebas o la línea de comandos.

```

using Unity.NetCode;

void OnPlayButtonClicked()
{
    var serverWorld = ClientServerBootstrap.CreateServerWorld("ServerWorld");
    var clientWorld = ClientServerBootstrap.CreateClientWorld("ClientWorld");

    AutomaticThinClientWorldsUtility.NumThinClientsRequested = 4;
    AutomaticThinClientWorldsUtility.BootstrapThinClientWorlds();
}

```

AutomaticThinClientWorldsUtility se incorporó en la línea 1.5.0 y sirve para crear o eliminar Thin Client worlds en tiempo de ejecución.

4. Configuración de la frecuencia de tick

La velocidad de simulación del servidor se configura con el singleton ClientServerTickRate.

Propiedad	Valor por defecto	Significado
SimulationTickRate	60	Ticks de simulación del servidor por segundo.
NetworkTickRate	Igual que SimulationTickRate	Tasa de envío de snapshots en ticks por segundo.
MaxSimulationStepsPer Frame	4	Número máximo de pasos de simulación por fotograma.
MaxSimulationStepBatch Size	4	Número de ticks que se pueden agrupar con deltaTime escalado.
TargetFrameRateMode	BusyWait	BusyWait, Sleep o Auto.

```

using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

public class TickRateAuthoring : MonoBehaviour
{
    public int SimulationTickRate = 60;
    public int NetworkTickRate = 30;
}

```

```

class Baker : Baker<TickRateAuthoring>
{
    public override void Bake(TickRateAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new ClientServerTickRate
        {
            SimulationTickRate = authoring.SimulationTickRate,
            NetworkTickRate = authoring.NetworkTickRate,
        });
    }
}

```

Bajar NetworkTickRate por debajo de SimulationTickRate ahorra ancho de banda. En ese caso, la carga de GhostSendSystem se reparte entre varios ticks de simulación con una planificación round-robin.

5. Flujo de actualización de servidor y cliente

```

Server World
└─ Fixed timestep, 60 Hz default
    └─ SimulationSystemGroup
        ├── GhostSendSystem
        └─ CommandReceiveSystem

Client World
├─ Dynamic timestep
│   └─ SimulationSystemGroup
│       ├── GhostReceiveSystem
│       └─ CommandSendSystem
├─ Fixed timestep, same as the server
│   └─ PredictedSimulationSystemGroup
│       └─ Prediction code and rollback loop
└─ PresentationSystemGroup
    └─ Rendering and interpolation

```

El servidor usa un paso de tiempo fijo para lograr una simulación autoritativa determinista. El cliente usa un paso de tiempo dinámico para la simulación normal y la presentación, mientras que el código de predicción corre en el bucle predicho de paso fijo.

6. Migración de world

Conviene usar DriverMigrationSystem cuando una transición de world debe conservar el estado de la conexión.

```

using Unity.Entities;
using Unity.NetCode;

public World MigrateServerWorld(World sourceWorld)
{
    DriverMigrationSystem migrationSystem = null;
    foreach (var world in World.All)
    {
        if ((migrationSystem = world.GetExistingSystemManaged<DriverMigrationSystem>()) != null)
            break;
    }

    var ticket = migrationSystem.StoreWorld(sourceWorld);
    sourceWorld.Dispose();

    var newWorld = ClientServerBootstrap.CreateServerWorld("NewServerWorld");
    var newMigrationSystem = newWorld.GetExistingSystemManaged<DriverMigrationSystem>();
    newMigrationSystem.LoadWorld(ticket);
    return newWorld;
}

```

7. Documentación relacionada

- 03_ECS Core Concepts.md — Conceptos básicos de World, EntityManager, archetype y chunk.
- 09_System Group & Update Order.md — Grupos de system ECS base que Netcode extiende.
- 17_Netcode Network Connection & Approval.md — Escuchar, conectar, aprobar y entrar en el estado in-game.

8. Solución de problemas

Síntoma	Causa / Solución
Un system se ejecuta en el servidor cuando no debería	Añadir un WorldSystemFilter o moverlo al grupo de systems de Netcode correcto.
La configuración de tick-rate no se aplica	Comprobar que ClientServerTickRate se haya horneado en una SubScene cargada.
Los Thin Clients no se crean	Confirmar que AutomaticThinClientWorldsUtility.BootstrapThinClientWorlds() se llama después de establecer NumThinClientsRequested.
El cliente y el servidor se conectan automáticamente	Comprobar si AutoConnectPort está establecido en el bootstrap.
El código de presentación da errores en el modo Play	Los systems de renderizado se están ejecutando en un server world. Añadir un filtro ClientSimulation o moverlos a PresentationSystemGroup.

Capítulo 20. Conexión de red y aprobación en Netcode

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Descripción general

Netcode for Entities representa cada conexión de red como una **entidad**. La entidad de conexión lleva `NetworkStreamConnection`; al terminar la conexión, la entidad se destruye.

La conexión no se considera "in game" hasta que ambos extremos añaden `NetworkStreamInGame`. Sin ese componente, el servidor no envía snapshots ni el cliente envía comandos.

2. Componentes de conexión fundamentales

Componente	Significado
<code>NetworkStreamConnection</code>	Guarda el handle de transporte.
<code>NetworkId</code>	ID único asignado por el servidor tras la aprobación.
<code>NetworkStreamInGame</code>	Necesario para intercambiar snapshots y datos de comandos.
<code>CommandTarget</code>	Entidad que recibe los comandos del jugador; la mantiene el código del juego salvo que se use <code>AutoCommandTarget</code> .
<code>NetworkStreamRequestDisconnect</code>	Solicitud de desconexión; no llamar directamente al driver <code>disconnect</code> .
<code>NetworkStreamIsReconnected</code>	Componente de seguimiento de reconexión incorporado en la línea 1.6.1.

3. Tres métodos de conexión

3.1 Conexión manual con `NetworkStreamDriver`

```
using Unity.Entities;
using Unity.Networking.Transport;
using Unity.NetCode;

// Server.
```

```

var serverDriver = SystemAPI.GetSingletonRW<NetworkStreamDriver>();
serverDriver.ValueRW.Listen(NetworkEndpoint.AnyIpv4.WithPort(7979));

// Client.
var clientDriver = SystemAPI.GetSingletonRW<NetworkStreamDriver>();
clientDriver.ValueRW.Connect(
    state.EntityManager,
    NetworkEndpoint.Parse("127.0.0.1", 7979));

```

3.2 AutoConnectPort en el bootstrap

```

using Unity.NetCode;

public class AutoConnectBootstrap : ClientServerBootstrap
{
    public override bool Initialize(string defaultWorldName)
    {
        AutoConnectPort = 7979;
        CreateDefaultClientServerWorlds();
        return true;
    }
}

```

El servidor escucha en una dirección comodín y el cliente se conecta a la dirección de loopback.

3.3 Componentes de solicitud

```

using Unity.Entities;
using Unity.NetCode;

var listenReq = serverWorld.EntityManager.CreateEntity(
    typeof(NetworkStreamRequestListen));
serverWorld.EntityManager.SetComponentData(listenReq,
    new NetworkStreamRequestListen { Endpoint = serverEndpoint });

var connectReq = clientWorld.EntityManager.CreateEntity(
    typeof(NetworkStreamRequestConnect));
clientWorld.EntityManager.SetComponentData(connectReq,
    new NetworkStreamRequestConnect { Endpoint = serverEndpoint });

```

4. Máquina de estados de conexión

4.1 Estado del cliente

Connecting → Handshake → [Approval] → Connected → Disconnected

Estado	Significado
Connecting	Se activa una sola vez tras Connect.

Estado	Significado
Handshake	Conexión de transporte establecida; queda pendiente el handshake de protocolo.
Approval	Solo existe si la aprobación de conexión está activada.
Connected	El servidor ha asignado el NetworkId.
Disconnected	Desconexión o timeout; se rellena DisconnectReason.

4.2 Estado del servidor

Handshake → [Approval] → Connected → Disconnected

Estado	Significado
Handshake	Cliente aceptado; se intercambia NetworkProtocolVersion.
Approval	Solo existe si la aprobación de conexión está activada.
Connected	Conexión aprobada y NetworkId asignado.
Disconnected	Cliente desconectado.

4.3 Escuchar eventos de conexión

```
using Unity.Burst;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[UpdateAfter(typeof(NetworkReceiveSystemGroup))]
[BurstCompile]
public partial struct ConnectionEventListener : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var events = SystemAPI.GetSingleton<NetworkStreamDriver>()
            .ConnectionEventsForTick;

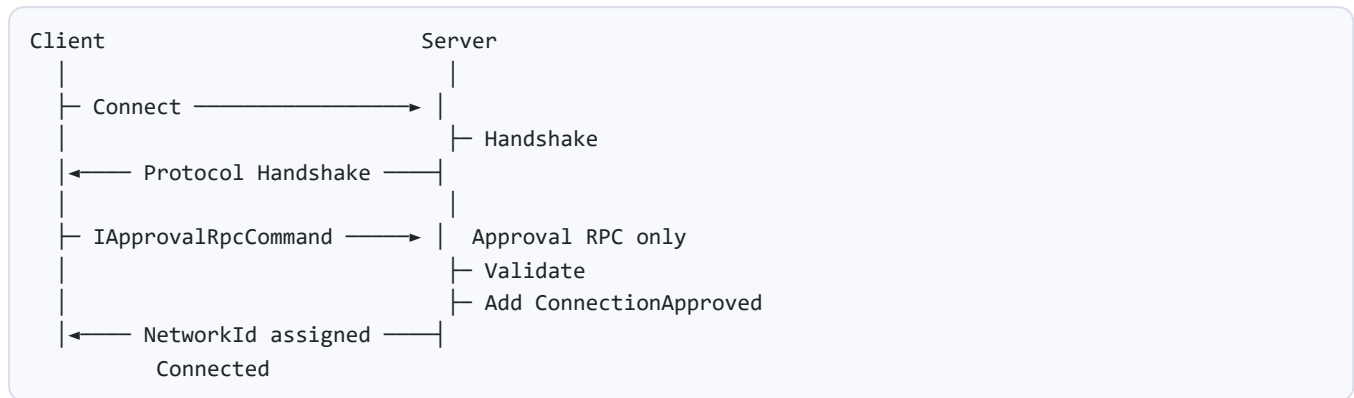
        foreach (var evt in events)
        {
            Debug.Log($"[{state.WorldUnmanaged.Name}] {evt.ToFixedString()}!");
        }
    }
}
```

ConnectionEventsForTick solo es válido dentro de SimulationSystemGroup; leerlo en otro sitio puede omitir o duplicar eventos.

5. Aprobación de conexión

La aprobación de conexión permite que el servidor valide una lista blanca, una contraseña, un token, un ticket de matchmaking u otra barrera similar antes de asignar el NetworkId.

5.1 Flujo de aprobación



5.2 Activar la aprobación

```
using Unity.Entities;
using Unity.NetCode;

using var drvQuery = serverWorld.EntityManager
    .CreateEntityQuery(ComponentType.ReadWrite<NetworkStreamDriver>());

drvQuery.GetSingletonRW<NetworkStreamDriver>().ValueRW
    .RequireConnectionApproval = true;
```

Hay que aplicar el mismo requisito de aprobación en el lado del cliente.

5.3 RPC de aprobación

```
using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;

public struct ApprovalRpc : IApprovalRpcCommand
{
    public FixedString512Bytes Token;
}

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation |
    WorldSystemFilterFlags.ThinClientSimulation)]
public partial struct ClientApprovalSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (connection, entity) in
            SystemAPI.Query<RefRW<NetworkStreamConnection>>())
```

```

        .WithNone<NetworkId>()
        .WithNone<ApprovalSent>()
        .WithEntityAccess()
    {
        var rpcEntity = ecb.CreateEntity();
        ecb.AddComponent(rpcEntity, new ApprovalRpc { Token = "MY_TOKEN" });
        ecb.AddComponent<SendRpcCommandRequest>(rpcEntity);
        ecb.AddComponent<ApprovalSent>(entity);
    }

    ecb.Playback(state.EntityManager);
}

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerApprovalSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (req, rpc, entity) in
            SystemAPI.Query<RefRO<ReceiveRpcCommandRequest>, RefRO<ApprovalRpc>>()
                .WithEntityAccess())
        {
            var connEntity = req.ValueRO.SourceConnection;

            if (rpc.ValueRO.Token.Equals("MY_TOKEN"))
                ecb.AddComponent<ConnectionApproved>(connEntity);
            else
                ecb.AddComponent<NetworkStreamRequestDisconnect>(connEntity);

            ecb.DestroyEntity(entity);
        }

        ecb.Playback(state.EntityManager);
    }
}

```

El tiempo de espera de la aprobación lo controla

`ClientServerTickRate.HandshakeApprovalTimeoutMS`; el valor por defecto es 5000 ms.

6. Entrar en estado in-game

Tras la aprobación, tanto el cliente como el servidor deben añadir `NetworkStreamInGame` antes de poder intercambiar datos de juego.

```

using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerGoInGameSystem : ISystem
{

```

```

public void OnUpdate(ref SystemState state)
{
    var ecb = new EntityCommandBuffer(Allocator.Temp);

    foreach (var (id, entity) in
        SystemAPI.Query<RefRO<NetworkId>>()
            .WithNone<NetworkStreamInGame>()
            .WithEntityAccess())
    {
        ecb.AddComponent<NetworkStreamInGame>(entity);
        // Spawn the player entity and set CommandTarget as needed.
    }

    ecb.Playback(state.EntityManager);
}

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation |
    WorldSystemFilterFlags.ThinClientSimulation)]
public partial struct ClientGoInGameSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (id, entity) in
            SystemAPI.Query<RefRO<NetworkId>>()
                .WithNone<NetworkStreamInGame>()
                .WithEntityAccess())
        {
            ecb.AddComponent<NetworkStreamInGame>(entity);
        }

        ecb.Playback(state.EntityManager);
    }
}

```

7. Buffers del flujo de datos

Buffer	Dirección	Datos
IncomingRpcDataStreamBuffer	Recepción	Datos de RPC.
IncomingCommandDataStreamBuffer	Recepción	Datos de comandos.
IncomingSnapshotDataStreamBuffer	Recepción en cliente	Datos de snapshot.
OutgoingRpcDataStreamBuffer	Envío	Datos de RPC.
OutgoingCommandDataStreamBuffer	Envío en cliente	Datos de comandos.

8. Migración de host

Host Migration preserva el estado de conexión del cliente cuando cambia el servidor.

Elemento	Significado
Activación	Activada por defecto desde la línea 1.7.0; las versiones anteriores requerían <code>ENABLE_HOST_MIGRATION</code> .
Seguimiento de reconexión	Tras la reconexión, <code>NetworkStreamIsReconnected</code> se añade a las dos entidades de conexión.
Prespawn Ghosts	Los IDs de Prespawn Ghost se preservan durante la Host Migration.

La línea 1.7.0 reorganizó a fondo los nombres de la API de Host Migration, así que los samples anteriores hay que migrarlos antes de usarlos.

9. Documentación relacionada

- 16_Netcode Client-Server World & Bootstrap.md — cómo se crean los worlds de Netcode.
- 18_Netcode Ghost Snapshot & Synchronization.md — por qué `NetworkStreamInGame` controla el intercambio de snapshots.
- 21_Netcode RPC.md — RPCs normales y RPCs de aprobación.

10. Solución de problemas

Síntoma	Causa / Solución
La conexión se establece, pero los Ghosts no aparecen	Añadir <code>NetworkStreamInGame</code> tanto en el cliente como en el servidor.
Los datos siguen sin intercambiarse después de la aprobación	Confirmar que se añadió <code>ConnectionApproved</code> a la entidad de conexión.
La conexión se cae cuando la app pierde el foco	Poner <code>Application.runInBackground = true</code> .
Discrepancia de versión de protocolo	Usar builds de cliente y servidor idénticas, con la misma configuración de Ghost.
La aprobación caduca por timeout	Ajustar <code>HandshakeApprovalTimeoutMS</code> o depurar la ruta del RPC de aprobación.

Capítulo 21. Snapshot de Ghost y sincronización en Netcode

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Descripción general

Un **Ghost** es una entidad de red que se simula en el servidor y se replica en los clientes. El servidor envía periódicamente el estado del Ghost en forma de snapshots, y cada cliente los aplica mediante interpolación o predicción.

Mecanismo	Uso
Ghost	Consistencia eventual sobre paquetes no fiables, con optimización de replicación integrada.
RPC	Evento fiable y puntual.

2. Ghost vs RPC

Usar Ghost para	Usar RPC para
El estado de entidades espaciales cercanas a un cliente.	Eventos de alto nivel del flujo de juego.
Datos que requieren predicción en el cliente.	Comandos puntuales, como el chat o los cambios de configuración.
Estado que debe seguir existiendo tras un ciclo de desconexión y reconexión.	Eventos sin estado replicado persistente.

3. Authoring de Ghost

Añadir GhostAuthoringComponent al prefab que se quiere replicar.

Propiedad	Significado
Name	Identificador del Ghost.
Importance	Prioridad en el snapshot cuando el ancho de banda es limitado.

Propiedad	Significado
Supported Ghost Mode	All, Interpolated o Predicted.
Default Ghost Mode	Interpolated, Predicted o OwnerPredicted.
Optimization Mode	Dynamic por defecto, o Static para objetos que apenas cambian.
MaxSendRate	Tasa máxima de replicación en Hz, limitada por NetworkTickRate.
UseSingleBaseline	Opción de compresión delta con una sola línea base, introducida en la línea 1.5.0.

3.1 Modos de Ghost

Modo	Significado
Interpolated	El cliente interpola entre los snapshots del servidor. La imagen es fluida, pero llega con retraso.
Predicted	El cliente simula por adelantado para reducir la latencia del input. El coste de CPU es mayor.
OwnerPredicted	El cliente propietario predice; los demás clientes interpolan.

4. Atributos de serialización

4.1 GhostField

GhostField se aplica a los campos de componente que deben serializarse.

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

public struct MyComponent : IComponentData
{
    [GhostField(Quantization = 1000, Smoothing = SmoothingAction.Interpolate)]
    public float3 Position;

    [GhostField(Quantization = 100)]
    public float Health;

    [GhostField]
    public int Score;

    [GhostField(SendData = false)]
    public float LocalOnly;
}
```

Propiedad de GhostField	Significado
Quantization	Multiplicador de float a int; con 1000 se conservan tres decimales.

Propiedad de GhostField	Significado
Smoothing	Clamp, Interpolate o InterpolateAndExtrapolate.
MaxSmoothingDistance	Umbral de distancia que desactiva la interpolación en los teletransportes.
Composite	Trata el struct como un solo bit de cambio.
SendData	Ponerlo en false para excluir el campo de la serialización.
SubType	Selecciona una regla de serialización especializada.

4.2 GhostEnabledBit

GhostEnabledBit sirve para replicar el estado de habilitado de un enableable component.

```
using Unity.Entities;
using Unity.NetCode;

[GhostEnabledBit]
public struct MyEnableableComp : IComponentData, IEnableableComponent
{
    [GhostField] public int Value;
}
```

4.3 GhostComponent

GhostComponent sirve para controlar las variantes de prefab, la serialización de las entidades hijas y el alcance de la replicación.

```
using Unity.Entities;
using Unity.NetCode;

[GhostComponent(
    PrefabType = GhostPrefabType.All,
    SendTypeOptimization = GhostSendType.AllClients,
    SendToOwner = SendToOwnerType.SendToNonOwner)]
public struct EnemyData : IComponentData
{
    [GhostField] public float Health;
}
```

Propiedad	Opciones
PrefabType	InterpolatedClient, PredictedClient, Client, Server, AllPredicted, All.
SendTypeOptimization	None, Interpolated, Predicted, All.
SendToOwner	SendToOwner, SendToNonOwner, All.
SendDataForChildEntity	Replica los datos de componente de las entidades hijas; es más lento que con los datos de la entidad raíz.

5. Serialización de buffers

Para `IBufferElementData`, cada campo público debe tener `[GhostField]`.

```
using Unity.Entities;
using Unity.NetCode;

[InternalBufferCapacity(8)]
public struct MyBuffer : IBufferElementData
{
    [GhostField] public int Value;
    [GhostField] public float Score;
}
```

Los buffers son más estrictos que los componentes: en un elemento de buffer hay que anotar todos los campos públicos, no basta con uno solo.

6. Variantes de Ghost component

Una variante permite definir un esquema de serialización alternativo sin tocar el tipo de componente original.

```
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[GhostComponentVariation(typeof(LocalTransform), "Transform - 2D")]
[GhostComponent(PrefabType = GhostPrefabType.All)]
public struct Transform2dVariant
{
    [GhostField(Quantization = 1000, Smoothing = SmoothingAction.InterpolateAndExtrapolate)]
    public float3 Position;
}
```

Variante	Uso
ClientOnlyVariant	El componente solo existe en los clientes.
ServerOnlyVariant	El componente solo existe en el servidor.
DontSerializeVariant	Desactiva la serialización por completo.

7. Sistema de snapshots

7.1 Snapshots parciales

Si un snapshot superara el MTU, Netcode envía primero los Ghosts de mayor importancia. El resto se envía en ticks posteriores, de modo que el estado del world se transmite de forma progresiva.

7.2 NetworkTickRate

Cuando NetworkTickRate es menor que SimulationTickRate, el trabajo de GhostSendSystem se reparte entre varios ticks de simulación.

```
SimulationTickRate = 60, NetworkTickRate = 30
Tick 1: GhostSendSystem sends a snapshot
Tick 2: GhostSendSystem skips
Tick 3: GhostSendSystem sends a snapshot
...
```

8. Flujo de registro de prefab de Ghost

1. Create a Ghost prefab
 - ├ Add GhostAuthoringComponent
 - └ Add the required authoring components
2. Place or reference it from a SubScene
 - ├ Put the prefab inside the SubScene, or
 - └ Let GhostCollection register the prefab
3. Bake
 - ├ Baker analyzes GhostField attributes
 - └ Source Generator emits serialization code
4. Runtime
 - ├ Server instantiates the Ghost entity
 - ├ GhostSendSystem sends snapshots
 - └ Client automatically spawns the Ghost

8.1 Prespawned Ghosts

Los Ghosts colocados en una SubScene se convierten en **Prespawned Ghosts**:

- El servidor y el cliente emparejan de forma automática la misma entidad al cargar la SubScene.
- No hace falta un mensaje de spawn aparte; basta con la sincronización de snapshots.
- A partir de la línea 1.6.1, los IDs de Prespawn Ghost se conservan durante la Host Migration.

Los objetos estáticos del mundo, como edificios y terreno, suelen funcionar mejor como Prespawned Ghosts con Static Optimization Mode.

9. Lista de verificación

- ☐ Añadir GhostAuthoringComponent al prefab de Ghost.
 - ☐ Marcar los campos replicados con [GhostField].
 - ☐ Elegir el modo de Ghost: Predicted, Interpolated o OwnerPredicted.
 - ☐ Añadir NetworkStreamInGame para que pueda comenzar el intercambio de snapshots.
 - ☐ Ajustar Quantization para ancho de banda y precisión.
 - ☐ Activar SendDataForChildEntity solo cuando haga falta replicar entidades hijas.
-

10. Documentación relacionada

- 22_Netcode Ghost Optimization · Importance · Relevancy.md — controles de ancho de banda, relevancia y tasa de envío.
 - 19_Netcode Prediction & Rollback.md — simulación de Ghost predicho.
 - ../Changelog/Netcode for Entities 1.4 → 6.5 Key Changes.md — cambios de serialización de Ghost entre versiones.
-

11. Solución de problemas

Síntoma	Causa / Solución
El Ghost no aparece en el cliente	Añadir NetworkStreamInGame y confirmar que el prefab de Ghost está registrado.
Los valores float tiemblan	Quantization es demasiado bajo, o falta el suavizado.
El teletransporte genera artefactos de interpolación	Definir MaxSmoothingDistance o usar una lógica de suavizado personalizada.
Discrepancia de versión de protocolo	Los esquemas de Ghost del cliente y el servidor difieren.
El buffer no se serializa	Todos los campos públicos del elemento de buffer deben tener [GhostField].

Capítulo 22. Predicción y rollback en Netcode

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Descripción general

La predicción en el cliente ejecuta el mismo código de simulación en el propio cliente, antes de que llegue el resultado autoritativo del servidor. Esto reduce la latencia del input y hace que el movimiento del jugador responda al instante.

La predicción tiene un coste real de CPU: cuando llega un snapshot autoritativo, el cliente puede retroceder a un tick anterior y volver a simular hasta el tick predicho actual.

2. Componentes y systems fundamentales

Componente / system	Significado
PredictedGhost	Se añade a los Ghosts predichos en los clientes y a todos los Ghosts en el servidor.
Simulate	Tag habilitable que se activa en las entidades que deben simularse en el tick de predicción actual.
PredictedSimulationSystemGroup	Grupo de paso de tiempo fijo que ejecuta el bucle de predicción.
NetworkTime	Contiene el tick del servidor, el tick de predicción y otros datos de temporización relacionados.

3. Flujo de predicción en el cliente

1. Receive a server snapshot
 - └ Apply the latest authoritative state to predicted entities
 2. Find the oldest applied tick
 - └ Use the oldest tick across affected predicted entities
 3. Roll back and re-simulate
 - └ PredictedSimulationSystemGroup repeats from oldest tick to target tick
 - └ TimeData and NetworkTime update for each tick

4. Render

- └ Present the predicted result

Una latencia alta multiplica el coste. Una conexión de 300 ms puede llegar a necesitar más de 20 fotogramas de resimulación, según la frecuencia de tick y la configuración de predicción.

4. Escribir systems de predicción

4.1 Usar el tag Simulate

Al comienzo del bucle de predicción, Netcode desactiva `Simulate` en los Ghosts predichos y luego lo vuelve a activar solo en las entidades que deben ejecutarse en el tick actual.

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct MovementSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var speed = 5f;
        var dt = SystemAPI.Time.DeltaTime;

        foreach (var (transform, input) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<PlayerInput>>()
                .WithAll<Simulate>())
        {
            var move = new float3(input.ValueRO.Horizontal, 0, input.ValueRO.Vertical);
            transform.ValueRW.Position += move * speed * dt;
        }
    }
}
```

Hay que incluir siempre `.WithAll<Simulate>()` en las queries de predicción. Sin esa cláusula, el system puede volver a simular entidades en ticks que ya están terminados.

4.2 Comportamiento en el servidor

En el servidor, el bucle de predicción se ejecuta exactamente una vez por cada tick del servidor:

- `TimeData` contiene los datos normales del tick del servidor.
- `Simulate` está siempre activado.
- El mismo código del system sirve tanto para el cliente como para el servidor.

5. Predicción de jugadores remotos

Si los datos de comandos de otro jugador se serializan, también es posible predecir a ese jugador remoto.

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct RemotePlayerMovement : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        foreach (var (transform, input) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<PlayerInput>>()
                .WithAll<PredictedGhost, Simulate>())
        {
            transform.ValueRW.Position +=
                new float3(input.ValueRO.Horizontal, 0, input.ValueRO.Vertical)
                    * SystemAPI.Time.DeltaTime;
        }
    }
}
```

Con `IInputComponentData`, el sistema de input proporciona automáticamente el input del tick actual.

6. Suavizado de predicción

El suavizado de predicción atenúa la corrección visible cuando el resultado del servidor no coincide con el predicho.

```
using Unity.NetCode;
using Unity.Transforms;

GhostPredictionSmoothing.RegisterSmoothingAction<LocalTransform>(
    state.EntityManager,
    CustomSmoothing.Action);
```

```
using Unity.Burst;
using Unity.Collections.LowLevel.Unsafe;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;
```

```
[BurstCompile]
public static unsafe class CustomSmoothing
{
    public static readonly PortableFunctionPointer<
        GhostPredictionSmoothing.SmoothingActionDelegate> Action =
        new(SmoothingAction);

    [BurstCompile(DisableDirectCall = true)]
    private static void SmoothingAction(
        void* currentData, void* previousData, void* userData)
    {
        ref var current = ref UnsafeUtility.AsRef<LocalTransform>(currentData);
        ref var backup = ref UnsafeUtility.AsRef<LocalTransform>(previousData);

        var dist = math.distance(current.Position, backup.Position);
        if (dist > 0)
        {
            current.Position = math.lerp(backup.Position, current.Position, 0.5f);
        }
    }
}
```

7. Cambio de predicción

La predicción es costosa, así que un Ghost puede alternar de forma dinámica entre Predicted e Interpolated.

7.1 Requisitos previos para el cambio

- Los Supported Ghost Modes deben ser All.
- El modo actual no debe ser OwnerPredicted.
- El Ghost no debe estar ya en proceso de cambio.

7.2 Poner en cola los cambios

```
using Unity.NetCode;

ref var switchQueues = ref SystemAPI
    .GetSingletonRW<GhostPredictionSwitchingQueues>()
    .ValueRW;

switchQueues.ConvertToPredictedQueue.Enqueue(new ConvertPredictionEntry
{
    TargetEntity = entity,
    TransitionDurationSeconds = 1f,
});

switchQueues.ConvertToInterpolatedQueue.Enqueue(new ConvertPredictionEntry
{
    TargetEntity = entity,
```

```
TransitionDurationSeconds = 0f,  
});
```

Los Ghosts interpolados y los predichos corren en líneas de tiempo distintas, a menudo separadas por unos dos intervalos de ping. `SwitchPredictionSmoothing` interpola la posición y la rotación durante la duración configurada para evitar teletransportes visibles.

8. Controles de rendimiento de predicción

Técnica	Efecto
Prediction Switching	Pasar a Interpolated los Ghosts lejanos o de baja importancia.
Restringir el acceso de escritura a GhostField	Usar acceso de solo lectura para los campos que no cambian dentro del bucle de predicción.
Agrupación de física	Usar <code>MaxPredictionStepBatchSizeRepeatedTick</code> para la resimulación repetida de la física.
Ajuste de tick-rate	Bajar <code>SimulationTickRate</code> reduce el número de resimulaciones, a costa de la capacidad de respuesta.

9. Documentación relacionada

- `20_Netcode Command Stream & Input.md` — datos de input que usan los systems de predicción.
- `22_Netcode Ghost Optimization · Importance · Relevancy.md` — el cambio de predicción como estrategia de optimización.
- `23_Netcode Physics Integration & Lag Compensation.md` — comportamiento de la física predicha.

10. Solución de problemas

Síntoma	Causa / Solución
La predicción no se ejecuta	Comprobar que el modo de Ghost sea Predicted y que las queries de predicción incluyan <code>Simulate</code> .
Temblor intenso	Registrar el suavizado de predicción y comprobar que la cuantización se ajuste a las necesidades del juego.

Síntoma	Causa / Solución
La entidad se teletransporta al corregir el servidor	Definir MaxSmoothingDistance o añadir un suavizado personalizado.
La CPU de predicción es demasiado alta	Usar prediction switching, bajar el tick rate y agrupar la resimulación de física donde tenga sentido.
La entidad simula más de una vez por tick	A alguna query de predicción le falta <code>.WithAll<Simulate>()</code> .

Capítulo 23. Flujo de comandos e input en Netcode

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Descripción general

Cuando un cliente está in-game, envía al servidor un **flujo de comandos** continuo. Ese flujo incluye los datos de input y los datos de ACK de snapshot, y sigue activo aunque el jugador no esté pulsando ninguna tecla, para que la conexión se mantenga sincronizada.

2. ICommandData vs IInputComponentData

	ICommandData	IInputComponentData
Uso	Control de comandos a bajo nivel.	Flujo de input a alto nivel.
Gestión del buffer	Manual, con AddCommandData.	Automática, mediante código generado.
Asociación al tick	Manual, con GetDataAtTick.	Automática; se entrega el input del tick actual.
InputEvent	No compatible.	Compatible con eventos puntuales.
Payload máximo	1024 bytes.	1024 bytes.
Más adecuado para	Pipelines de comandos personalizados.	Input normal del jugador.

3. IInputComponentData

3.1 Definir el input

```
using Unity.NetCode;

public struct PlayerInput : IInputComponentData
{
    public int Horizontal;
    public int Vertical;
    public InputEvent Jump;
}
```

InputEvent conserva los eventos puntuales como GetKeyDown para que se registren exactamente una vez en el servidor, aunque la red descarte un paquete de comando redundante.

3.2 Recoger el input en el cliente

```
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[UpdateInGroup(typeof(GhostInputSystemGroup))]
public partial struct GatherInputSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<PlayerInput>();
    }

    public void OnUpdate(ref SystemState state)
    {
        bool jump = Input.GetKeyDown(KeyCode.Space);
        bool left = Input.GetKey(KeyCode.A);
        bool right = Input.GetKey(KeyCode.D);

        foreach (var input in
            SystemAPI.Query<RefRW<PlayerInput>>()
                .WithAll<GhostOwnerIsLocal>())
        {
            input.ValueRW = default;
            if (jump) input.ValueRW.Jump.Set();
            if (left) input.ValueRW.Horizontal -= 1;
            if (right) input.ValueRW.Horizontal += 1;
        }
    }
}
```

3.3 Procesar el input en el servidor y en el bucle de predicción

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct ProcessInputSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<PlayerInput>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var speed = 3f;
        var dt = SystemAPI.Time.DeltaTime;
```

```

        foreach (var (input, transform) in
            SystemAPI.Query<RefRO<PlayerInput>, RefRW<LocalTransform>>()
                .WithAll<Simulate>())
        {
            if (input.ValueRO.Jump.IsSet)
            {
                // Jump logic; runs once on the input tick.
            }

            transform.ValueRW.Position +=
                new float3(input.ValueRO.Horizontal, 0, 0) * speed * dt;
        }
    }
}

```

El bucle de predicción puede ejecutar este system muchas veces durante la resimulación. `InputEvent.IsSet` solo es true en el tick correcto.

4. ICommandData

4.1 Definir un comando

```

using Unity.Mathematics;
using Unity.NetCode;

public struct MyCommand : ICommandData
{
    public NetworkTick Tick { get; set; }
    public int Action;
    public float2 Direction;
}

```

4.2 Añadir y leer datos de comando

```

using Unity.Entities;
using Unity.NetCode;

DynamicBuffer<MyCommand> buffer = default;
NetworkTick networkTick = default;

buffer.AddCommandData(new MyCommand
{
    Tick = networkTick,
    Action = 1,
    Direction = new float2(1, 0),
});

if (buffer.GetDataAtTick(networkTick, out var cmd))
{

```

```
// Use cmd.Action and cmd.Direction.  
}
```

5. Mecanismo de envío de comandos

```
Command packet  
├ Tick, Command  
├ Delta(Tick - 1)  
├ Delta(Tick - 2)  
└ Delta(Tick - 3)
```

The last three inputs are redundantly sent with delta compression.
Maximum payload: 1024 bytes.

CommandSendPacketSystem hace los envíos al ritmo de `SimulationTickRate`, y `CommandReceiveSystem` recibe los datos de comando en el servidor y los reenvía a la entidad de destino.

6. AutoCommandTarget

Si un Ghost tiene `AutoCommandTarget`, los comandos se envían automáticamente.

Requisito	Significado
El Ghost tiene owner activado	Activado en <code>GhostAuthoringComponent</code> .
Support Auto Command Target activado	Activado en <code>GhostAuthoringComponent</code> .
El cliente es el propietario	El servidor asigna a <code>GhostOwner.NetworkId</code> el <code>NetworkId.Value</code> del cliente.
El Ghost es Predicted u OwnerPredicted	Los datos de comando se dirigen a la predicción.
<code>AutoCommandTarget.Enabled</code> es true	Se puede desactivar en tiempo de ejecución.

Sin `AutoCommandTarget`, hay que asignar `CommandTarget` a mano en la entidad de conexión.

```
using Unity.Entities;  
using Unity.NetCode;  
  
Entity connectionEntity = default;  
Entity playerEntity = default;  
  
state.EntityManager.SetComponentData(connectionEntity,  
    new CommandTarget { targetEntity = playerEntity });
```

7. Identificar las entidades de propiedad local

7.1 GhostOwnerIsLocal

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Transforms;

foreach (var (input, transform) in
    SystemAPI.Query<RefRW<PlayerInput>, RefRW<LocalTransform>>()
        .WithAll<GhostOwnerIsLocal>())
{
    // Local-owned entity only.
}
```

7.2 Verificación manual del propietario

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Transforms;

var localId = SystemAPI.GetSingleton<NetworkId>().Value;

foreach (var (owner, transform) in
    SystemAPI.Query<RefRO<GhostOwner>, RefRW<LocalTransform>>())
{
    if (owner.ValueRO.NetworkId == localId)
    {
        // Local-owned entity.
    }
}
```

8. Latencia de entrada forzada

`ClientTickRate.ForcedInputLatencyTicks` añade un retardo de input deliberado.

Configuración	Significado
ForcedInputLatencyTicks	Latencia de input adicional, en ticks; el valor por defecto es 0.
Caso de uso	Dar a todos los jugadores el mismo retardo de input en géneros como los juegos de lucha.
Efecto	Se incluye de forma automática en los cálculos de <code>MaxPredictAheadTimeMS</code> .

8.1 InputTargetTick vs ServerTick

En los systems de input, cuando la latencia de input forzada está activa, conviene usar `NetworkTime.InputTargetTick` en lugar de `NetworkTime.ServerTick`.

```
using Unity.NetCode;

var inputTick = SystemAPI.GetSingleton<NetworkTime>().InputTargetTick;
var serverTick = SystemAPI.GetSingleton<NetworkTime>().ServerTick;
```

`InputTargetTick` y `ServerTick` no coinciden cuando `ForcedInputLatencyTicks` es distinto de cero. El input debe apuntar a `InputTargetTick`.

9. Documentación relacionada

- [19_Netcode Prediction & Rollback.md](#) — systems de predicción que consumen el input.
- [17_Netcode Network Connection & Approval.md](#) — `NetworkStreamInGame` y entidades de conexión.
- [21_Netcode RPC.md](#) — mensajes fiables y puntuales, no input continuo.

10. Solución de problemas

Síntoma	Causa / Solución
El servidor no recibe input	Añadir <code>NetworkStreamInGame</code> , asignar <code>CommandTarget</code> o cumplir los requisitos de <code>AutoCommandTarget</code> .
El salto se dispara más de una vez	Usar <code>InputEvent.Set()</code> / <code>.IsSet</code> en lugar de un bool normal.
Errores de tamaño de payload	Mantener los structs de comando en 1024 bytes como máximo.
Falta input durante la predicción	Es preferible el mapeo automático de ticks de <code>IInputComponentData</code> , o comprobar <code>GetDataAtTick</code> .
El código del servidor se ejecuta en <code>GhostInputSystemGroup</code>	Ese grupo es del lado del cliente; la lógica del servidor debe ir en <code>PredictedSimulationSystemGroup</code> .

Capítulo 24. RPC de Netcode

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Descripción general

Un RPC es un **mensaje fiable y puntual** entre cliente y servidor. El código de serialización y deserialización del RPC se genera de forma automática, y su gestión sigue encajando en el marco de ECS y el Job System.

Los RPCs no sustituyen a la replicación de Ghosts ni a los flujos de comandos. Conviene usarlos para eventos puntuales del flujo de juego, no para estado de alta frecuencia.

2. RPC vs Ghost vs Command

	RPC	Ghost	Command
Comportamiento de transporte	Fiable, ordenado.	No fiable con optimización de replicación.	No fiable con envíos redundantes.
Dirección	Bidireccional.	Servidor → Cliente.	Cliente → Servidor.
Frecuencia	Puntual.	En cada tick de red, según haga falta.	En cada tick de simulación.
Uso	Flujo de juego, lobby, chat.	Estado replicado de entidades.	Input del jugador.
Límite en vuelo	Sí, el límite del pipeline fiable.	No.	No.

Como los RPCs son ordenados y fiables, un tráfico de RPC frecuente puede quedar atascado detrás de los paquetes en vuelo.

3. Definir RPCs

3.1 RPC vacío

```
using Unity.NetCode;
```

```
public struct PingRpc : IRpcCommand
{
}
```

3.2 RPC con datos

```
using Unity.Collections;
using Unity.NetCode;

public struct ChatMessageRpc : IRpcCommand
{
    public FixedString128Bytes Message;
    public int SenderId;
}
```

4. Enviar RPCs

4.1 Cliente a servidor

```
using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation)]
public partial struct SendChatSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<NetworkId>();
    }

    public void OnUpdate(ref SystemState state)
    {
        if (!Input.GetKeyDown(KeyCode.Space))
            return;

        var ecb = new EntityCommandBuffer(Allocator.Temp);
        var entity = ecb.CreateEntity();

        ecb.AddComponent(entity, new ChatMessageRpc
        {
            Message = "Hello!",
            SenderId = SystemAPI.GetSingleton<NetworkId>().Value,
        });

        ecb.AddComponent<SendRpcCommandRequest>(entity);
        ecb.Playback(state.EntityManager);
    }
}
```

ISystem es un struct. El estado que persiste de un fotograma a otro no debe guardarse en campos de instancia del system, sino en componentes o singletons.

4.2 Servidor a un cliente

```
using Unity.Entities;
using Unity.NetCode;

Entity targetConnectionEntity = default;

var entity = ecb.CreateEntity();
ecb.AddComponent(entity, new NotifyRpc());
ecb.AddComponent(entity, new SendRpcCommandRequest
{
    TargetConnection = targetConnectionEntity,
});
```

4.3 Difusión del servidor

```
using Unity.Entities;
using Unity.NetCode;

var entity = ecb.CreateEntity();
ecb.AddComponent(entity, new GameStartRpc());
ecb.AddComponent(entity, new SendRpcCommandRequest
{
    TargetConnection = Entity.Null,
});
```

Entity.Null difunde el RPC.

5. Recibir RPCs

```
using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ReceiveChatSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (req, chat, entity) in
            SystemAPI.Query<RefRO<ReceiveRpcCommandRequest>, RefRO<ChatMessageRpc>>()
                .WithEntityAccess())
        {
            var sourceConnection = req.ValueRO.SourceConnection;
            Debug.Log($"Player {chat.ValueRO.SenderId}: {chat.ValueRO.Message}");
        }
    }
}
```

```

        ecb.DestroyEntity(entity);
    }

    ecb.Playback(state.EntityManager);
}

```

Hay que destruir siempre la entidad del RPC recibido tras procesarlo. Si no, el mismo RPC vuelve a procesarse en cada fotograma.

6. RpcQueue

RpcQueue solo se usa cuando hace falta programar los RPC manualmente.

```

using Unity.Entities;
using Unity.NetCode;

var rpcQueue = SystemAPI.GetSingleton<RpcCollection>()
    .GetRpcQueue<MyRpc, MyRpc>();
var bufferLookup = SystemAPI.GetBufferLookup<OutgoingRpcDataStreamBuffer>();

if (bufferLookup.HasBuffer(connectionEntity))
{
    var buffer = bufferLookup[connectionEntity];
    rpcQueue.Schedule(buffer, new MyRpc());
}

```

7. Serialización manual

La generación automática es la opción por defecto. Si se desactiva, la serialización y la deserialización deben ser simétricas.

```

using Unity.Burst;
using Unity.Entities;
using Unity.NetCode;
using Unity.Networking.Transport;

[BurstCompile]
public struct MyCustomRpc : IComponentData, IRpcCommandSerializer<MyCustomRpc>
{
    public int Data;

    public void Serialize(ref DataStreamWriter writer, in RpcSerializerState state,
        in MyCustomRpc data)
    {
        writer.WriteInt(data.Data);
    }
}

```

```

public void Deserialize(ref DataStreamReader reader, in RpcDeserializerState state,
    ref MyCustomRpc data)
{
    data.Data = reader.ReadInt();
}

// Execute implementation omitted.
}

```

8. IApprovalRpcCommand

IApprovalRpcCommand es un RPC especial que solo se usa durante la aprobación de conexión. Los mensajes normales de IRpcCommand quedan bloqueados durante los estados Handshake y Approval.

```

using Unity.Collections;
using Unity.NetCode;

public struct LoginApproval : IApprovalRpcCommand
{
    public FixedString512Bytes AuthToken;
}

```

El flujo de aprobación completo está en 17_Netcode Network Connection & Approval.md.

9. Patrones comunes

Patrón	RPCs de ejemplo
Lobby / matchmaking	JoinLobbyRpc, ReadyRpc.
Flujo de juego	GameStartRpc, RoundEndRpc.
Chat	ChatMessageRpc.
Intercambio de ítems	TradeRequestRpc, TradeAcceptRpc.
Aprobación de conexión	Implementaciones de IApprovalRpcCommand.

10. Documentación relacionada

- 18_Netcode Ghost Snapshot & Synchronization.md — replicación de Ghost para el estado.
 - 20_Netcode Command Stream & Input.md — flujos de comandos para la entrada.
 - 17_Netcode Network Connection & Approval.md — RPCs de aprobación.
-

11. Solución de problemas

Síntoma	Causa / Solución
El RPC no se recibe	Comprobar que se añadió SendRpcCommandRequest y que el system se ejecuta en el world previsto.
El RPC se procesa en cada fotograma	Destruir la entidad del RPC recibido después de procesarlo.
El orden del RPC parece retrasado	El transporte fiable y ordenado puede atascarse detrás de los paquetes en vuelo. Para datos frecuentes, usar Ghosts o comandos.
El RPC está bloqueado durante la aprobación	Usar IApprovalRpcCommand, no IRpcCommand.
La serialización manual falla	Comprobar que Serialize y Deserialize leen y escriben los campos en el mismo orden.

Capítulo 25. Optimización de Ghost, importancia y relevancia en Netcode

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Descripción general

Los worlds multijugador grandes no pueden enviar todos los Ghosts a todos los clientes en cada tick. Netcode for Entities recurre a **Optimization Mode**, **Importance Scaling**, **Relevancy** y a controles de tasa de envío por Ghost para reducir el coste de CPU y el ancho de banda.

La regla básica es sencilla: el servidor debe gastar el presupuesto de snapshot en los Ghosts que, en ese momento, le importan a una conexión concreta.

2. Optimization Mode

Optimization Mode se configura en GhostAuthoringComponent.

Modo	Significado	Adecuado para
Dynamic	El modo por defecto. Optimiza el tamaño del snapshot tanto si los datos cambian como si no.	Ghosts que se mueven con normalidad.
Static	Evita reenviar datos cuando el Ghost no ha cambiado.	Edificios, terreno, props estáticos.

La línea 1.11.0 incorporó una optimización para Ghosts estáticos que puede reducir bastante el tiempo de GhostUpdateSystem en los Ghosts estáticos que no han cambiado.

3. Límites de tamaño de snapshot

El presupuesto de snapshot se puede limitar en varios niveles.

Configuración	Significado
NetworkStreamSnapshotTargetSize	Objetivo de bytes orientativo por conexión.
GhostSendSystemData.MaxSendEntities	Máximo de entidades por snapshot.

Configuración	Significado
<code>GhostSendSystemData.MaxSendChunks</code>	Máximo de chunks por snapshot.

Cuando el presupuesto es limitado, se envían primero los Ghosts de mayor importancia.

4. Importance Scaling

Importance Scaling calcula en el servidor la prioridad de envío de cada Ghost.

4.1 Entradas

`GhostImportance` combina tres tipos de datos.

Datos	Significado
Por conexión	Datos propios de la conexión, como la posición del jugador.
Datos singleton	Configuración global, como el tamaño del tile.
Datos compartidos por chunk	Parámetros de escala a nivel de chunk.

4.2 Importancia basada en distancia

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

var gridSingleton = state.EntityManager.CreateSingleton(
    new GhostDistanceData
    {
        TileSize = new int3(50, 50, 256),
        TileCenter = new int3(0, 0, 128),
        TileBorderWidth = new float3(1f, 1f, 1f),
    });

state.EntityManager.AddComponentData(gridSingleton, new GhostImportance
{
    ScaleImportanceFunction = GhostDistanceImportance.ScaleFunctionPointer,
    GhostConnectionComponentType = ComponentType.ReadOnly<GhostConnectionPosition>(),
    GhostImportanceDataType = ComponentType.ReadOnly<GhostDistanceData>(),
    GhostImportancePerChunkDataType = ComponentType.ReadOnly<GhostDistancePartitionShared>(),
});
```

Cambio que rompe compatibilidad en la línea 1.8.0: las funciones de escala de `GhostDistanceImportance` ya no multiplican `baseImportance` por 1000. El código de importancia personalizada que daba por hecho el multiplicador antiguo hay que revisarlo.

4.3 GhostDistancePartitionShared automático

```
using Unity.NetCode;

GhostDistancePartitioningSystem.AutomaticallyAddGhostDistancePartitionSharedComponent = true;
```

4.4 Importance Visualizer

Desde la línea 1.8.0, la PlayMode Tool incluye un Importance Visualizer. Conviene usarlo para examinar la distribución de importancia en tiempo de ejecución en vez de deducirla del código.

5. Ghost Relevancy

Ghost Relevancy permite que el servidor oculte o muestre Ghosts concretos según la conexión.

Modo	Significado
Disabled	El modo por defecto. Sin filtrado; todos los Ghosts son candidatos.
SetIsRelevant	Solo se replican a esa conexión los Ghosts de la lista.
SetIsIrrelevant	Se excluyen de esa conexión los Ghosts de la lista.

5.1 Casos de uso

Escenario	Idea de implementación
Descarte por distancia	Marcar como irrelevantes los Ghosts lejanos.
Niebla de guerra	Ocultar entidades fuera del campo de visión.
Ghosts propios de cada cliente	Marcar un Ghost como relevante solo para un cliente.
Filtrado por equipo	Ocultar datos internos que solo conoce el enemigo.

5.2 Ejemplo de relevancia

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
[UpdateInGroup(typeof(GhostSimulationSystemGroup))]
[UpdateBefore(typeof(GhostSendSystem))]
public partial struct RelevancySystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
    }
```

```

var relevancy = SystemAPI.GetSingletonRW<GhostRelevancy>();

relevancy.ValueRW.GhostRelevancyMode = GhostRelevancyMode.SetIsRelevant;
relevancy.ValueRW.GhostRelevancySet.Clear();

foreach (var (ghostInstance, transform) in
    SystemAPI.Query<RefRO<GhostInstance>, RefRO<LocalTransform>>())
{
    foreach (var (connId, connPos) in
        SystemAPI.Query<RefRO<NetworkId>, RefRO<GhostConnectionPosition>>())
    {
        float dist = math.distance(
            transform.ValueRO.Position,
            connPos.ValueRO.Position);

        if (dist < 100f)
        {
            relevancy.ValueRW.GhostRelevancySet.TryAdd(
                new RelevantGhostForConnection(
                    connId.ValueRO.Value,
                    ghostInstance.ValueRO.ghostId,
                    1);
        }
    }
}
}
}

```

Este ejemplo simplificado es $O(\text{Ghosts} \times \text{conexiones})$. En worlds grandes es preferible la partición de importancia por distancia y reservar la relevancia para reglas que la partición no puede expresar, como la niebla de guerra o la visibilidad por equipos.

5.3 DefaultRelevancyQuery

DefaultRelevancyQuery define los chunks de Ghost que siempre son relevantes para todas las conexiones, como los Ghosts de estado global del juego o el estado de red de la UI.

5.4 Relevancia más importancia

Desde la línea 1.6.1 se pueden combinar Ghost Relevancy e Importance Scaling mediante `PrioChunks.isRelevant`, lo que habilita una ruta de relevancia más rápida.

6. Optimización de la CPU de predicción

6.1 Prediction Switching

Pasar de Predicted a Interpolated los Ghosts lejanos o poco importantes.

```

using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

float distance = math.distance(playerPos, ghostPos);
if (distance > predictionRange)
{
    switchQueues.ConvertToInterpolatedQueue.Enqueue(new ConvertPredictionEntry
    {
        TargetEntity = ghostEntity,
        TransitionDurationSeconds = 0.5f,
    });
}

```

6.2 Minimizar el acceso de escritura a GhostField

Los jobs con acceso de escritura a componentes GhostField pueden disparar comprobaciones de serialización. Conviene usar acceso de solo lectura para los campos que no cambian dentro del bucle de predicción.

6.3 Resimulación de física

Para casos de alta latencia que necesitan más de 20 fotogramas de resimulación predicha:

- Desactivar la física multihilo mediante el singleton PhysicsStep cuando salga más barato consolidar en el hilo principal.
- Usar MaxPredictionStepBatchSizeRepeatedTick para agrupar los ticks de física repetidos.

7. MaxIterateChunks

GhostSendSystemData.MaxIterateChunks limita el número de chunks que se procesan en un solo tick.

Configuración	Significado
MaxIterateChunks	Limita el coste de CPU de la serialización por tick.
MaxSendChunks	Limita el ancho de banda al acotar cuántos chunks entran en un snapshot.

Estos parámetros resuelven problemas distintos: MaxIterateChunks reparte el trabajo de CPU, mientras que MaxSendChunks limita el tamaño del snapshot.

8. MaxSendRate

`GhostAuthoringComponent.MaxSendRate` limita la frecuencia de envío de un Ghost en Hz.

```
NetworkTickRate = 60, MaxSendRate = 10  
→ This Ghost can be included once every 6 network ticks.
```

Es la opción para Ghosts de baja frecuencia o de poca importancia.

9. UseSingleBaseline

`GhostAuthoringComponent.UseSingleBaseline = true` usa una sola línea base para la compresión delta. Esto puede reducir el coste de CPU, posiblemente a cambio de más ancho de banda.

10. Lista de verificación de optimización

- ☐ Usar el modo de optimización **Static** para Ghosts que apenas cambian.
 - ☐ Definir `NetworkStreamSnapshotTargetSize` para limitar el ancho de banda por conexión.
 - ☐ Implementar Importance Scaling basado en distancia.
 - ☐ Aplicar Relevancy solo donde la partición no pueda expresar la regla.
 - ☐ Pasar los Ghosts lejanos al modo Interpolated.
 - ☐ Usar `MaxSendRate` para Ghosts de baja frecuencia.
 - ☐ Evitar el acceso de escritura a datos de `GhostField` que no han cambiado.
 - ☐ Inspeccionar la prioridad en tiempo de ejecución con el Importance Visualizer.
-

11. Documentación relacionada

- 18_Netcode Ghost Snapshot & Synchronization.md — flujo de serialización y snapshot de Ghost.
 - 19_Netcode Prediction & Rollback.md — comportamiento de cambio de predicción.
 - 24_Netcode Profiler & Debugging.md — análisis de ancho de banda y coste de predicción.
-

12. Solución de problemas

Síntoma	Causa / Solución
Un Ghost tarda demasiado en aparecer	La Importance es demasiado baja o el presupuesto de snapshot es demasiado ajustado.
Los Ghosts dejan de enviarse tras aplicar Importance Scaling	Verificar que GhostDistancePartitionShared está presente o usar la opción de adición automática.
El Ghost parpadea con Relevancy activada	Revisar las condiciones de límite y reconstruir el conjunto de relevancy de forma consistente en cada frame.
El ancho de banda está disponible pero faltan Ghosts	Revisar los límites de MaxSendEntities y MaxSendChunks.

Capítulo 26. Integración de física y compensación de lag en Netcode

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Descripción general

Netcode for Entities se integra con **Unity Physics** para la simulación de física en red. El comportamiento físico depende de si el Ghost es Interpolated o Predicted.

Para que se ejecute la física predicha, en la escena debe existir al menos un Ghost Predicted.

2. Física interpolada vs. predicha

2.1 Ghosts Interpolated en clientes

- La posición y la rotación provienen de los snapshots del servidor.
- Simulate está desactivado, así que el objeto físico se comporta como kinematic.
- PhysicsVelocity se ignora.

2.2 Ghosts Predicted en clientes

- La física corre dentro del bucle de predicción y puede ejecutarse varias veces por fotograma renderizado.
- PhysicsSystemGroup se ejecuta dentro de PredictedFixedStepSimulationSystemGroup.
- En la física predicha, se espera que el cliente obtenga el mismo resultado de simulación que el servidor.

Netcode initialization

- └ PhysicsSystemGroup moves under PredictedFixedStepSimulationSystemGroup
 - └ FixedStepSimulationSystemGroup moves under PredictedFixedStepSimulationSystemGroup
-

3. Rendimiento de la física predicha

La física predicha es costosa porque el rollback puede ejecutarla varias veces seguidas.

3.1 Agrupamiento en el servidor

```
using Unity.NetCode;

ClientServerTickRate tickRate = default;
tickRate.MaxSimulationStepBatchSize = 4;
tickRate.MaxSimulationStepsPerFrame = 4;
```

3.2 Agrupamiento en el cliente

```
using Unity.NetCode;

ClientTickRate clientRate = default;
clientRate.MaxPredictionStepBatchSizeFirstTimeTick = 2;
clientRate.MaxPredictionStepBatchSizeRepeatedTick = 4;
```

El agrupamiento puede aumentar el riesgo de errores de predicción, así que conviene usarlo como ajuste de rendimiento puntual, no como valor por defecto.

3.3 Cuantización

La cuantización por defecto de Transform y Velocity para los Ghosts físicos es 1000.

Mayor cuantización	Menor cuantización
Simulación más precisa.	Menor ancho de banda.
Mayor ancho de banda.	Mayor pérdida de precisión.

4. Compensación de lag

Como los clientes predicen por delante del servidor, el servidor autoritativo a veces necesita consultar un mundo de colisiones más antiguo, el que corresponde al momento en que el cliente actuó.

4.1 Habilitar la compensación de lag

Añadir `NetCodePhysicsConfig` a una `SubScene` y poner `EnableLagCompensation = true`.

4.2 Consultar un mundo de colisiones antiguo

```
using Unity.NetCode;
using Unity.Physics;

var physicsHistory = SystemAPI.GetSingleton<PhysicsWorldHistorySingleton>();

physicsHistory.GetCollisionWorldFromTick(
    serverTick,
    interpolationDelay,
    ref physicsWorld,
    out var collisionWorld);

// Use collisionWorld for raycasts or other physics queries.
```

4.3 Clonación profunda de Collider

Parámetro	Valor predeterminado	Significado
DeepCopyDynamicColliders	true	Clonación profunda de los colliders dinámicos; activada por defecto en 6.5.0.
DeepCopyStaticColliders	false	Clonación profunda de los colliders estáticos.

La línea 6.5.0 activa por defecto la clonación profunda de los colliders de Ghosts dinámicos para evitar aserciones de blob asset.

4.4 Búfer de historial de física

Versión	Tamaño del historial
Hasta 1.4.x	16 ticks.
1.5.0+	32 ticks.
Personalizado	GhostSystemConstants.SnapshotHistorySize mediante una define del compilador.

5. Multi Physics World

Se usa un mundo de física aparte para los efectos exclusivos del cliente, como los VFX o las partículas, que no participan en la simulación en red.

5.1 Configuración

1. Añadir NetCodePhysicsConfig a la SubScene.

2. Poner **Client Non Ghost World** en un valor distinto de cero, como 1.
3. Añadir PhysicsWorldIndex con ese mismo valor a los objetos físicos exclusivos del cliente.

5.2 Grupo de systems de física personalizado

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Physics.Systems;

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation)]
[UpdateInGroup(typeof(FixedStepSimulationSystemGroup))]
public partial class VfxPhysicsGroup : CustomPhysicsSystemGroup
{
    public const int WorldIndex = 1;

    public VfxPhysicsGroup() : base(WorldIndex, shareStaticColliders: true)
    {
    }
}
```

Con `shareStaticColliders: true`, el mundo de física personalizado comparte los colliders estáticos con el mundo de física principal.

6. Physics Proxy

El Physics Proxy se usa cuando un Ghost Predicted debe interactuar con una entidad de física exclusiva del cliente.

- CustomPhysicsProxyAuthoring crea la entidad proxy.
- CustomPhysicsProxyDriver vincula el proxy al Ghost Predicted.
- SyncCustomPhysicsProxySystem sincroniza posición y rotación.

7. PredictedFixedStepSimulationSystemGroup

PredictedFixedStepSimulationSystemGroup solo se actualiza cuando se cumplen todas las condiciones necesarias.

Requisito	Significado
El singleton NetworkStreamInGame existe	La conexión está en juego.
Existe al menos un Ghost Predicted	La física predicha tiene algo que simular.

Requisito	Significado
La tasa de ticks fija está en marcha	La simulación de paso de tiempo fijo está activa.

Mientras una conexión de red no esté in-game o no hayan llegado los Ghosts Predicted, la física predicha no se ejecuta.

7.1 Física antes de la conexión

Si la física debe ejecutarse antes de que exista una conexión de red, hay que desactivar la configuración de física predicha para que pueda correr la física normal de FixedStepSimulationSystemGroup.

```
using Unity.Entities;
using Unity.NetCode;

[UpdateInGroup(typeof(InitializationSystemGroup))]
[CreateAfter(typeof(PredictedPhysicsConfigSystem))]
public partial struct DisablePredictedPhysics : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.World.GetExistingSystemManaged<PredictedPhysicsConfigSystem>()
            .Enabled = false;
    }
}
```

8. Limitaciones

Limitación	Significado
Tick parcial no compatible	La física no usa ticks parciales; en su lugar, hay que usar Physics Interpolation.
Límites de depuración multi-world	Los sistemas de depuración de Unity Physics no son totalmente compatibles con configuraciones multi-world.
Ghost Predicted mínimo	Para que la física predicha se ejecute hace falta al menos un Ghost Predicted.

9. Documentación relacionada

- 19_Netcode Prediction & Rollback.md — comportamiento del bucle de predicción.
 - 22_Netcode Ghost Optimization · Importance · Relevancy.md — cambio de Ghosts fuera de la predicción.
 - 24_Netcode Profiler & Debugging.md — análisis de costes de física predicha.
-

10. Solución de problemas

Síntoma	Causa / Solución
La física no se ejecuta	Comprobar que existe un Ghost Predicted y un NetworkStreamInGame.
La física da tirones	Revisar la cuantización y el suavizado de predicción.
El raycast de compensación de lag falla	Activar EnableLagCompensation y comprobar la cobertura del buffer de historial.
Aserción de blob asset	Revisar la configuración de clonación profunda de colliders; la clonación dinámica viene activada por defecto en 6.5.0.
La física exclusiva del cliente no se ejecuta	Hacer que PhysicsWorldIndex coincida con el valor de Client Non Ghost World .
La CPU de física predicha es demasiado alta	Activar el agrupamiento y pasar a Interpolated los Ghosts que no lo necesiten.

Capítulo 27. Profiler y depuración de Netcode

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Descripción general

Netcode for Entities incluye un **Netcode Profiler** para analizar el estado de la conexión, los snapshots, la predicción, la interpolación y el ancho de banda. En la línea 1.12.0, el Network Debugger basado en navegador se sustituyó por los módulos integrados del Unity Profiler.

2. Netcode Profiler

2.1 Acceso

El Unity Profiler incluye dos módulos Netcode en Unity 6.0+.

Módulo	Qué inspeccionar
Server Profiler	Envío de Ghosts del lado del servidor, composición del snapshot y ancho de banda.
Client Profiler	Recepción del cliente, ticks de predicción y estado de interpolación.

2.2 Funciones clave

Función	Significado
Ghost Snapshot View	Vista en árbol del tamaño y la composición del snapshot de cada Ghost.
Tick navigation	Navegación entre los fotogramas en los que se enviaron o recibieron snapshots.
Search fields	Búsqueda en Ghost Snapshot TreeView y Prediction Error List.
Client ticks	Visualización de Prediction Tick e Interpolation Tick.
Context menu	Inspeccionar Ghost Prefabs y Components desde las etiquetas del TreeView, en la línea 6.5.0.
Average per-entity column	Muestra el tamaño promedio de datos por entidad.

2.3 NETCODE_PROFILER_ENABLED

La línea 1.12.0 eliminó la necesidad de la define de scripting `NETCODE_PROFILER_ENABLED`. El Profiler está disponible sin esa define.

3. Network Debugger

El Network Debugger basado en navegador está obsoleto desde la línea 1.12.0; en su lugar conviene usar el Netcode Profiler.

El flujo antiguo era **Multiplayer** → **Open NetDbg**, que abría una vista en el navegador para la composición de los snapshots, la distribución de tipos de Ghost y el análisis de tamaño.

4. Ventana Multiplayer PlayMode

La ventana Multiplayer PlayMode es el flujo de trabajo principal del Editor para probar configuraciones de cliente y servidor en modo Play.

Función	Significado
Play Mode Type	Elegir entre Client + Server, Client Only o Server Only.
Thin Client count	Configurar clientes ficticios.
Simulator	Simular latencia de red y pérdida de paquetes.
Importance Visualizer	Visualizar Importance Scaling en la línea 1.8.0+.

5. Listas de verificación de depuración

5.1 Conexión

- ☐ `Application.runInBackground = true`
- ☐ Protocol versions match between client and server
- ☐ `NetworkStreamInGame` is present
- ☐ Firewall and port are open
- ☐ Unity Transport version is compatible

5.2 Sincronización de Ghost

- ☐ `GhostAuthoringComponent` is present
- ☐ `GhostField` attributes are correct
- ☐ Quantization is appropriate

- Smoothing is configured
- Ghost prefab is registered on both sides

5.3 Predicción

- Systems run in PredictedSimulationSystemGroup
- Queries include Simulate
- Simulation code is deterministic enough for prediction
- Client and server run the same simulation code
- Prediction smoothing is registered where needed

5.4 Input y comandos

- Input is gathered in GhostInputSystemGroup
- AutoCommandTarget requirements are met, or CommandTarget is set manually
- Command payload is 1024 bytes or less
- One-shot input uses InputEvent

6. Menú Prefab List

Desde la línea 1.11.0 hay un menú Prefab List para listar e inspeccionar con rapidez los Ghost Prefabs registrados en el proyecto o en la jerarquía de la escena.

7. Componentes útiles para depuración

Componente	Uso
NetworkStreamConnection	Inspeccionar el estado de la conexión.
NetworkId	Inspeccionar el ID de cliente.
GhostInstance	Inspeccionar el tipo y el ID del Ghost.
PredictedGhost	Inspeccionar el estado de predicción y el tick aplicado.
NetworkSnapshotAck	Inspeccionar el estado del ACK del snapshot y el RTT estimado.
GhostOwner	Inspeccionar el ID del propietario del Ghost.

7.1 Entity Inspector

En Entities Hierarchy, seleccionar una entidad Ghost para inspeccionar:

- valores GhostField en vivo,

- datos de tick de predicción,
 - estado de conexión de red.
-

8. Volcados de paquetes

Los volcados de paquetes se usan cuando hace falta analizar paquetes de red a bajo nivel. En la línea 1.6.2, los registros de marcas de tiempo de los paquetes incorporaron `UnityEngine.Time.frameCount`, lo que facilita correlacionarlos por fotograma.

9. Cuellos de botella frecuentes

Área	Dónde buscar
Ancho de banda del snapshot	Profiler Ghost Snapshot View; inspeccionar el tamaño por entidad.
Resimulación de predicción	Profiler Client Tick; revisar el número de ticks predichos.
GhostSendSystem	Profiler Timeline; revisar la duración del system.
Coste de serialización	Revisar el acceso de escritura a GhostField y la frecuencia de cambios.
Resimulación de física	Revisar la duración de PredictedFixedStepSimulationSystemGroup.

10. Editor Analytics

La línea 1.12.0 recopila analíticas del Editor sobre cómo se usa el Netcode Profiler.

11. Documentación relacionada

- `Optimizations and Debugging/03_Profiler · Bottleneck Analysis.md` — flujo de trabajo general de análisis de rendimiento en DOTS.
 - `22_Netcode Ghost Optimization · Importance · Relevancy.md` — controles de ancho de banda.
 - `23_Netcode Physics Integration & Lag Compensation.md` — costes de física predicha.
-

12. Solución de problemas

Síntoma	Causa / Solución
Sin datos del Netcode Profiler	Comprobar que la prueba se ejecuta en Play Mode cliente-servidor y con Unity 6.0+.
El Ghost Snapshot View es grande, pero el juego va bien	Buscar Ghosts de baja prioridad con demasiados campos o una frecuencia de envío alta.
Los errores de predicción aumentan	Revisar las queries de Simulate, las rutas de código deterministas y el manejo del tick del input.
No aparece el flujo de trabajo del Network Debugger	Está obsoleto; usar los módulos Netcode del Unity Profiler.
Cuesta correlacionar los registros a nivel de paquete	Desde la línea 1.6.2, usar volcados de paquetes con marcas de tiempo que incluyen el número de fotograma.

Parte III. Optimización y depuración

Capítulo 28. Disposición de chunks y TypeManager

1. El chunk de 16 KB

Cada archetype guarda sus entidades en **chunks de 16 KB**. El chunk contiene:

- 1. Una pequeña cabecera (metadatos: ID del archetype, números de versión, máscaras de bits de habilitación y versiones de cambio).
- 2. Una región **SoA** contigua: una columna por cada tipo de componente, y cada columna es un array de valores.
- 3. Relleno hasta el límite de los 16 KB.

La capacidad del chunk, es decir, cuántas entidades caben, se deduce de esta fórmula:

$$\text{capacity} \approx (16 \text{ KB} - \text{header}) / \text{sum}(\text{sizeof}(\text{component_i}))$$

Un archetype ajustado, con pocos componentes pequeños, puede albergar cientos de entidades. Uno ancho, con muchos componentes grandes, solo unas pocas. Los chunks poco ocupados desperdician los bytes que podrían haber alojado más entidades.

2. TypeManager — el registro de componentes

TypeManager registra cada componente, buffer y system al arrancar el world y le asigna un **índice de tipo** estable. De ese registro, el runtime lee:

Campo	Para qué se usa
TypeIndex	Identificador compacto que se usa en las queries y en los metadatos del chunk.
SizeInChunk	Bytes que ocupa este componente por entidad dentro de un chunk.
Alignment	Alineación de la columna dentro del chunk.
ElementSize	Tamaño de cada elemento del buffer (para IBufferElementData).
HasBlobReferences / HasEntityReferences	Activa la validación durante la serialización y las operaciones de movimiento.

Rara vez se accede directamente a `TypeManager`. Lo importante es que **añadir nuevos tipos de componente, renombrarlos o moverlos entre ensamblados dispara una reconstrucción del registro de tipos**, y ahí se va el tiempo de arranque en los proyectos grandes.

En Entities 1.4 se anunció que `TypeManager.Initialize` pasó a ser unas 2 veces más rápido en proyectos grandes al evitar escanear ensamblados innecesarios. La versión 6.5 hereda esa optimización.

3. Inspeccionar la disposición de los chunks: la ventana Archetypes

Window → Entities → Archetypes lista todos los archetypes del world. Para cada fila:

- **Entities** — número de entidades en vivo.
- **Chunks** — número de chunks en uso.
- **Entities / Chunk** — ocupación media.
- **Capacity** — máximo teórico por chunk.

Qué vigilar:

Lectura	Interpretación
Muchos archetypes con 1 entidad por chunk	Explosión de archetypes por tags demasiado granulares. Conviene consolidarlos.
Un archetype con muchas entidades y alta ocupación	Es lo deseable: enemigos, proyectiles, balas.
Entities / Chunk ≪ Capacity	Fragmentación: la ocupación es irregular porque las entidades no dejan de migrar a otros archetypes (cambios estructurales).
Capacity = 1	El archetype es demasiado ancho (componentes demasiado grandes). Revisar si hay componentes managed o shared sobredimensionados.

4. Diseñar componentes que respeten los archetypes

Reglas prácticas, en orden aproximado de impacto:

1. **Mantener no gestionados los structs no gestionados.** Un solo `IComponentData` gestionado obliga a almacenar los datos en una tabla aparte por entidad y rompe Burst.

2. **No dejar relleno en los structs.** Poner float3 antes que float, no después: los huecos de alineación inflan SizeInChunk.
 3. **Separar los campos de uso poco frecuente.** Si solo un system lee un campo, conviene moverlo a un componente aparte. Las columnas que no se usan siguen costando bytes por entidad.
 4. **Preferir componentes habilitables a añadir y quitar componentes en cada fotograma.** Ver 13_Structural Change & Safety.md: conmutar un bit es mejor que reorganizar el archetype.
 5. **Limitar el conjunto de valores de los componentes shared.** Cada valor nuevo crea un bucket de chunk nuevo.
 6. **Los tag components cuestan cero bytes, pero aun así añaden un archetype.** No conviene etiquetar cada entidad con un marcador único.
-

5. Números de versión del chunk

Cada columna de componente lleva una **versión de cambio**: un contador de 32 bits que se incrementa cada vez que un system escribe mediante RefRW<T>.ValueRW, SetComponentData o el SetComponent de un ECB.

Los systems pueden saltarse los chunks cuya versión no ha cambiado desde la última ejecución:

```
var query = state.GetEntityQuery(ComponentType.ReadWrite<Health>());
query.SetChangedVersionFilter(typeof(Health));
```

Cuando una query se recorre con un filtro de cambio, solo se visitan los chunks modificados desde la última ejecución del system. Así es como un patrón del tipo "al cambiar Health, actualizar la UI" sale barato.

Advertencias:

- El filtro trabaja a **nivel de chunk**, no de entidad. La escritura de una sola entidad ensucia el chunk entero.
 - Leer mediante RefRW<T>.ValueRW, aunque no se modifique nada, incrementa la versión.
 - Conmutar un componente habilitable también incrementa la versión.
-

6. Almacenamiento de BlobAsset

BlobAssetReference<T> guarda datos grandes y de solo lectura (mallas, tablas de LOD, campos de navegación) **fuera** del chunk. El chunk solo guarda un puntero. Así se evita reducir la capacidad del chunk cuando muchas entidades comparten los mismos datos horneados.

Consejo práctico: si un componente está dominado por un FixedList o un DynamicBuffer que es idéntico en todas las entidades de un mismo tipo, conviene guardar esa carga grande en un blob asset al que apunte un componente ligero.

7. Ejemplos

7.1 Un archetype ancho que conviene evitar

```
public struct EnemyState : IComponentData
{
    public float3 Position;
    public quaternion Rotation;
    public float Health;
    public float HealthMax;
    public float Stamina;
    public float StaminaMax;
    public int AIState;
    public float3 LastKnownPlayerPos;
    public float TargetDistance;
    public Entity Target;
    public FixedList512Bytes<float3> PathBuffer;    // 512 bytes per entity!
}
```

El campo PathBuffer consume él solo 512 bytes por entidad. La capacidad de cada chunk de 16 KB cae a unas pocas decenas. Conviene dividirlo:

```
public struct EnemyStats : IComponentData
{
    public float Health, HealthMax, Stamina, StaminaMax;
    public int AIState;
    public Entity Target;
}

public struct EnemyPathPoint : IBufferElementData
{
    public float3 Value;
}
```

Ahora el buffer dinámico vive en su propio almacenamiento y la densidad del chunk se recupera.

7.2 Filtro de cambio para actualizar la UI a bajo coste

```

var query = state.GetEntityQuery(ComponentType.ReadWrite<Health>(),
                                ComponentType.ReadWrite<HealthBarLink>());
query.SetChangedVersionFilter(typeof(Health));

state.Dependency = new UpdateHealthBarJob { /* ... */ }
    .ScheduleParallel(query, state.Dependency);

```

Solo se recorren los chunks en los que Health ha cambiado.

8. Solución de problemas

Síntoma	Causa / Solución
Inicio lento en proyecto grande	TypeManager.Initialize escanea demasiados ensamblados. Revisar la traza de arranque del Profiler y comprobar que se usa una versión reciente de Entities.
Capacidad de 1 en un archetype	El componente es demasiado grande (capacidad del buffer, blob embebido). Trasladar los datos voluminosos a un BlobAssetReference<T> o a un IBufferElementData.
El filtro de cambio se dispara en cada fotograma aunque "no haya cambiado nada"	Algún código lee mediante RefRW<T>.ValueRW sin escribir. Cambiar a RefRO<T> donde se pueda.
El número de archetypes no deja de crecer	Tags por entidad de alta cardinalidad, o componentes shared con valores continuos. Consolidarlos.
Entities / Chunk muy por debajo de Capacity	Cambios estructurales: las adiciones y eliminaciones dispersan las entidades entre archetypes. Cambiar a componentes habilitables.
Un componente managed acapara el perfil de CPU	Trasladar el campo managed a un UnityObjectRef<T> o dividir el componente; las rutas no gestionadas son mucho más rápidas.

Capítulo 29. Systems · Entity Inspector · Query Window

1. Descripción general

Las tres ventanas de herramientas de Entities en las que más se trabaja:

Ventana	Se abre desde	Responde a
Entities Hierarchy	Window → Entities → Hierarchy	"¿Qué entidades existen? ¿Qué componentes tienen?"
Systems	Window → Entities → Systems	"¿Qué systems se están ejecutando, en qué orden y cuánto tardan?"
Query	Window → Entities → Query	"Dado un conjunto de componentes, ¿qué entidades coinciden? ¿Qué systems la consultan?"

Hay una cuarta, **Archetypes** (Window → Entities → Archetypes), que se trata en 01_Chunk Layout & TypeManager.md.

2. Entities Hierarchy

Funciona como la Hierarchy en vivo, pero con entidades en lugar de GameObjects.

Selector de world (parte superior de la ventana)

Permite alternar entre DefaultGameObjectInjectionWorld (un jugador), ClientWorld/ServerWorld (Netcode) o un world de un fixture de prueba. Al cambiar de world, el árbol se vuelve a calcular sobre las entidades de ese world.

Vista en árbol

- Cada fila es una entidad; las filas hijas son entidades enlazadas mediante Parent o LinkedEntityGroup.
- Columnas de la derecha: número de componentes y número de hijos.
- Al seleccionar una fila, sus componentes aparecen en el **Inspector**.

Búsqueda

- `c:Health` — filtra las entidades que tienen un componente `Health`.
- `n:Enemy` — filtro por nombre.
- `w:Server` — filtro por world.
- Se pueden combinar: `c:Health c:Enemy n:boss`.

Inspector

El Inspector de una entidad muestra:

- Sección **Components**: cada `IComponentData` con sus valores actuales. Se pueden editar en vivo en modo Play.
- **Relationships**: enlaces con otras entidades (Parent, Child, LinkedEntityGroup).
- **Enabled state**: un conmutador por cada `IEnableableComponent`.

La edición en vivo es **de escritura directa**: cambiar un campo incrementa al instante la versión de cambio de ese chunk.

3. Systems

La ventana Systems lista todos los systems registrados, agrupados por su `SystemGroup`, en el orden real en que los ejecuta el planificador.

Columnas que conviene leer primero

Columna	Significado
Name	El tipo del system.
Namespace	Dónde reside (a menudo permite distinguir un paquete del código propio).
Time (ms)	Tiempo del hilo principal dedicado a <code>OnUpdate</code> . Se actualiza casi en cada fotograma.
Queries	Número de queries que ha usado el system.
Entity Count	Entidades que coinciden con la query primaria.

Qué buscar

Lectura	Interpretación
Un system muestra 0 ms pero aparece en verde	Lo omite <code>RequireForUpdate<T></code> o <code>[RequireMatchingQueriesForUpdate]</code> . Es correcto.

Lectura	Interpretación
Un system muestra siempre un valor de ms alto	Es trabajo real. Profundizar en el Profiler para ver el desglose.
Un system muestra -- en el número de entidades	No hay ninguna query coincidente (o la query aún no se ha construido).
Dos systems aparecen fuera del orden esperado	La cadena [UpdateBefore] / [UpdateAfter] es incorrecta, o están en grupos distintos.

Habilitar / deshabilitar

Cada system tiene una casilla: al desmarcarla, el system se omite en modo Play sin necesidad de recompilar. Ideal para comparaciones A/B.

Panel de detalle del system

Al seleccionar un system se abre un panel lateral con:

- **Queries** — el conjunto de componentes de cada query (All/Any/None/Disabled/Present/Absent).
- **Dependencies** — los systems sobre los que este tiene [UpdateBefore]/[UpdateAfter], ya resueltos al orden real de ejecución.
- **Type Info** — indica si [BurstCompile] está aplicado a la clase y a cada método.

Desde la 1.4+ / 6.x, la **pestaña Queries muestra los estados de componente Disabled / Present / Absent / None**, así que las queries de componentes habilitables ya no resultan ambiguas.

4. Ventana Query

Diseñada a medida para responder preguntas del tipo "¿qué coincide con X?".

Flujo de trabajo:

1. Añadir componentes All con el botón de más.
2. Añadir Any / None si hace falta.
3. La ventana lista las **entidades coincidentes** en el panel izquierdo y los **systems que ejecutan esta query** en el panel derecho.

Casos de uso:

- "¿Qué systems leen Health?" → añadir Health a All; todos los systems listados a la derecha lo usan.

- "¿Por qué esta entidad no coincide con la query de mi system?" → pegar la query del system y comparar los componentes presentes con los de la entidad en la ventana Hierarchy.
- "¿Se cuelan prefabs en esta query?" → los prefabs se distinguen por un icono en la lista de resultados (añadido en 1.4+).

La ventana Query también permite guardar una query y reutilizarla durante la sesión.

5. Journaling — el diagnóstico de último recurso

Window → Analysis → Entities Journaling (que se activa en Project Settings → Entities → Journaling) registra en un buffer circular cada cambio estructural, cada escritura de componente y cada actualización de system. Vistas de reproducción:

- **Operations** — cada cambio registrado, con su marca de tiempo, el system de origen y la entidad afectada.
- **Entities** — una línea de tiempo por entidad de quién la modificó y cuándo.

El Journaling tiene un coste en tiempo de ejecución: conviene activarlo solo mientras se investiga un error y dejarlo desactivado durante el desarrollo normal.

6. Flujo de trabajo práctico — depurar una actualización que no llega

Error habitual: "Mi system procesó la entidad pero el componente no cambió". Procedimiento:

1. **Entities Hierarchy** → encontrar la entidad, confirmar que tiene el componente.
 2. **Inspector** → antes de Play, anotar el valor. Pulsar Play y observarlo.
 3. Si el valor no cambia: **ventana Systems** → localizar el system implicado y comprobar **Time (ms)**. Si es 0, se está omitiendo.
 4. Si el tiempo es mayor que 0 pero el valor sigue sin cambiar: **ventana Query** → pegar la query del system y los componentes de la entidad. Comprobar que la entidad coincide.
 5. Si aún no se resuelve: activar el **Journaling** durante 1 o 2 fotogramas. Buscar escrituras en ese componente desde otros systems que pisen el resultado.
-

7. Solución de problemas

Síntoma	Causa / Solución
Entities Hierarchy está vacía en modo Play	El selector de world apunta a un world equivocado (por ejemplo, solo Client, sin Server).
Un campo en vivo no se actualiza	Se está viendo una entidad cacheada tras una recarga de dominio. Volver a seleccionarla.
La ventana Systems indica 0 ms, pero el system sin duda se ejecuta	El Profiler puede mostrarlo bajo el marcador de un chunk job si el trabajo real está en ScheduleParallel. En ese caso, el OnUpdate del hilo principal es realmente 0; hay que mirar los tiempos de los jobs en el Profiler.
La ventana Query encuentra una entidad que mi system no encuentra	El system añade un WithAll<T>() o un filtro de cambio que no se le indicó a la ventana Query. Deben coincidir exactamente.
La ventana Journaling está vacía	El Journaling no está activado en Project Settings, o el buffer circular se sobrescribió.
Las entidades y los GameObjects parecen desincronizados	La SubScene está abierta (ícono blanco). Cerrarla (ícono amarillo) para que el Baker se ejecute y aparezcan las entidades.

Capítulo 30. Profiler · Análisis de cuellos de botella

1. Descripción general

El Unity Profiler (Window → Analysis → Profiler) es la herramienta principal para responder, en DOTS, a la pregunta de "¿por qué va lento este fotograma?". Entities emite suficientes marcadores como para atribuir el tiempo a systems, jobs y chunks concretos, una vez que se sabe dónde mirar.

Esta página explica cómo leer el perfil, cuáles son los patrones típicos de cuello de botella en DOTS y cómo actuar ante ellos.

2. Antes de analizar el rendimiento

1. **Usar Deep Profile con moderación.** Es útil para seguir una ruta gestionada concreta, pero distorsiona los tiempos: nunca hay que fiarse de Deep Profile para las cifras de rendimiento.
 2. **Activar "Record in Editor & Standalone"** para capturar también las builds, no solo los fotogramas del Editor.
 3. **Apuntar el Profiler al player**, no al Editor, al medir el rendimiento de producción. La sobrecarga del Editor es notable.
 4. **Dejar correr el juego unos segundos antes de leer los datos;** los primeros fotogramas incluyen la recarga de dominio y el trabajo de JIT.
-

3. Interpretación del módulo CPU Usage

Marcadores de nivel superior dentro de un fotograma de DOTS:

```
PlayerLoop
├─ Initialization.*
├─ Update.ScriptRunBehaviourUpdate           (MonoBehaviour.Update)
├─ SimulationSystemGroup                    ← DOTS simulation lives here
│   ├── SystemA.OnUpdate
│   ├── SystemB.OnUpdate
│   └─ EndSimulationEntityCommandBufferSystem.Playback
├─ FixedStepSimulationSystemGroup
├─ PresentationSystemGroup
├─ Render
└─ WaitForTargetFPS
```

Al desplegar un marcador `SystemA.OnUpdate` se ven sus subllamadas: `ScheduleParallel`, esperas de finalización de jobs, llamadas a `EntityManager`.

4. Dónde se va realmente el tiempo: patrones habituales

4.1 Un `OnUpdate` que acapara el hilo principal

Si un solo `SystemX.OnUpdate` acumula más de 10 ms, el trabajo se está ejecutando en el hilo principal. Conviene comprobar:

- ¿Usa `SystemAPI.Query` en un bucle compacto que debería ser un `IJobEntity.ScheduleParallel`?
- ¿Es un `SystemBase` que un campo gestionado obliga a ejecutar en el hilo principal?
- ¿Se está llamando a `EntityManager.AddComponent/RemoveComponent` entidad por entidad?

4.2 Tiempos de jobs

Los jobs aparecen en la vista **Timeline** (o bajo "Workers" en la vista Hierarchy). Conviene buscar:

- Una lane de job totalmente saturada mientras otras están inactivas: el trabajo no se reparte entre los chunks.
- El hilo principal esperando un job (marcador `JobHandle.Complete()`): se ha llamado a `.Complete()` demasiado pronto; conviene retrasarlo dentro del fotograma.
- Nombres de función de Burst muy largos: es normal; el símbolo es la instanciación genérica completa.

4.3 Puntos calientes de `EntityManager`

Conviene buscar marcadores con estos nombres:

- `EntityManager.SetArchetype / Move Entity`: exceso de cambios estructurales. Ver `13_Structural Change & Safety.md`.
- `EntityCommandBufferSystem.OnUpdate (Playback)`: una reproducción de ECB costosa. Suele significar que se registran demasiados comandos por fotograma.
- `TypeManager.Initialize`: solo al arrancar; si aparece en cada fotograma, algo está recargando el world.

4.4 Marcadores `GC.Alloc`

Cualquier `GC.Alloc` dentro de la simulación es una regresión en DOTS. Causas habituales:

- Un `SystemBase` que reserva memoria en su `OnUpdate`.
- Un `foreach` sobre una colección gestionada dentro de un `system`.
- Boxing, por ejemplo `object.ToString()` sobre un `struct`.

Los `systems` con Burst deben mostrar cero asignaciones en el módulo Memory del Profiler.

5. Módulos del Profiler que conviene fijar

- **CPU Usage**: la línea de tiempo y la jerarquía de marcadores.
- **Memory**: el heap reservado; vigilar que `GC.Used Memory` no suba.
- **Entities** (si está disponible): un módulo específico de DOTS que muestra los tiempos de cada `system` a lo largo del tiempo.

El módulo Entities Profiler llegó con la 1.8.0 / 6.x y se alinea con la ventana Systems sobre una línea de tiempo. Ante picos que abarcan varios fotogramas, es la forma más rápida de atribuirlos.

En proyectos en red, conviene combinarlo con `../DOTS Workflows/24_Netcode Profiler & Debugging.md` para examinar los snapshots de Ghost, los ticks de predicción y el ancho de banda propio de Netcode.

6. Cuellos de botella habituales y soluciones

Patrón	Síntoma en el Profiler	Solución
Entities.ForEach por entidad (legacy)	Un <code>OnUpdate</code> que carga mucho el hilo principal, con muchas muestras pequeñas	Migrar a <code>IJobEntity.ScheduleParallel</code> .
AddComponent/RemoveComponent en cada fotograma	El marcador <code>Move Entity</code> acapara el tiempo	Convertir el estado a un componente habilitable.
Sobrecarga de ECB	<code>...ECBS.Playback</code> consume varios ms	Agrupar los comandos (uno por chunk en vez de por entidad) o subir el tamaño del lote.
Explosión de valores de componentes <code>shared</code>	<code>SetSharedComponent</code> acapara el tiempo	Agrupar el valor en intervalos (cuantizarlo a menos claves distintas).

Patrón	Síntoma en el Profiler	Solución
<code>.Complete()</code> llamado a mitad del fotograma	Una espera larga de <code>JobHandle.Complete</code> en el hilo principal	Retrasar la finalización al final del fotograma, o reestructurar el código para que el hilo principal aún no necesite el resultado.
Burst recurre al fallback	El método caliente aparece con el nombre de C# sin transformar	Un tipo capturado no blittable hizo que volviera a IL. Revisar el Burst Inspector.
Fragmentación de archetypes	Muchos chunks pequeños; marcadores <code>EntityManager.*Query</code> altos	Los archetypes son demasiado granulares: consolidar tags o usar componentes habilitables.
Regresión en el tiempo de baking	Marcadores Bake durante Play	La recarga de dominio volvió a disparar el baking. Suele ser solo del Editor; confirmarlo en una build.

7. Ejemplo práctico de presupuesto de fotograma

Imaginemos un presupuesto de 16,6 ms (60 Hz). Tras descartar Render:

- `SimulationSystemGroup`: 6 ms
 - `EnemyAISystem.OnUpdate`: 2,1 ms (Burst, jobs paralelos en 4 workers)
 - `MovementSystem.OnUpdate`: 1,4 ms (Burst, paralelo)
 - `EndSimulationECBS.Playback`: 1,8 ms ← sospechoso
 - todo lo demás: 0,7 ms

1,8 ms de reproducción de ECB para lo que deberían ser unos pocos cambios estructurales suele ser la mayor palanca de mejora. Conviene revisar cuántos comandos hay:

1. Entities Journaling durante 1 fotograma → recuento de `Instantiate/AddComponent/DestroyEntity` por system.
2. Si un system registra cientos de comandos en cada fotograma, agruparlos (un comando por chunk, o un grabador paralelo con claves de ordenación por índice de chunk).

8. Perfilar una build independiente

Para una build real:

1. **File** → **Build Profiles** → **Development Build + Autoconnect Profiler**.
 2. El player se ejecuta y el Profiler captura desde el proceso remoto.
 3. Los resultados reflejan mejor lo que verán los jugadores. Las mediciones del Editor pueden ser de 2 a 5 veces más lentas por motivos ajenos al código.
-

9. Solución de problemas

Síntoma	Causa / Solución
El Profiler no muestra marcadores de DOTS	El módulo del Profiler los está filtrando. Activar "Scripts" y "Jobs" en la configuración del módulo.
El fotograma va lento, pero no hay ningún marcador caliente	Puede ser una espera del hilo de render. Activar el módulo GPU y la vista de timeline.
Deep Profile muestra puntos calientes distintos	Deep Profile distorsiona las asignaciones y el recuento de llamadas. Para los tiempos relativos, fiarse del perfil sin Deep Profile.
El método de Burst muestra una llamada gestionada en la pila	Es un fallback: Burst no pudo compilarlo. Revisar la salida del Burst Inspector.
Picos en el marcador GC.Collect	Hay asignaciones gestionadas en algún system. Acotarlas con el módulo Memory → Snapshot.
El tiempo del system oscila mucho de un fotograma a otro	El trabajo crece con el número de entidades y los chunks entran y salen de la query en cada fotograma. Revisar los cambios estructurales.

Capítulo 31. Auditoría de referencias a objetos administrados

1. Descripción general

Las referencias a objetos Unity gestionados son una de las maneras más fáciles de sacar sin querer a un system ECS de la ruta rápida. Esta auditoría ayuda a encontrar los puntos donde los datos de componentes, los systems o el código de renderizado guardan referencias a `UnityEngine.Object` de formas que bloquean Burst, los jobs o un baking limpio de la SubScene.

Conviene aplicarla tras la migración principal a ECS, o siempre que una traza del Profiler revele trabajo gestionado donde se esperaba código orientado a datos.

2. Qué buscar

Patrón	Por qué importa
<code>class .*: IComponentData</code>	Datos de componente gestionados; válidos, pero por lo general poco idóneos para las rutas calientes.
<code>UnityEngine.Object</code>	Indicio general de referencias a objetos Unity gestionados.
<code>GameObject</code> , <code>Transform</code> , <code>Component</code>	Las referencias a objetos de la escena en tiempo de ejecución no suelen tener cabida en los datos de simulación.
<code>Mesh</code> , <code>Material</code> , <code>Texture</code> , <code>AudioClip</code>	A menudo deberían ser <code>UnityEngine.ObjectRef<T></code> o datos cacheados del lado de presentación.
<code>Resources.Load</code>	Búsqueda de assets en tiempo de ejecución; suele ser mejor gestionarla con sistemas de baking o de contenido.
<code>ScriptableObject</code>	Útil como fuente de authoring, pero los jobs de tiempo de ejecución deben leer valores horneados o blobs.

Conviene acotar la auditoría al código que da a ECS. Los `MonoBehaviours` de UI o las herramientas de Editor no tienen por qué seguir patrones DOTS.

3. Reglas de clasificación

Hallazgo	Acción probable
Un componente gestionado en una query de simulación caliente	Convertirlo a un IComponentData no gestionado, a un BlobAssetReference<T> o a un UnityObjectRef<T>.
Un componente gestionado que solo usa la presentación	Conservarlo si está aislado de las rutas de simulación con Burst o jobs.
Un ScriptableObject que se lee en cada fotograma	Hornear los valores necesarios en un componente o en un blob asset.
Una referencia a una malla o un material en datos ECS	Usar UnityObjectRef<Mesh> / UnityObjectRef<Material> y desreferenciar en el código de presentación.
Un GameObject o Transform guardado en datos de componente	Sustituirlo por datos ECS o llevar la referencia a un puente de presentación híbrido.

4. Ejemplo de auditoría

```
// Suspicious: managed component data used as per-entity visual state.
public class EnemyVisual : IComponentData
{
    public Material Material;
}
```

Preferir:

```
using Unity.Entities;
using UnityEngine;

public struct EnemyVisual : IComponentData
{
    public UnityObjectRef<Material> Material;
}
```

Después, desreferenciar en un system de presentación del hilo principal o cachear el material ya resuelto en un registro de renderizadores.

5. Indicios en el Profiler

Síntoma en el Profiler	Qué revisar
Un system de simulación reserva memoria en cada fotograma	Una búsqueda de objeto gestionado, o una asignación de LINQ o de una colección en OnUpdate.

Síntoma en el Profiler	Qué revisar
Burst está desactivado para un system que debería ser solo de datos	Acceso a un componente gestionado o a una API gestionada en la ruta de la query.
El hilo principal hace muchas búsquedas de renderizado pequeñas	Cachear los resultados de <code>UnityObjectRef<T>.Value</code> fuera del bucle por entidad.
La SubScene se rehornea con más frecuencia de la esperada	Los scripts de authoring o las referencias a objetos están creando dependencias innecesarias.

6. Validación

1. Volver a ejecutar los patrones de búsqueda del apartado 2.
2. Comprobar que los systems de simulación calientes solo consultan componentes no gestionados.
3. Revisar el Burst Inspector para los systems y jobs que deberían compilar con Burst.
4. Perfilar una escena representativa y comprobar que las asignaciones gestionadas han desaparecido de la ruta caliente.
5. Entrar en modo Play desde un arranque limpio del Editor para verificar que el baking de la SubScene sigue resolviendo las referencias.

7. Solución de problemas

Síntoma	Causa / Solución
El componente convertido sigue sin poder consultarse en Burst	Queda algún campo gestionado. Revisar también los structs anidados.
<code>UnityObjectRef<T>.Value</code> devuelve null en modo Play	El asset referenciado no se horneó o no se cargó. Revisar el Baker y el flujo de SubScene o de contenido.
El system visual sigue yendo lento tras la conversión	Se está desreferenciando por entidad en cada fotograma. Cachear por asset compartido u hornear una clave de búsqueda compacta.
Los cambios en el <code>ScriptableObject</code> no afectan a los datos en tiempo de ejecución	Los valores ya se hornearon. Volver a importar u hornear la SubScene, o añadir las dependencias de baking adecuadas.
Las referencias exclusivas del Editor fallan en las builds del player	Llevar las APIs del Editor dentro de <code>#if UNITY_EDITOR</code> y hornear datos válidos en tiempo de ejecución.

Parte IV. Migración

Capítulo 32. Entities 1.x → 6.5 Resumen general

1. En qué consiste

Actualizar de Entities 1.x a 6.5 consiste, sobre todo, en **tres migraciones de DOTS** apoyadas en una transición de distribución de paquetes:

1. **Actualización de la distribución del Editor y del paquete.** Se pasa a Unity 6000.4+ (`com.unity.entities 6.4`) o 6000.5+ (Entities 6.5), donde Entities es un Core Package integrado en el Editor.
2. **Limpieza de los datos ECS.** Revisar las referencias a objetos Unity gestionados en el código que da a ECS y migrar los datos de componente de la ruta caliente hacia componentes no gestionados, valores horneados, `BlobAssetReference<T>` o `UnityObjectRef<T>`.
3. **Migración de la API de ECS.** Las APIs antiguas marcadas como obsoletas en Entities 1.4 (`Entities.ForEach`, `IAAspect`) siguen siéndolo en 6.5. Además:
`ComponentLookup.GetRefRWOptional` → `TryGetRefRW`.

Cada migración se documenta en su propio subdocumento de esta carpeta. Esta página es el **mapa**: cuándo hacer cada una y en qué orden.

Cuidado con la nomenclatura: la versión del **paquete** Entities (1.4 / 6.4 / 6.5) es distinta de la versión del Editor de Unity (6000.3 / 6000.4 / 6000.5). Desde la 6.4 se alinean por convención, pero se documentan en notas de versión separadas. Este documento las trata como dos ejes; la página del Changelog (`../Changelog/Entities 1.4 → 6.5 Key Changes.md`) hace lo mismo.

2. Orden recomendado

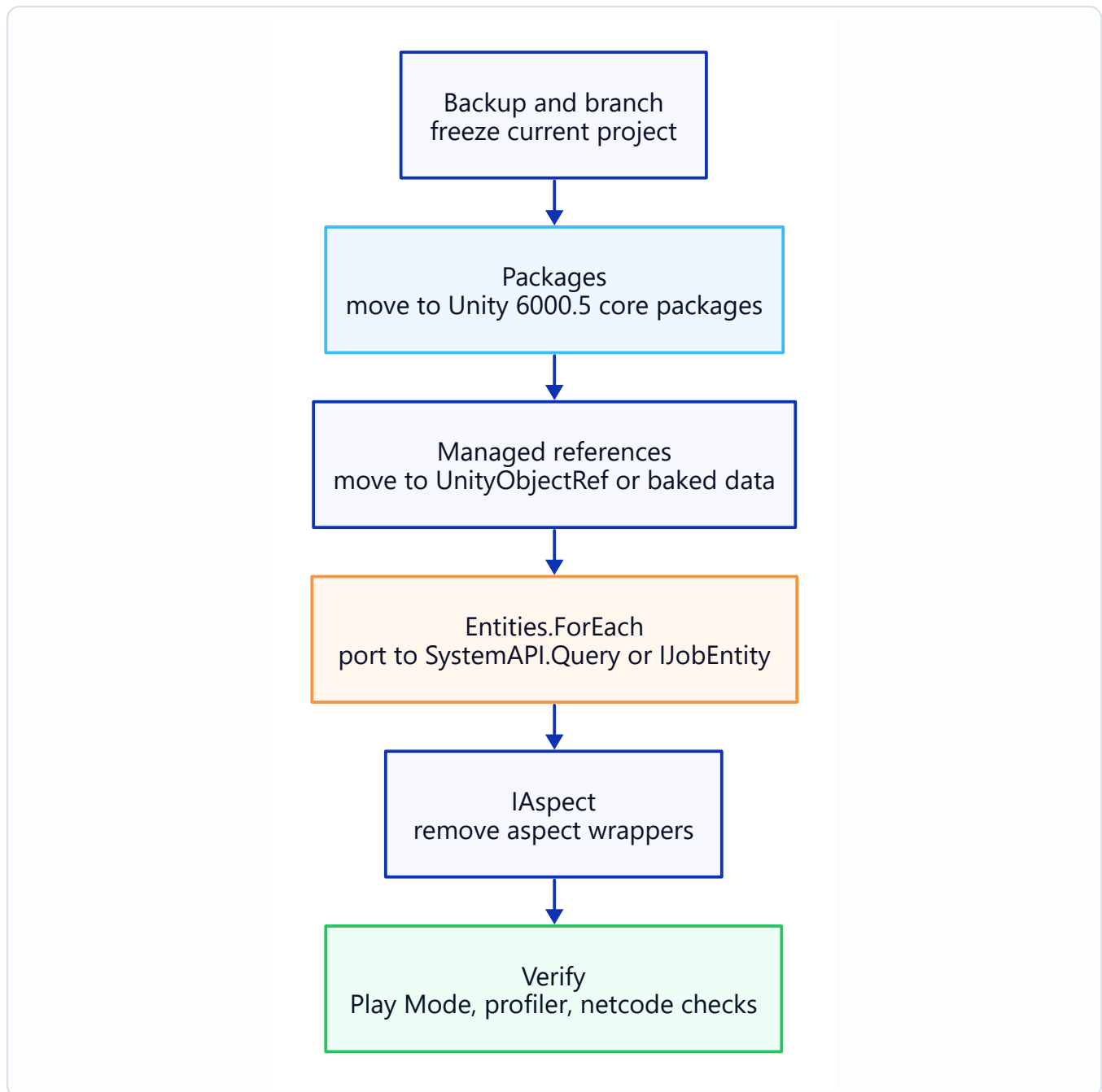


Figure: Entities 1.x to 6.5 Migration Order.

Por qué este orden:

- **El Editor primero**, porque nada más compilará contra las APIs de 6.x hasta que el Editor las conozca.
- **Después, Package Manager → Core Package**: sin este paso, la resolución de paquetes queda duplicada o entra en conflicto.
- **La limpieza de referencias gestionadas antes de refactorizar la iteración**: deja claro el modelo de datos antes de reescribir los systems y los jobs.

- **foreach e IAspect al final**: son internos de ECS y no bloquean el resto del código.

3. Qué permanece igual

Bastante más de lo que parece, lo que hace que la migración asuste menos de lo esperado.

Sin cambios	Notas
IComponentData, IBufferData, ISharedComponentData	Las mismas interfaces, la misma semántica.
ISystem / SystemBase	Los mismos hooks de ciclo de vida, los mismos atributos.
Superficie de la API de EntityManager	CreateEntity, AddComponent, etc.: idénticos.
SystemAPI.Query	Introducida en 1.x y sigue siendo la API central de iteración.
IJobEntity, IJobChunk	La misma estructura; IJobEntity es la opción recomendada por defecto.
EntityCommandBuffer + ECBS	Igual.
Burst, Jobs, Collections, Mathematics	Igual.
Conceptos de Netcode for Entities (Ghost, RPC, Prediction)	La misma arquitectura; lo único que cambia es cómo se distribuye el paquete.

Si el código 1.x ya usa IJobEntity y SystemAPI.Query (y no el antiguo Entities.ForEach), los pasos 4 y 5 de la migración prácticamente desaparecen.

4. Qué cambia, en resumen

Área	1.x	6.5
Distribución	Paquete UPM com.unity.entities	Core Package (sin entrada en el manifest)
Número de versión	1.4.x	6.5.x (alineado con Unity 6000.5)
Referencias a objetos Unity managed en datos ECS	A menudo guardadas en componentes gestionados	Preferir datos horneados, blobs o UnityObjectRef<T> donde corresponda
Iteración de componentes (legacy)	Entities.ForEach marcado como obsoleto en 1.4	Sigue obsoleto y aún compila, con advertencias; su eliminación está prevista para una versión mayor futura

Área	1.x	6.5
Aspects	IAAspect marcado como obsoleto en 1.4	Sigue obsoleto y aún compila; su eliminación está prevista para una versión mayor futura
Referencias opcionales	GetRefRWOptional / GetRefR0Optional obsoletos	TryGetRefRW / TryGetRefR0

El changelog completo del periodo está en `../Changelog/Entities 1.4 → 6.5 Key Changes.md`.

5. Estrategia de copia de seguridad

Obligatorio antes de comenzar:

1. **Crear una rama en git.** Con un nombre descriptivo (`migrate/entities-6.5`). No migrar en `main`.
 2. **Hacer commit de `manifest.json` y `packages-lock.json` en su estado previo a la migración.** Son la referencia fiable para revertir si la resolución falla.
 3. **Etiquetar el commit previo a la migración.** Con `git tag pre-entities-6.5`, las reversiones son triviales.
 4. **Mantener el proyecto abierto en la versión antigua del Editor hasta estar listos.** Abrirlo una vez en 6000.5 cambia el `ProjectVersion.txt` y fuerza la ruta de migración.
-

6. Listas de verificación por paso

6.1 Actualización del Editor

- Instalar Unity 6000.5.x desde Hub.
- No abrir aún el proyecto. Primero, hacer commit de cualquier trabajo pendiente.
- Abrir el proyecto en el nuevo Editor: ofrecerá actualizar `ProjectVersion.txt`. Aceptar.
- Dejar que termine la importación inicial; aparecerán advertencias sobre APIs obsoletas y paquetes que faltan. Los pasos siguientes las resuelven.

6.2 Limpieza de paquetes → `02_Package Manager` → `Core Package.md`

- Quitar `com.unity.entities`, `com.unity.collections`, `com.unity.mathematics` y `com.unity.entities.graphics` de `manifest.json`.
- Borrar `packages-lock.json` y dejar que el Editor lo regenere.
- Quitar también Netcode for Entities si el proyecto lo usa (la 6.5 es una versión de Core Package).

6.3 Referencias a objetos managed → 03_Managed Object References → `UnityObjectRef.md`

- Revisar el código que da a ECS en busca de campos de objetos Unity gestionados.
- Convertir los datos de componente de la ruta caliente a datos ECS no gestionados siempre que se pueda.
- Usar `UnityObjectRef<T>` para las referencias del lado de ECS a assets u objetos de Unity.
- Usar `BlobAssetReference<T>` para datos de juego inmutables de gran tamaño.

6.4 ForEach legacy → 04_foreach → `IJobEntity.md`

- Para los systems que aún usan `Entities.ForEach` o `Job.WithCode`, migrarlos a `IJobEntity + SystemAPI.Query`.

6.5 Eliminación de Aspect → 05_IAspect Removal.md

- Incorporar los métodos del aspect en el punto de llamada con parámetros `RefRW<T>` / `RefRO<T>`.
- Eliminar el struct del `IAAspect`.

7. Pruebas tras la migración

Pruebas de humo que conviene ejecutar antes de dar la migración por terminada:

1. **Modo Play en una SubScene representativa.** Las entidades existen, los systems se actualizan y nada lanza excepciones al arrancar.
2. **Compilar un player independiente.** Los fallos que solo aparecen al compilar (normalmente de serialización) salen aquí.
3. **Ejecutar las pruebas de regresión de gameplay existentes.** Si el rendimiento importa, capturar una traza de referencia del Profiler antes de migrar y compararla.
4. **Comprobar el guardado y la carga.** Los handles de Entity son locales al world y no son estables. En su lugar, guardar IDs propios del juego, claves de contenido o datos de authoring.

5. **Probar en la plataforma de destino.** Algunos backends específicos de plataforma (rarezas de IL2CPP, AOT) se manifiestan aquí.

8. Cuándo revertir

Motivos para una reversión limpia:

- El Editor no consigue abrir el proyecto en absoluto (raro; suele ser una discrepancia en `ProjectVersion.txt`).
- La resolución de `packages-lock.json` genera un grafo sin sentido y borrar el archivo no lo arregla.
- Un paquete de terceros crítico del que se depende aún no es compatible con 6.5.

Plan: `git checkout pre-entities-6.5`, reinstalar la versión antigua de Unity y comprobar que el juego sigue funcionando con la versión anterior.

Las migraciones que se quedaron a medias y bloqueadas deben abandonarse de forma limpia y reiniciarse desde la etiqueta: nunca se debe distribuir código a medio migrar.

9. Solución de problemas

Síntoma	Causa / Solución
El Editor no se abre — "El proyecto se guardó en una versión más reciente"	Alguien lo abrió antes en 6000.5; revertir <code>ProjectSettings/ProjectVersion.txt</code> a la cadena de la versión anterior.
Tras la limpieza del manifest, error de compilación "Unity.Entities assembly not found"	El Editor es anterior a 6000.4, donde el Core Package no está disponible. Actualizar el Editor o volver a añadir el paquete UPM.
Cientos de advertencias de obsolescencia	Es lo esperado. Conviene tratarlas como la lista de tareas de la migración: cada una señala algo que corregir.
La compilación tiene éxito pero el juego falla al inicio	Suele ser un problema de ECB o de serialización introducido al eliminar un componente. Comparar el tamaño del asset <code>EntityScene</code> horneado; una reducción grande sugiere que un componente se eliminó sin avisar.
El rendimiento empeora tras la migración	Suele deberse a que se introdujo un <code>SystemBase</code> que no debería existir (un campo gestionado), o a que un <code>IJobEntity</code> dejó de usar <code>Burst</code> . Revisar el Profiler y el Burst Inspector.

Capítulo 33. Package Manager → Core Package

1. Qué cambia

En Unity 6000.4+, Entities (y sus paquetes DOTS complementarios) pasan a ser **Core Packages**: se instalan desde el propio Editor en lugar de descargarse de un registro remoto, y su versión la fija el Editor. **Se siguen añadiendo desde el Package Manager** y permanecen listados como dependencias en el proyecto: ser un Core Package cambia *de dónde viene el paquete y quién elige su versión*, no si el proyecto tiene que solicitarlo. Lo que cambia al migrar un proyecto 1.x:

- **Packages/manifest.json**: las entradas `com.unity.entities*` **se mantienen**, pero sus versiones **1.x** fijadas se reemplazan por las versiones Core del Editor (volver a añadirlas desde el Package Manager).
- **Packages/packages-lock.json**: los paquetes afectados ahora se resuelven con `"source": "builtin"`.
- **Ventana del Package Manager**: los paquetes siguen figurando en "In Project"; ya no se puede elegir su versión (sigue a la del Editor).

El código C# **no** cambia. Los nombres de ensamblado (`Unity.Entities`, `Unity.Collections`, etc.) siguen siendo los mismos y las referencias de los `.asmdef` no se tocan.

2. Qué paquetes se trasladan

Pasaron a ser **Core Package** en 6000.4+:

ID de UPM (antes)	Después
<code>com.unity.entities</code>	Core Package, se distribuye con el Editor
<code>com.unity.collections</code>	Core Package
<code>com.unity.mathematics</code>	Core Package
<code>com.unity.entities.graphics</code>	Core Package
<code>com.unity.netcode</code> (Netcode for Entities)	Core Package en 6.5

Permanecen **en el Package Manager**:

- `com.unity.physics` (Unity Physics)
- `com.unity.charactercontroller` (Character Controller)

Integrados en el motor (sin cambios):

- Jobs system
- Burst (`com.unity.burst` aún figura como entrada UPM en algunas configuraciones, pero en 6000.4+ está ligado a la versión del Editor; lo más seguro es dejar el manifest tal cual).

3. Edición de `manifest.json`

Abrir `Packages/manifest.json`. Tendrá un aspecto parecido a este:

```
{
  "dependencies": {
    "com.unity.entities": "1.4.2",
    "com.unity.collections": "2.4.5",
    "com.unity.mathematics": "1.3.2",
    "com.unity.entities.graphics": "1.4.2",
    "com.unity.netcode": "1.12.0",
    "com.unity.physics": "1.3.14",
    "com.unity.render-pipelines.universal": "17.0.4",
    ...
  }
}
```

En un Editor 6000.4+, esas versiones **1.x** fijadas ya no se resuelven: el registro deja de servir las para este Editor. **No** basta con eliminar las líneas (eso quita la dependencia, y un Core Package *no* se vuelve a añadir automáticamente). En su lugar, apuntar cada Core Package a la versión del Editor. Hay dos formas, y ambas mantienen los paquetes como dependencias:

Recomendado — volver a añadir desde el Package Manager. Para cada Core Package, quitar su versión 1.x obsoleta y volver a añadirlo desde **Window** → **Package Manager** → **Unity Registry** → **Install** (o + → **Add package by name...**, p. ej. `com.unity.entities`). El Package Manager escribe por ti la versión Core del Editor en `manifest.json` e incorpora automáticamente los Core Packages dependientes.

A mano. Reemplazar la versión 1.x de cada Core Package por la versión Core del Editor (por ejemplo 6.5.0 en Unity 6000.5):

```
{
  "dependencies": {
    - "com.unity.entities": "1.4.2",
    + "com.unity.entities": "6.5.0",
    - "com.unity.entities.graphics": "1.4.2",
```

```
+ "com.unity.entities.graphics": "6.5.0",
- "com.unity.netcode": "1.12.0",
+ "com.unity.netcode": "6.5.0",
  "com.unity.physics": "1.3.14",
  "com.unity.render-pipelines.universal": "17.0.4",
  ...
}
```

`com.unity.collections` y `com.unity.mathematics` entran como dependencias de `Entities`, así que normalmente no hace falta listarlos a mano. Dejar `com.unity.physics` y cualquier otro paquete que siga siendo de registro (UPM) tal cual.

Guardar el archivo.

4. Reconstrucción de `packages-lock.json`

Dos opciones:

4.1 Dejar que el Editor lo regenere

Cerrar el Editor, borrar `Packages/packages-lock.json` y volver a abrirlo. Unity lo regenera. Es la opción más limpia.

4.2 Editar manualmente

Abrir `packages-lock.json` y actualizar las entradas obsoletas de los paquetes anteriores (eliminarlas y dejar que el Editor las vuelva a resolver). El Editor lo reconcilia en la siguiente apertura. Es algo más rápido con archivos lock complejos, pero más propenso a errores.

Las dos opciones dan como resultado un archivo lock final con entradas como esta:

```
"com.unity.entities": {
  "version": "6.5.0",
  "depth": 0,
  "source": "builtin",
  "dependencies": {}
}
```

El `"source": "builtin"` es la señal de que el paquete ya es un Core Package (suministrado por el Editor), mientras que `"depth": 0` muestra que sigue siendo una **dependencia directa** declarada en `manifest.json`: la entrada no desapareció.

5. Verificación del resultado

1. **Ventana del Package Manager** (Window → Package Manager → "In Project") — Entities, Collections, Mathematics, Entities Graphics (y Netcode for Entities si se usa) deben aparecer en la lista, con su versión fijada por el Editor (sin desplegable de versión).
2. **Menús del Editor** — Window → Entities → Hierarchy/Systems/Archetypes existen y funcionan.
3. **Compilación** — pulsar Play; el proyecto debe entrar en modo Play sin errores de compilación por ensamblados que falten.

6. Solución de problemas

Síntoma	Causa / Solución
Unable to find package 'com.unity.entities@1.x.y' tras actualizar el Editor	El manifest aún fija una versión 1.x. Actualizarla a la versión Core (volver a añadirla desde el Package Manager), luego borrar packages-lock.json y dejar que el Editor lo regenere.
The type or namespace Unity.Entities could not be found	Entities no está añadido al proyecto (añadirlo desde el Package Manager) o el Editor es anterior a 6000.4 (actualizarlo).
El Package Manager muestra una versión de Entities obsoleta o en conflicto	Aún queda una versión 1.x fijada en manifest.json. Actualizarla a la versión Core (volver a añadirla desde el Package Manager) y regenerar packages-lock.json.
Un asset de terceros no compila porque fija com.unity.entities@1.x	El asset necesita actualizarse. Esperar, hacer un fork o quedarse en la línea 1.x para este proyecto.
Unity Physics falla porque exigía una versión concreta de Entities	Actualizar Unity Physics a una versión compatible con Entities 6.5. Consultar el changelog del paquete en docs.unity3d.com.
La regeneración de packages-lock.json falla	Un problema de red o un registro privado fijado. Revisar los logs del Editor para ver el fallo de resolución concreto.
Netcode for Entities sigue resolviéndose a una versión 1.x antigua	En 6000.5+, Netcode es un Core Package 6.5. Actualizar com.unity.netcode a la versión 6.5 (volver a añadirlo desde el Package Manager) y regenerar.

Capítulo 34. Referencias a objetos administrados → UnityObjectRef

1. Descripción general

Al migrar código ECS antiguo, un problema habitual es que las referencias a objetos gestionados se cuelen en los datos de componente de tiempo de ejecución. En Entities 6.5 conviene mantener no gestionados los componentes de gameplay siempre que se pueda y usar `UnityObjectRef<T>` cuando los datos ECS deban apuntar a un objeto de Unity como una malla, un material, una textura, un clip de audio o un `ScriptableObject` de authoring.

Esta página se ciñe al diseño de componentes DOTS. No cubre las APIs de identificación de objetos de Unity que abarcan todo el motor.

2. Qué buscar

Buscar los componentes que guardan objetos Unity gestionados de forma directa:

Patrón	Por qué revisarlo
Componentes <code>class</code> que implementan <code>IComponentData</code>	Los componentes gestionados están permitidos, pero son más lentos y quedan restringidos al hilo principal.
Campos <code>UnityEngine.Object</code> dentro de los datos de un componente	Suele indicar un componente gestionado o un diseño de componente no gestionado inválido.
Campos <code>GameObject</code> , <code>Transform</code> , <code>Material</code> , <code>Mesh</code> , <code>Texture</code> , <code>AudioClip</code>	Decidir si su sitio está en el authoring, en la presentación o en un <code>UnityObjectRef<T></code> .
Systems de tiempo de ejecución que llaman a <code>Resources.Load</code> por entidad	Llevar la resolución de assets al baking o a un sistema de contenido centralizado.

3. Antes y después

Antes — componente gestionado

```
using Unity.Entities;
using UnityEngine;
```

```
public class ProjectileVisual : IComponentData
{
    public Mesh Mesh;
    public Material Material;
}
```

Esto funciona, pero convierte el componente en gestionado. Los componentes gestionados restan flexibilidad a Burst y a los jobs, y por lo general no encajan bien con los datos de gameplay por entidad.

Después — componente no gestionado con `UnityObjectRef<T>`

```
using Unity.Entities;
using UnityEngine;

public struct ProjectileVisual : IComponentData
{
    public UnityObjectRef<Mesh> Mesh;
    public UnityObjectRef<Material> Material;
}
```

Hornear las referencias una sola vez:

```
using Unity.Entities;
using UnityEngine;

public class ProjectileVisualAuthoring : MonoBehaviour
{
    public Mesh Mesh;
    public Material Material;
}

public class ProjectileVisualBaker : Baker<ProjectileVisualAuthoring>
{
    public override void Bake(ProjectileVisualAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new ProjectileVisual
        {
            Mesh = new UnityObjectRef<Mesh>(authoring.Mesh),
            Material = new UnityObjectRef<Material>(authoring.Material)
        });
    }
}
```

4. Patrón de acceso en tiempo de ejecución

Mantener los jobs de simulación puramente orientados a datos. Desreferenciar objetos de Unity solo allí donde se permiten las APIs gestionadas de Unity.

```

using Unity.Entities;
using UnityEngine;

public partial class ProjectileVisualUploadSystem : SystemBase
{
    protected override void OnUpdate()
    {
        foreach (var visual in SystemAPI.Query<RefRO<ProjectileVisual>>())
        {
            Mesh mesh = visual.ValueRO.Mesh.Value;
            Material material = visual.ValueRO.Material.Value;

            if (mesh == null || material == null)
                continue;

            // Upload to a rendering registry, material property block cache, etc.
        }
    }
}

```

Para renderizado a gran escala, conviene evitar la desreferencia por entidad en cada fotograma. Cachear los objetos ya resueltos en un registro del lado de presentación, indexado por un valor ECS compacto.

5. Cuándo no usar UnityObjectRef<T>

Caso	Mejor alternativa
Datos de gameplay puros	Guardar datos primitivos, matemáticos o de blob directamente en un IComponentData.
Tablas grandes de solo lectura	BlobAssetReference<T>.
Un prefab de entidad horneado en la misma SubScene	Una referencia de prefab de tipo Entity.
Un prefab que se carga mediante flujos de contenido	EntityPrefabReference.
Comportamiento gestionado complejo por entidad	Mantenerlo fuera de la ruta de simulación o aislarlo en un system de presentación gestionado.

6. Lista de verificación de migración

- ☐ Localizar datos de componente gestionados que solo guardan referencias a objetos de Unity.
 - ☐ Sustituir esos campos por `UnityEngine.ObjectReference<T>` donde proceda un componente no gestionado.
 - ☐ Llevar la asignación de objetos a los Bakers.
 - ☐ Mantener la desreferencia de `.Value` fuera de los Burst jobs.
 - ☐ Cachear los objetos ya desreferenciados cuando muchas entidades comparten el mismo asset.
 - ☐ Usar `BlobAssetReference<T>` para datos de gameplay inmutables de gran tamaño, en vez de leer `ScriptableObjects` en tiempo de ejecución.
-

7. Solución de problemas

Síntoma	Causa / Solución
El componente no se puede usar en un Burst job	Sigue siendo gestionado. Convertir los campos de objetos de Unity a <code>UnityEngine.ObjectReference<T></code> o separar los datos de presentación.
<code>.Value</code> es null	El asset referenciado no está cargado o se destruyó. Revisar la referencia de authoring y la ruta de carga de contenido.
El system reserva memoria en cada fotograma	Se están resolviendo o cargando assets por entidad. Horneear las referencias y cachear los objetos gestionados ya resueltos.
Un system de gameplay depende de un <code>ScriptableObject</code> en tiempo de ejecución	Horneear los valores necesarios en datos de componente o en un blob asset.
La build descarta un asset	Comprobar que el asset se referencia mediante flujos de baking o de contenido, y no solo mediante una ruta de string en tiempo de ejecución.

Capítulo 35. Migración de foreach → IJobEntity

1. Estado — obsoleto, aún no eliminado

Las antiguas APIs de iteración de componentes están **obsoletas** en Entities 1.4+ / 6.5, pero aún compilan:

Legacy	Estado en 6.5	Reemplazo
Entities.ForEach	Obsoleto desde 1.4; advertencias de obsolescencia en cada uso; el changelog de Unity indica que se eliminará en una versión mayor futura	SystemAPI.Query<...> (hilo principal) o IJobEntity (jobs)
Job.WithCode	Obsoleto junto con ForEach	Struct IJob simple, o inline en OnUpdate
IJobForEach (anterior a 1.0)	Eliminado mucho antes de 1.4	IJobEntity

Se llega aquí porque se abrió un proyecto 1.x en 6000.5+ y el compilador emite muchas advertencias, o porque se quieren eliminar estas advertencias antes de que una versión mayor futura elimine la API. La refactorización es mecánica una vez que se identifica el patrón.

2. Las dos APIs de reemplazo

- **SystemAPI.Query<T, U, ...>** — iteración en el hilo principal dentro de OnUpdate. Simple, sin sobrecarga de jobs, ideal para cargas de trabajo pequeñas o cuando se necesitan APIs managed.
- **IJobEntity** — job paralelo generado por código. Reemplazo directo si se está listo para el trabajo multi-hilo, que es donde DOTS marca la diferencia.

Referencia: ../DOTS Workflows/11_IJobEntity · SystemAPI.Query.md.

3. Migración — Entities.ForEach → SystemAPI.Query

3.1 Antes

```

public partial class HealthRegenSystem : SystemBase
{
    protected override void OnUpdate()
    {
        float dt = Time.DeltaTime;

        Entities
            .WithAll<Alive>()
            .WithNone<Dead>()
            .ForEach((ref Health health, in Regen regen) =>
            {
                health.Value = math.min(health.Value + regen.Rate * dt, health.Max);
            })
            .ScheduleParallel();
    }
}

```

3.2 Después — SystemAPI.Query (hilo principal)

```

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (health, regen) in
            SystemAPI.Query<RefRW<Health>, RefRO<Regen>>()
                .WithAll<Alive>()
                .WithNone<Dead>())
        {
            health.ValueRW.Value = math.min(
                health.ValueRO.Value + regen.ValueRO.Rate * dt,
                health.ValueRO.Max);
        }
    }
}

```

Notas sobre los cambios de forma:

- `SystemBase` → `ISystem` (más económico, compatible con Burst). Puede ser necesario mantener `SystemBase` si el código antiguo capturaba campos `managed`.
- `ref Health` → elemento de tupla `RefRW<Health>`. Lectura mediante `.ValueRO`, escritura mediante `.ValueRW`.
- `in Regen` → `RefRO<Regen>`.
- `Time.DeltaTime` → `SystemAPI.Time.DeltaTime`.
- Los filtros (`WithAll/WithNone`) se pasan al constructor de `Query`.

4. Migración — Entities.ForEach.ScheduleParallel → IJobEntity

Cuando el ForEach se planificaba en paralelo, la forma de job es una migración directa 1:1 que mantiene el paralelismo.

4.1 Después — IJobEntity

```
[BurstCompile]
public partial struct HealthRegenJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Regen regen)
    {
        health.Value = math.min(health.Value + regen.Rate * DeltaTime, health.Max);
    }
}

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new HealthRegenJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```

Los filtros se aplican al struct del job como atributos:

```
[WithAll(typeof(Alive))]
[WithNone(typeof(Dead))]
public partial struct HealthRegenJob : IJobEntity { ... }
```

O se pasa una query en el momento de planificación:

```
var q = SystemAPI.QueryBuilder()
    .WithAll<Health, Regen, Alive>()
    .WithNone<Dead>()
    .Build();
new HealthRegenJob { DeltaTime = dt }.ScheduleParallel(q);
```

5. Capturas y sus reemplazos

El legacy Entities.ForEach permitía capturar variables locales con .WithReadOnly, .WithDisposeOnCompletion, etc. En IJobEntity, las capturas son **campos del struct del job** — Burst sabe exactamente qué se está pasando.

```
// BEFORE
var buffer = new NativeArray<int>(count, Allocator.TempJob);
Entities.ForEach((ref Foo f) => { /* use buffer */ })
    .WithDisposeOnCompletion(buffer)
    .ScheduleParallel();

// AFTER
[BurstCompile]
partial struct FooJob : IJobEntity
{
    [ReadOnly] public NativeArray<int> Buffer;

    void Execute(ref Foo f) { /* use Buffer */ }
}

var buffer = new NativeArray<int>(count, Allocator.TempJob);
state.Dependency = new FooJob { Buffer = buffer }.ScheduleParallel(state.Dependency);
buffer.Dispose(state.Dependency); // chained dispose replaces WithDisposeOnCompletion
```

Reemplazos de atributos:

- `.WithReadOnly(x)` → `[ReadOnly] public NativeArray<T> X;`
- `.WithDisposeOnCompletion(x)` → `x.Dispose(handle)` después de planificar.
- `.WithoutBurst()` → `[WithoutBurst]` en el struct del job (pero considerar seriamente eliminar la razón por la que fue necesario).
- `.WithEntityAccess()` → añadir el parámetro `Entity entity` a `Execute`.

6. Cuándo sigue siendo necesario un `SystemBase`

Si el sistema original tenía un campo managed genuino (Input System, UI Toolkit, servicio externo), no es posible convertirlo mecánicamente a `ISystem`. Dos opciones:

1. **Mantenerlo como `SystemBase`** y usar `SystemAPI.Query` dentro de `OnUpdate`. Un solo hilo.
2. **Dividirlo** en un `SystemBase` que haga de puente con el mundo managed y un `ISystem` que realice el trabajo Burst — ver `../DOTS Workflows/08_System - ISystem vs SystemBase.md`.

7. Validación

Después de migrar un sistema:

1. **Probar el comportamiento con pruebas unitarias.** La migración no debe cambiar la lógica; si la salida difiere, la refactorización introdujo un error.
2. **Analizar el rendimiento.** El `ForEach` paralelo y `IJobEntity.ScheduleParallel` deben estar dentro del 5% de diferencia para trabajo equivalente. Una regresión grande significa que Burst no está actuando — inspeccionar el Burst Inspector.
3. **Eliminar el archivo del sistema antiguo.** Resistir la tentación de mantener un fallback `#if LEGACY` — se deteriora.

8. Solución de problemas

Síntoma	Causa / Solución
<code>warning: Entities.ForEach is obsolete</code>	Esperado en 6.5 — el código aún compila. Migrar como se indica para eliminar la advertencia antes de que una versión mayor futura elimine la API.
<code>error: 'SystemAPI' does not contain 'Query'</code>	El sistema no es <code>partial</code> , o el struct no implementa <code>ISystem</code> correctamente.
<code>IJobEntity</code> compila pero no se ejecuta	Falta <code>[BurstCompile]</code> en el struct y en <code>OnUpdate</code> ; el generador de código no planifica tipos no parciales.
El job lee valores desactualizados	Se capturó una variable local antes de llamar a <code>.Schedule*</code> . Los campos deben establecerse en el struct del job antes de planificar.
El rendimiento regresa tras la migración	Revisar el fallback de Burst (Burst Inspector), o que un tipo managed se coló en un campo.
Solo hay problemas en tiempo de compilación tras la migración	Verificar que se eliminaron las variables capturadas mediante atributos y no mediante la sintaxis de cierre de lambda, que solo se aplicaba a la API de <code>ForEach</code> .

Capítulo 36. Deprecación de IAspect — migrar fuera de los Aspects

1. Estado — obsoleto, aún no eliminado

IAspect fue marcado como **obsoleto en Entities 1.4** y sigue siendo obsoleto en la línea 6.x. Aún compila, aún se ejecuta y aún está listado como un tipo de parámetro válido de `IJobEntity.Execute` en la referencia de la API 6.5 — pero el compilador emite una advertencia de obsolescencia, y Unity ha indicado que el tipo se **eliminará en una versión mayor futura**. El nuevo código no debe usarlo; el código existente debe migrarlo a un ritmo progresivo.

IAspect se introdujo para permitir que los sistemas declararan una "vista" sobre un conjunto de componentes y la usaran como un único argumento en `Entities.ForEach` o `IJobEntity`. En la práctica, añadió una capa de generación de código y un sistema de tipos paralelo que no se justificó:

- Escribir un aspect duplicaba la lista de parámetros de componentes.
- Los depuradores avanzaban por el código wrapper generado.
- `RefRW<T>` / `RefRO<T>` en `IJobEntity` y `SystemAPI.Query` logran la misma ergonomía de "pasar componentes como un conjunto" sin el tipo extra.

Dado que la superficie de API aún está presente, el código no se rompe al actualizar el Editor; esta migración consiste en limpiar advertencias y prepararse para la versión 2.0.

2. Refactorización mecánica

2.1 Punto de partida

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

public readonly partial struct MovementAspect : IAspect
{
    public readonly RefRW<LocalTransform> Transform;
    public readonly RefRO<Velocity> Velocity;
    public readonly RefRO<Speed> Speed;

    public void Step(float dt)
    {
```

```

        Transform.ValueRW.Position += Velocity.ValueRO.Direction * Speed.ValueRO.Value * dt;
    }
}

public partial struct MovementSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;
        foreach (var m in SystemAPI.Query<MovementAspect>())
            m.Step(dt);
    }
}

```

2.2 Después — parámetros de componentes directos

```

using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

public partial struct MovementSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;
        foreach (var (transform, velocity, speed) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<Velocity>, RefRO<Speed>>())
        {
            transform.ValueRW.Position += velocity.ValueRO.Direction * speed.ValueRO.Value * dt;
        }
    }
}

```

Dos cambios:

1. El tipo `aspect` desaparece. Sus campos se convierten en los elementos de la tupla de `SystemAPI.Query`.
2. El método que estaba en el `aspect` (`Step`) se incorpora en línea. Si es demasiado grande para incorporarlo, se puede extraer como un **helper estático** que tome los mismos parámetros — esto lo mantiene compatible con Burst:

```

static void Step(ref LocalTransform t, in Velocity v, in Speed s, float dt)
{
    t.Position += v.Direction * s.Value * dt;
}

```

Para llamarlo:

```

Step(ref transform.ValueRW, velocity.ValueRO, speed.ValueRO, dt);

```

3. Aspects en IJobEntity

3.1 Antes

```
[BurstCompile]
public partial struct MoveJob : IJobEntity
{
    public float DeltaTime;
    void Execute(MovementAspect aspect) => aspect.Step(DeltaTime);
}
```

3.2 Después — componentes como parámetros de Execute

```
[BurstCompile]
public partial struct MoveJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref LocalTransform transform, in Velocity velocity, in Speed speed)
    {
        transform.Position += velocity.Direction * speed.Value * DeltaTime;
    }
}
```

Nótese que IJobEntity usa parámetros simples ref / in, no RefRW<T> / RefRO<T>. (El generador envuelve los componentes en accesos a arrays de chunk por debajo.)

4. Aspects que envolvían un DynamicBuffer

```
// BEFORE
public readonly partial struct PathAspect : IAspect
{
    public readonly DynamicBuffer<Waypoint> Path;
    public void Advance(int n) => Path.RemoveRange(0, n);
}

// AFTER — access buffer directly
foreach (var (path, e) in SystemAPI.Query<DynamicBuffer<Waypoint>>().WithEntityAccess())
{
    path.RemoveRange(0, 1);
}
```

En los jobs, DynamicBuffer<T> es un parámetro válido de Execute — no se necesita nada adicional.

5. Aspects entre entidades

Ocasionalmente un aspect envolvía un `ComponentLookup<T>` para "necesito ver el componente de otra entidad". Reemplazo:

```
// BEFORE
public readonly partial struct AITargetAspect : IAspect
{
    public readonly RefRO<Target> Target;
    [ReadOnly] public readonly ComponentLookup<Health> HealthLookup;
    public float TargetHealth =>
        HealthLookup.TryGetComponent(Target.ValueRO.Entity, out var h) ? h.Value : 0f;
}

// AFTER – declare the lookup on the system / job directly
public partial struct AISystem : ISystem
{
    private ComponentLookup<Health> _healthLookup;

    public void OnCreate(ref SystemState state)
    {
        _healthLookup = state.GetComponentLookup<Health>(true);
    }

    public void OnUpdate(ref SystemState state)
    {
        _healthLookup.Update(ref state);

        foreach (var (target, e) in SystemAPI
            .Query<RefRO<Target>>()
            .WithEntityAccess())
        {
            float hp = _healthLookup.TryGetComponent(target.ValueRO.Entity, out var h)
                ? h.Value : 0f;
            // ...
        }
    }
}
```

Este patrón también es más ligero que la versión con aspect porque la búsqueda se comparte entre entidades.

6. Cuando un aspect era complejo

Si el aspect tenía muchos campos, encapsulaba estado o participaba en herencia no trivial — eso es señal de considerar una refactorización genuina, no solo una migración. A menudo el código queda mejor como dos sistemas que pasan datos mediante componentes, en lugar de un sistema con un aspect dios.

7. Solución de problemas

Síntoma	Causa / Solución
Advertencia de compilación <code>IAspect is obsolete</code>	Esperado en 6.5. Migrar como se indica para eliminar la advertencia; el código sigue ejecutándose hasta la eliminación en una versión mayor futura.
<code>type or namespace IAspect not found</code>	Se está en una versión donde <code>IAspect</code> ya fue eliminado. Completar la migración — no hay fallback.
La tupla de Query tiene demasiados tipos y no compila	<code>SystemAPI.Query</code> admite una tupla amplia; dividir una query muy ancha en dos queries o pasar a <code>IJobChunk</code> suele ser más legible.
El job ya no compila con Burst	Un método del aspect pudo haber capturado un tipo que Burst no admitía. Revisar directamente los parámetros de <code>Execute</code> .
El aspect entre entidades migrado de forma naive es lento	<code>ComponentLookup<T>.Update(ref state)</code> debe llamarse en <code>OnUpdate</code> antes de usar; olvidarlo hace que las búsquedas sean inválidas.
El aspect de <code>DynamicBuffer</code> no se puede migrar	<code>DynamicBuffer<T></code> es válido directamente como tipo de <code>SystemAPI.Query</code> y parámetro de <code>IJobEntity.Execute</code> — no se necesita ningún wrapper.
Las pruebas que comprobaban el tipo aspect fallan	Eliminar o reescribir esas pruebas con la nueva firma; no hay nada que conservar.

Apéndice. Registros de cambios

Apéndice A. Cambios clave de Entities 1.4 → 6.5

Resumen acotado de las entradas del changelog oficial del paquete `com.unity.entities` que importan al mover un proyecto ECS de la línea 1.4 a la línea del Core Package 6.5. Esta página cubre intencionadamente solo los cambios del paquete Entities.

1. Ramas de versiones

Entities 6.x se alinea con el Editor de Unity 6000.x y se distribuye como Core Package. Las versiones anteriores de Entities 1.x eran paquetes del Package Manager.

Rama	Distribución	Ejemplo
Entities 1.x	Package Manager (<code>com.unity.entities</code>)	1.4.2 en el historial de changelog oficial de 6.5.
Entities 6.x	Core Package integrado en el Editor	6.4.0 / 6.5.0.

Esta distinción es sobre la distribución e identificación de versiones de paquetes. No es una licencia para incluir migraciones no relacionadas de UnityEngine o de ámbito general del Editor en la documentación de programación DOTS.

2. Entities 6.5.0

La entrada del changelog oficial del paquete para Entities 6.5.0 es:

"Placeholder for 6.5.0"

Se debe tratar 6.5.0 como la versión del paquete alineada con Unity 6000.5. El changelog del paquete no lista nueva superficie de API de Entities para esta versión.

3. Entities 6.4.0

La entrada del changelog oficial del paquete para Entities 6.4.0 indica:

"Package is now a core package embedded in Unity."

Efectos prácticos:

Área	Cambio
Instalación	En Unity 6000.4+ se sigue añadiendo <code>com.unity.entities</code> desde el Package Manager, pero se instala desde el propio paquete del Editor: no hay versión que elegir.
Elección de versión	La versión de Entities proviene de la instalación del Editor.
Archivo lock	<code>packages-lock.json</code> marca el paquete como built-in/core en lugar de dependencia de registro.
Flujo de actualización	Reemplazar cualquier versión 1.x fijada de <code>com.unity.entities</code> (volver a añadirlo desde el Package Manager) y dejar que el Editor la resuelva a la versión Core.

Ver `../Migration/02_Package Manager → Core Package.md`.

4. Entities 1.4.2

El historial del changelog oficial incluido con `com.unity.entities@6.5` lista 1.4.2 antes de la transición al Core Package.

Cambios

Cambio	Por qué importa
Dependencia de Burst actualizada a 1.8.24	Actualización de dependencia para proyectos aún en la línea 1.4.
Clases USS cambiadas de porcentaje a tamaño en píxeles	Detalle de estilización de UI del Editor.
Campo <code>font-size</code> cambiado de porcentaje a tamaño en píxeles	Detalle de estilización de UI del Editor.

Correcciones

Corrección	Por qué importa
Generación de código incorrecta para <code>public partial SystemBase</code>	Relevante si el código antiguo de 1.4 tuvo errores de generación de código.

Corrección	Por qué importa
SGICE002 con SystemAPI dentro de Entities.ForEach más WithDeferredPlaybackSystem	Contexto útil durante la migración fuera de Entities.ForEach.
Fuga de memoria en la serialización del world	Estabilidad en tiempo de ejecución/editor.
Inicialización del Inspector al salir del modo Play con una entidad seleccionada	Estabilidad de las herramientas del Editor.

5. Entradas de pre-lanzamiento de 1.4.0 que justifican la migración

El historial de preview/pre-release de 1.4.0 es donde aparece la dirección importante de la API ECS.

Área	Cambio oficial	Objetivo de migración
Iteración	Entities.ForEach marcado como obsoleto; eliminación mayor futura prevista.	Usar IJobEntity o SystemAPI.Query.
Aspects	IAAspect marcado como obsoleto; eliminación mayor futura prevista.	Usar APIs directas de componente/query.
Referencias opcionales de componente	ComponentLookup.GetRefRWOptional() / GetRefROOptional() obsoletos.	Usar TryGetRefRW<T>() / TryGetRefRO<T>().
Buffers de chunk	ArchetypeChunk.GetBufferAccessorR0<T>() / GetBufferAccessorRW<T>() añadidos.	Solicitar el modo de acceso que se necesita realmente.
Buffers sin tipo	GetUntypedBufferAccessorReinterpret<T>() añadido.	Usar solo cuando los tamaños de elemento origen/destino y las capacidades del buffer sean aliasables de forma segura.
Handles de sistema	SystemTypeIndex puede obtenerse desde SystemHandle.	Mejor acceso al sistema de bajo nivel.
Herramientas de query	Columnas Disabled, Present, Absent, None; pestaña Dependency; distinción de Prefab en la ventana Query.	Mejor inspección/depuración de queries.
Referencias de objeto en ECS	Documentación manual para UnityObjectRef añadida.	Usar UnityObjectRef<T> para referencias a objetos Unity managed en datos ECS no administrados.

6. Lista de verificación de migración

- ☐ En Unity 6000.4+, volver a añadir los Core Packages de DOTS desde el Package Manager, reemplazando cualquier versión 1.x fijada en `manifest.json`.
 - ☐ Reemplazar los `Entities.ForEach` restantes con `IJobEntity` o `SystemAPI.Query`.
 - ☐ Reemplazar el uso de `IAAspect` con acceso directo a componentes.
 - ☐ Reemplazar `GetRefRWOptional` / `GetRefROOptional` con `TryGetRefRW<T>()` / `TryGetRefRO<T>()`.
 - ☐ Revisar las referencias a objetos Unity managed y usar `UnityObjectRef<T>` o datos baked donde corresponda.
 - ☐ Volver a comprobar las ventanas Systems, Query y Entity Inspector después de la migración.
-

7. Referencias

- [Entities 6.5 Manual](#)
- [Entities 6.5 Changelog](#)

Apéndice B. Cambios clave de Netcode for Entities 1.4 → 6.5

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

Una guía de migración para los cambios de Netcode for Entities acumulados desde la línea 1.4 hasta la línea del Core Package 6.5. Netcode 6.5.0 pasó al Editor de Unity 6000.5 como paquete integrado, por lo que las entradas posteriores se orientan hacia las notas de versión del Editor en lugar de este changelog del paquete.

1. Aspectos destacados por versión

6.5.0 (2026-02-05) — paquete integrado en el Editor

Área	Cambio
Distribución	Netcode pasó a ser un paquete integrado en el Editor a partir de Unity 6000.5.
Dependencia de Entities	La línea del changelog lista <code>com.unity.entities 1.4.2</code> . Mantener esto distinto del objetivo Unity 6000.5 / Entities Core Package del manual DOTS.
Profiler	Las etiquetas de TreeView de Ghost Snapshot View obtuvieron menús contextuales para inspección rápida de Prefab/componente.
Source Generator	La ejecución del Source Generator en el lado del IDE se deshabilitó para Rider y Visual Studio, mejorando el rendimiento del IDE.
Breaking	El ECB de desconexión cambió de <code>BeginSimulationECBS</code> a <code>NetworkGroupCommandBufferSystem</code> .

1.12.0 (2026-02-01)

Área	Cambio
Profiler	Navegación basada en ticks, campos de búsqueda y visualización del tick de Prediction/Interpolation del cliente.
Dependencia de Entities	Actualizada de 1.4.2 a 1.4.4 .
Obsoleto	Network Debugger basado en navegador obsoleto en favor del Netcode Profiler.
Eliminado	<code>NETCODE_PROFILER_ENABLED</code> ya no es necesario.

Área	Cambio
Breaking	Comportamiento de TransformUsageFlags de GhostAuthoringComponentBaker cambió.

1.11.0 (2025-12-12)

Área	Cambio
Menú Prefab List	Inspección rápida de Ghost Prefab.
Ghost de GameObject	Se inició el trabajo de compatibilidad con Ghost a nivel de GameObject.
Optimización de Ghost estático	El tiempo del job de Ghost estático se redujo sustancialmente para datos sin cambios.
Migración de Ghost	IncludeInMigration admite la migración de datos no Ghost.
Serialización de bool	Se mejoraron los costes de serialización de bool empaquetada/desempaquetada.
Breaking de API	PrefabDebugName.Name completamente obsoleto.

1.10.0 (2025-11-09)

Área	Cambio
Componentes de tamaño cero	El guardado del estado del world admite componentes de tamaño cero.
Source Generator	Ejecución en el lado del IDE deshabilitada; reflejado en el comportamiento de 6.5.0.
Correcciones	Assert de PredictedGhostSpawnSystem, spam de advertencias en modo Sleep, excepción del bucle de predicción AlwaysRun, y correcciones relacionadas.

1.9.x (2025-09 a 2025-10)

Área	Cambio
Documentación de optimización de Ghost	Se incorporó documentación específica sobre la optimización de Ghost.
Estabilización	Se estabilizaron los módulos de Profiler y el Importance Visualizer de la 1.8.0.

1.8.0 (2025-08-17)

Área	Cambio
Importance Visualizer	Se incorporó a la herramienta de PlayMode.
Forced Input Latency	Se añadió ClientTickRate.ForcedInputLatencyTicks.

Área	Cambio
Profiler Modules	Módulos experimentales de Server / Client Profiler para Unity 6+.
Breaking	Las funciones de escala de GhostDistanceImportance ya no multiplican baseImportance por 1000.
Transport	Se actualizó com.unity.transport a la 2.5.3 .

1.7.0 (2025-07-29)

Área	Cambio
Host Migration	Se eliminó la define ENABLE_HOST_MIGRATION; Host Migration queda activo de forma predeterminada.
API de Host Migration	Se refactorizaron los nombres de clases y métodos; conviene consultar la documentación y los ejemplos vigentes del paquete para conocer los nombres de tipo exactos.
Serialización	Se mejoró la serialización de componentes Ghost para Host Migration.
Transport	Se actualizó com.unity.transport a la 2.1.0 .

1.6.1 a 1.6.2 (2025-05 a 2025-07)

Área	Cambio
ECB predicho	Se añadieron sistemas ECB de inicio y fin para PredictedSimulationSystemGroup.
Spawning predicho	Se añadió PredictedSpawningSystemGroup tras EndPredictedSimulationCommandBufferSystem.
Host Migration	Función experimental, oculta tras ENABLE_HOST_MIGRATION.
Seguimiento de reconexión	Se añadió el componente NetworkStreamIsReconnected.
Historial de snapshots	GhostSystemConstants.SnapshotHistorySize se puede configurar mediante una define de compilación.
Relevancy + Importance	Ruta rápida para combinar Ghost Relevancy e Importance Scaling.

1.5.0 (2025-04-22)

Área	Cambio
Thin Clients	Se añadió AutomaticThinClientWorldsUtility para crear Thin Clients en tiempo de ejecución.
Serialización	Replicación de FixedList y de búferes fijos unsafe.
Baselines	Se añadió GhostAuthoringComponent.UseSingleBaseline para ahorrar CPU.

Área	Cambio
Serialización de enteros	Plantillas específicas para byte y short.
Lag Compensation	El búfer de historial de física pasó de 16 a 32 ticks.
Relay	La ruta del paquete Relay pasó a <code>com.unity.services.multiplayer v1.1.0</code> .

1.4.0 (2024-11-14)

Área	Cambio
Frecuencia de envío	<code>GhostAuthoringComponent.MaxSendRate</code> limita la frecuencia con que se envía un Ghost.
Iteración de chunks	<code>GhostSendSystemData.MaxIterateChunks</code> limita los chunks procesados por tick.
NetCodeConfig	Se añadieron varios <code>NetworkConfigParameters</code> de Unity Transport.
ACK de snapshots	La capacidad del historial de ACK pasó a ser configurable.
Correcciones	Fallos de ACK de snapshots más allá de los 256 ticks, jitter en la interpolación de física y otras correcciones afines.

2. Cambios breaking acumulados

Versión	Cambio breaking
6.5.0	El ECB de desconexión pasó de <code>BeginSimulationECBS</code> a <code>NetworkGroupCommandBufferSystem</code> .
1.12.0	Cambió el comportamiento de <code>TransformUsageFlags</code> en <code>GhostAuthoringComponentBaker</code> .
1.11.0	<code>PrefabDebugName.Name</code> quedó totalmente obsoleto.
1.8.0	Cambió el comportamiento del multiplicador de <code>GhostDistanceImportance</code> .
1.7.0	Se refactorizaron los nombres de clases y métodos de la API de Host Migration.
1.5.0	Se endureció la validación de comandos y cambió el comportamiento del timeout de handshake.
1.4.0	Cambió el alcance de <code>MaxSendChunks</code> .

3. Tendencias a recordar

1. **La depuración empieza por el Profiler** — el antiguo Network Debugger del navegador quedó obsoleto; conviene usar los módulos del Netcode Profiler.

2. **Host Migration pasó a ser predeterminado** — experimental en la 1.6.1 y activo por defecto desde la 1.7.0.
 3. **Mejóro el rendimiento del IDE** — los Source Generators de Netcode ya no se ejecutan dentro del análisis del IDE.
 4. **La serialización ganó eficiencia** — el empaquetado de bool, las plantillas de byte/short y la opción de single-baseline alivian la carga de CPU y de ancho de banda.
 5. **Transición al Core Package** — Netcode 6.5.0 se distribuye con Unity 6000.5; los cambios futuros tenderán a las notas de versión del Editor.
-

4. Documentación relacionada

- `../DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md`
 - `../DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md`
 - `../DOTS Workflows/22_Netcode Ghost Optimization · Importance · Relevancy.md`
 - `../DOTS Workflows/24_Netcode Profiler & Debugging.md`
-

5. Referencias

- [Netcode for Entities 6.5 Manual](#)
- [Netcode for Entities 6.5 Changelog](#)
- [Netcode for Entities 1.12 Changelog](#)

Referencia. Glosario

Glosario

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

Definiciones de una línea para cada término no evidente de este manual. Los enlaces cruzados llevan al documento donde se explica cada término en profundidad.

1. Sustantivos centrales — las cinco piezas que forman ECS

Término	Definición	Ver
Entity	Un ID opaco de 64 bits (<code>struct Entity</code> , espacio de nombres <code>Unity.Entities</code>). No guarda datos ni comportamiento: es solo una clave para localizar componentes.	DOTS Workflows/03_ECS Core Concepts.md
Component	Datos asociados a una entidad. Hay varios tipos; ver §2.	DOTS Workflows/05_Component Types.md
Archetype	El conjunto de tipos de componente que tiene una entidad. Las entidades con el mismo Archetype comparten almacenamiento.	DOTS Workflows/03_ECS Core Concepts.md
Chunk	Bloque de 16 KB con entidades que comparten un Archetype. Dentro del chunk, los componentes se almacenan por columnas (SoA).	Optimizations and Debugging/01_Chunk Layout & TypeManager.md
World	Contenedor de entidades y sistemas. El runtime arranca con un "Default World".	DOTS Workflows/03_ECS Core Concepts.md

2. Tipos de componente

Tipo	Qué es
IComponentData	Componente struct no administrado; es el tipo predeterminado.
IBufferElementData	Tipo de elemento de un <code>DynamicBuffer<T></code> asociado a una entidad.
ISharedComponentData	Componente con clave por valor: las entidades con el mismo valor comparten un bucket de chunk.

Tipo	Qué es
ICleanupComponentData	Componente que sobrevive a DestroyEntity para que un sistema pueda hacer limpieza antes de que la entidad desaparezca del todo.
Componente de etiqueta	Struct IComponentData vacío: ocupa cero bytes y sirve únicamente como filtro de query.
Componente de chunk	Datos por chunk: una instancia por chunk, no por entidad.
IEnableableComponent	Componente cuya presencia se alterna mediante una máscara de bits por chunk, sin cambio estructural .

Detalles: DOTS Workflows/05_Component Types.md · DOTS Workflows/06_Enableable Component.md.

3. Sistemas y jobs

Término	Qué es
ISystem	Interfaz de sistema moderna basada en struct. Compatible con Burst. Es la opción recomendada para sistemas nuevos.
SystemBase	Interfaz de sistema legacy basada en clase. Conviene usarla solo cuando un sistema necesite acceso administrado en el hilo principal.
SystemGroup	Contenedor ordenado de sistemas. Grupos principales: InitializationSystemGroup, SimulationSystemGroup, PresentationSystemGroup.
IJobEntity	Job que recorre las entidades que coinciden con una query. Se genera por código y se compila con Burst.
IJobChunk	Job que recorre chunks en lugar de entidades sueltas; útil para agregar datos por chunk.
SystemAPI	Conjunto de helpers estáticos disponibles dentro de los sistemas: Query<>(), GetComponentLookup<T>(), Time, GetSingleton<T>(), etc.

Detalles: DOTS Workflows/08_System – ISystem vs SystemBase.md · DOTS Workflows/11_IJobEntity · SystemAPI.Query.md · DOTS Workflows/12_IJobEntity vs IJobChunk.md.

4. Authoring → runtime

Término	Qué es
Baker	Convierte un GameObject de Authoring con su MonoBehaviour en componentes ECS al guardar o compilar el SubScene.
SubScene	Contenedor de Authoring: los GameObjects que contiene se bakean a entidades y se serializan en un asset EntityScene.
EntityScene	El asset bakeado que genera un SubScene; se carga en un World en tiempo de ejecución.

Detalles: [DOTS Workflows/01_Baker Pattern & SubScene.md](#).

5. Seguridad ante cambios estructurales

Término	Qué es
Cambio estructural	Agrega o elimina un componente, o crea o destruye una entidad. Como mueve entidades entre chunks, resulta costoso y no es seguro a mitad de una iteración.
EntityCommandBuffer (ECB)	Registra los cambios estructurales durante la iteración y los reproduce en un punto seguro.
EntityCommandBuffer.ParallelWriter	Variante de ECB segura para hilos, pensada para jobs paralelos; cada llamada recibe un sortKey que garantiza una reproducción determinista.
Entidad diferida	Entidad creada con <code>ecb.CreateEntity()</code> o <code>Instantiate()</code> que aún no existe: su índice provisional se resuelve al reproducir el ECB.
Sort key	Entero que se pasa a cada comando de un ECB paralelo; el valor predeterminado es <code>[ChunkIndexInQuery]</code> .

Detalles: [DOTS Workflows/13_Structural Change & Safety.md](#) · [DOTS Workflows/14_EntityCommandBuffer](#) · [Deferred Entity.md](#) · [DOTS Workflows/15_ParallelWriter](#) · [Deterministic Playback.md](#).

6. Tipos de referencia de entidad

Estos son los tipos de referencia de DOTS que se usan en los datos ECS y en los flujos de baking.

Tipo	Qué es
Entity	Handle de ECS (<code>Unity.Entities</code>). No es más que un ID dentro del world ECS.

Tipo	Qué es
EntityPrefabReference	Referencia a un asset de prefab de entidad bakeado, pensada para flujos de contenido y streaming (Unity.Entities.Serialization).
UnityObjectReference<T>	Referencia del lado ECS a un UnityEngine.Object: permite que las entidades apunten a assets administrados (materiales, mallas) sin boxing.

Detalles: DOTS Workflows/04_Entity References – Entity · EntityPrefabReference · UnityObjectRef.md.

7. Netcode for Entities

Término	Qué es	Ver
Netcode for Entities	La capa de netcode multijugador de Unity sobre DOTS, para juegos con autoridad de servidor y predicción de cliente.	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
Server World	World ECS autoritativo que ejecuta la simulación del servidor y envía los snapshots de Ghost.	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
Client World	World ECS que recibe snapshots, envía comandos, predice el gameplay local y presenta el estado interpolado.	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
Thin Client World	World de cliente simulado y ligero que sirve para probar la carga y el comportamiento de las conexiones.	DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md
NetworkStreamConnection	Componente de la entidad de conexión que guarda el handle de conexión del transport.	DOTS Workflows/17_Netcode Network Connection & Approval.md
NetworkId	ID de conexión que asigna el servidor una vez aprobada la conexión.	DOTS Workflows/17_Netcode Network Connection & Approval.md
NetworkStreamInGame	Componente que habilita el intercambio de datos de gameplay; sin él, snapshots y comandos no circulan.	DOTS Workflows/17_Netcode Network Connection & Approval.md
Ghost	Entidad en red con autoridad de servidor que se replica a los clientes mediante snapshots.	DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md

Término	Qué es	Ver
Snapshot	Estado serializado de un Ghost que el servidor envía al cliente, normalmente por un transport no confiable.	DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md
GhostField	Atributo que marca un campo de un componente para que se serialice como parte del Ghost.	DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md
PredictedGhost	Componente que identifica los Ghosts que participan en la predicción del cliente.	DOTS Workflows/19_Netcode Prediction & Rollback.md
Simulate	Etiqueta habilitable que Netcode usa para marcar qué entidades predichas deben ejecutarse en el tick de predicción actual.	DOTS Workflows/19_Netcode Prediction & Rollback.md
IInputComponentData	Componente de input de alto nivel cuya gestión del búfer de comandos se genera automáticamente.	DOTS Workflows/20_Netcode Command Stream & Input.md
ICommandData	Tipo de flujo de comandos de bajo nivel en el que el mapeo de ticks y el acceso al búfer se gestionan a mano.	DOTS Workflows/20_Netcode Command Stream & Input.md
InputEvent	Marcador de input puntual para eventos como saltar o disparar, que deben registrarse exactamente una vez en el tick objetivo.	DOTS Workflows/20_Netcode Command Stream & Input.md
IRpcCommand	Tipo de mensaje RPC puntual y confiable.	DOTS Workflows/21_Netcode RPC.md
IApprovalRpcCommand	Tipo de RPC especial que se permite durante la aprobación de la conexión.	DOTS Workflows/21_Netcode RPC.md
Ghost Importance	Puntuación de prioridad del servidor que decide qué Ghosts caben primero en el presupuesto de un snapshot.	DOTS Workflows/22_Netcode Ghost Optimization · Importance · Relevancy.md
Ghost Relevancy	Filtrado por conexión que decide si un Ghost debe replicarse a un cliente concreto.	DOTS Workflows/22_Netcode Ghost Optimization · Importance · Relevancy.md
Lag Compensation	Consulta de mundos de colisión históricos en el servidor para validar las acciones del cliente en el instante que este percibió.	DOTS Workflows/23_Netcode Physics Integration & Lag Compensation.md

8. Legacy / obsoleto (NO debe aparecer en código nuevo)

Término	Estado en 6.5	Objetivo de migración
Entities.Foreach	Obsoleto desde la 1.4; aún compila, pero con advertencias; su eliminación está prevista para una futura versión mayor.	SystemAPI.Query<>() o IJobEntity — Migration/04_foreach → IJobEntity.md
IAAspect	Obsoleto desde la 1.4; aún compila, pero con advertencias; su eliminación está prevista para una futura versión mayor.	Parámetros de componente directos (RefRW<T> / RefRO<T>) — Migration/05_IAspect Removal.md
ComponentLookup.GetRefRWOptional	Obsoleto.	TryGetRefRW — ver Migration/01_Entities 1.x → 6.5 Overview.md

9. Conceptos de apoyo

Término	Qué es
Burst	El compilador nativo de Unity para jobs y structs marcados con [BurstCompile]. Genera código SIMD nativo.
Change version	Contador de 32 bits que se incrementa cuando un sistema escribe en la columna de un componente de un chunk. Permite que SetChangedVersionFilter omita los chunks que no han cambiado.
SoA (Structure of Arrays)	Disposición de almacenamiento en la que cada campo forma su propio array; favorece la caché en la iteración escalar. Los chunks usan SoA.
RefRW<T> / RefRO<T>	Tipos wrapper para acceder a componentes en SystemAPI.Query con intención de lectura-escritura o de solo lectura.
EnabledRefRW<T> / EnabledRefRO<T>	Wrapper para leer o escribir el bit de activación de un componente habilitable sin provocar un cambio estructural.
TypeManager	El registro del runtime que asigna a cada componente un índice de tipo estable. Se reconstruye al arrancar; los proyectos grandes lo notan.
BlobAssetReference<T>	Puntero a un blob de datos grande y de solo lectura que se almacena fuera del chunk. Evita inflar la capacidad del chunk.